ENGLISH

Add up 3, 8 and 20, and show me the total.

PYTHON

```
print(sum([3, 8, 20]))
```

C

```
int n[] = {3, 8, 20}, total = 0; for (int i = 0; i < 3; i++) total += n[i];
```

ASSEMBLY

```
LOAD 9  ADD 10  ADD 11  STORE 12  PRINT 12  STOP
```

MACHINE CODE

```
109  210  211  312  412  000
```

BITS

```
01101101  11010010  11010011  100111000  110011100  00000000  00000011  00001000  00010100
```

VOLTAGE

A COURSE READER IN VIBE CODING · VOLUME ONE

Layers of Language

How software works,
from voltages to plain English

Dr Vipul Mathur

AILADYN AI

A COURSE READER IN VIBE CODING · VOLUME ONE

Layers of Language

How software works, from voltages to plain English

Dr Vipul Mathur

IIT Delhi · IIM Bangalore · IIM Calcutta
Ailadyn AI

Beta edition, October 2026. This edition is still being revised; corrections are welcome.

This book is the first of two readers for the course in vibe coding. It explains how software works and how it came to be written the way it is. The second volume, *Good Enough to Build*, is the practical guide to building software with AI agents.

© 2026 Vipul Mathur. The cover shows one small task, adding up three numbers, written at six of the layers this book describes, from plain English at the top to the voltages inside a chip at the bottom. The machine code and assembly are for the simple teaching machine used in Chapters 4 and 5. Set in Source Serif 4, Playfair Display, Libre Franklin and JetBrains Mono. Diagrams drawn for this book; picture credits appear at the end.

Contents

How to Read This Book 5

PART ONE · THE MACHINE UNDERNEATH

1 What Is a Program? 8

2 From Voltage to Bits 21

3 Gates and the Processor 36

PART TWO · CLIMBING THE LADDER

4 From Rewiring to Remembering 51

5 Assembly and the First Translators 65

6 Compilers 78

7 A Family Tree of Languages 92

8 The Operating System, Files and the Terminal 108

PART THREE · THE CONNECTED MACHINE

9 Networks, the Web and the Cloud 125

10 Anatomy of a Modern App 139

11 Programs That Learn 154

12 English Becomes Code 170

The Whole Ladder 185

A Timeline 186

Cast of Characters 190

Glossary 195
Further Reading 199
Picture Credits 206

BEFORE YOU BEGIN

How to Read This Book

THIS BOOK EXPLAINS HOW SOFTWARE WORKS, from the voltages inside a chip to the sentence of English you type to an AI agent. It is written for readers with no technical background, and it assumes no mathematics beyond arithmetic. Every technical word is explained the first time it appears, with a short definition in the margin; the glossary at the back collects them.

The chapters climb a ladder. Each one adds a layer of software and shows what that layer made easier, what it hid, and who built it. One small task, adding up a list of numbers and showing the total, appears again and again, written in the form each layer understands, so that you can watch the same instruction become shorter and more human as the book climbs. The small ladder at the end of every chapter keeps track of where we are, and the whole ladder appears at the back.

What you will find on the page

The main text tells the story and can be read on its own. Around it are a few kinds of supporting material.

- **A closer look** boxes give historical detail or a deeper explanation. You can skip them on a first reading.
- **The business view** boxes ask what an idea has meant for companies, markets and costs.
- **At the keyboard** boxes connect an idea to what you will do when you build software with an AI agent. They point to the second volume, *Good Enough to Build*.
- **Margin notes** define key terms and carry the captions for figures and photographs.
- **Code** appears in short, shaded listings. You are not expected to read code fluently: every listing is explained in the text around it, and every one has been run to check that it does what the book says.

Why the layers matter

You can build software with an AI agent without knowing any of this, in the same way that you can drive a car without knowing how its engine works. But every layer of software sometimes fails, and when it does, the detail it was hiding shows through. People who have a rough picture of what lies underneath ask better questions, notice sooner when something is wrong, and know where to look. The aim is not to make you a programmer. It is to make you a well-informed visitor to the city of software, who knows the layout of its streets.

PART ONE

The Machine Underneath

What a program is, and the machine that runs it: voltages that stand for bits, and bits combined by circuits into a processor that follows instructions.

A Jacquard loom with its chain of punched cards. Each card controls one pass of the shuttle.

CHAPTER ONE

What Is a Program?

Every piece of software is a list of instructions written in advance for a machine that has no judgement of its own. The idea is older than electricity in the home. It began with a silk loom, a calculating engine that was never finished, and a census.

EVERY APP ON YOUR PHONE is a program. So is the software that runs a bank's accounts, the system that books a railway ticket, and the chatbot that answers your questions. Underneath their differences they share one feature: each is a set of instructions, written in advance by people who are not present, for a machine that will carry them out exactly as written and will not stop to ask what was meant.

This book is about how those instructions are written. It follows the question from the bottom of the machine to the top: from the voltages inside a chip, through the numbers and codes that people once wrote by hand, through the programming languages that made the work easier, to the present, when a person can describe what they want in plain English and an AI system writes the code. Each of these is a layer, and each layer was built so that people could say more while writing less. The aim is not to turn you into a programmer. It is to give you a working picture of what lies underneath when you ask a machine to build something, and some respect for the work that every layer represents.

We begin before electronics, with the idea of a program itself. It came from three places that have little to do with one another: a weaving workshop in Lyon, a mathematician's study in London, and the office of the United States census.

Instructions written in advance

A cooking recipe is the most familiar set of instructions most people meet. It lists steps in order: boil the water, add the tea leaves, wait three minutes, add milk. It may repeat a step ("stir until the sugar dissolves") or make a choice ("if the tea is too strong, add more milk"). And it is written by someone who will not be in the kitchen when it is followed.

A computer program has the same three ingredients. Steps are carried out in order, one after another; some steps are repeated; and at certain points the next step depends on a test. Programmers call these *sequence*, *repetition* and *selection*, and almost every program ever written is built from them. A precise, step-by-step method for solving a problem is called an **algorithm**; a **program** is an algorithm written down in a form that a particular machine can carry out.

algorithm A precise, step-by-step method for solving a problem, written so that it can be followed without judgement.

program A set of instructions, written in advance, that tells a machine exactly what to do.

The difference between a recipe and a program lies in who follows it. A cook brings judgement. If a recipe says "a pinch of salt" the cook decides how much that is, and if it forgets to say "turn off the gas" the cook does it anyway. A machine brings no judgement at all. It does what

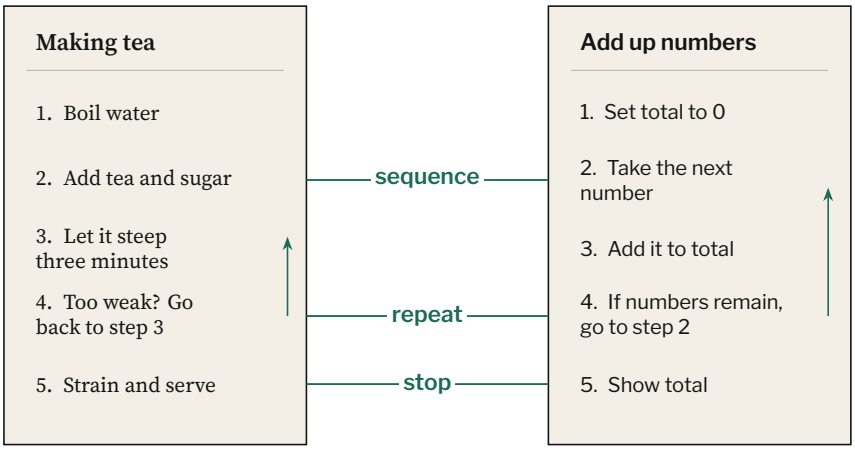


Figure 1.1 A recipe and a program share the same structure: steps in sequence, a step that repeats, and a point at which the work stops. The program on the right adds up a list of numbers, a task that recurs throughout this book.

the instructions say, in the order they say it, and nothing else. Every step that a person would take for granted has to be written down.

Throughout this book we will use one small task to show how instructions have changed. The task is to add up a list of numbers and show the total. Written as an algorithm in English, it might read like this: set the total to zero; take the next number from the list and add it to the total; if any numbers remain, go back and take the next one; when none remain, show the total. In later chapters the same task will appear as voltages, as raw numbers, as assembly code, and in several programming languages. It never changes. Only the language in which it is written does.

A loom that read cards

The first machine to be controlled by a changeable set of instructions was built to weave silk. In the early nineteenth century Lyon was the centre of the French silk industry, and its most valuable products were cloths woven with elaborate patterns of flowers, leaves and figures. Weaving such a pattern meant lifting a different combination of the lengthwise threads for every pass of the shuttle. On the old draw looms this was done by hand: an assistant, often a child, sat above the loom and pulled up the right threads at each pass, following the weaver's calls. The work was slow and the mistakes were frequent.

Joseph Marie Jacquard, a Lyon weaver, built on earlier French experiments to produce a loom that did the lifting itself. His device was first shown in Paris in 1801 and perfected around 1804. It read a chain of stiff cards, each punched with a row of holes. Above the loom sat a set of

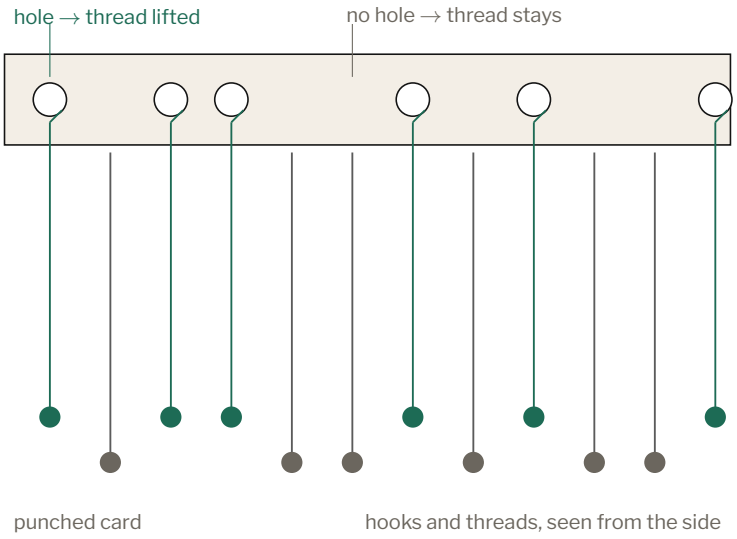


Figure 1.2 How a Jacquard card controls the loom. Where the card has a hole, a hook passes through and lifts its thread; where there is no hole, the thread stays down. One card governs one pass of the shuttle.

hooks, one for each group of threads. When a card was pressed against the hooks, those that met a hole passed through and lifted their threads; those that met solid card stayed down. Each card controlled one pass of the shuttle, and the chain of cards, laced together, described the whole pattern.

The important point was not the speed, though it was considerable. It was that the pattern no longer lived in the loom or in the weaver's memory. It lived in the cards. To weave a different design, a workshop did not need a different machine; it needed a different chain of cards. The cards could be stored, copied, sold and reused. A single loom could produce any pattern for which someone had punched the cards, and the people who designed patterns and punched cards became a trade of their own.

Punched cards of this kind had a long life. The most famous demonstration of what Jacquard's loom could do was a portrait of Jacquard himself, woven in silk in 1839 and so finely detailed that it was often mistaken for an engraving. It is said to have needed about 24,000 cards. Charles Babbage, the English mathematician, owned a copy and liked to ask visitors how they thought it had been made.

Punched cards Stiff cards in which the presence or absence of holes at set positions records information or instructions.

Babbage's engines

Babbage was interested in a different kind of pattern. In the early nineteenth century the navigation, engineering and insurance of the industrial world depended on printed tables of numbers: logarithms, tide

times, interest rates, the positions of stars. They were calculated by teams of clerks, then copied and typeset by hand, and they were full of errors. A mistake in a navigational table could wreck a ship.

In 1822 Babbage proposed a machine that would calculate such tables mechanically and print the results directly, removing the human errors at both ends. With money from the British government he began building what he called the Difference Engine, a large assembly of brass gear wheels that could carry out one particular method of calculation. The project ran into disputes with his engineer and rising costs, and work stopped in 1833 with only a part of the machine completed. (In 1991 the Science Museum in London built a later design of Babbage's to his drawings, and it worked.)

By then Babbage had a more ambitious idea. The Difference Engine could do one kind of calculation. From 1834 he designed what he called the Analytical Engine, which could in principle carry out any calculation at all, according to instructions fed into it. He borrowed Jacquard's cards to supply those instructions. The design had a part he called the *mill*, where arithmetic was done, and a part he called the *store*, which held the numbers being worked on, about a thousand of them. A century later the same division would reappear in electronic computers as the processor and the memory.

The Analytical Engine was never built. Babbage refined its design for the rest of his life, but without the government's backing, and with the precision engineering of the time stretched to its limits, it remained on paper. What survives are thousands of pages of drawings and notes, and a trial portion of the mill that was still being assembled when he died in 1871.

Lovelace's notes

The clearest account of what the Analytical Engine could do was written not by Babbage but by Ada Lovelace. Born in 1815, the daughter of the poet Lord Byron, she had been educated in mathematics at her mother's insistence and had known Babbage since she was seventeen. In 1842 an Italian engineer, Luigi Menabrea, published an article in French describing the engine, based on lectures Babbage had given in Turin. Lovelace translated it into English and, at Babbage's suggestion, added her own notes. When the translation appeared in 1843, signed only with her initials, her notes were about three times as long as the article itself.

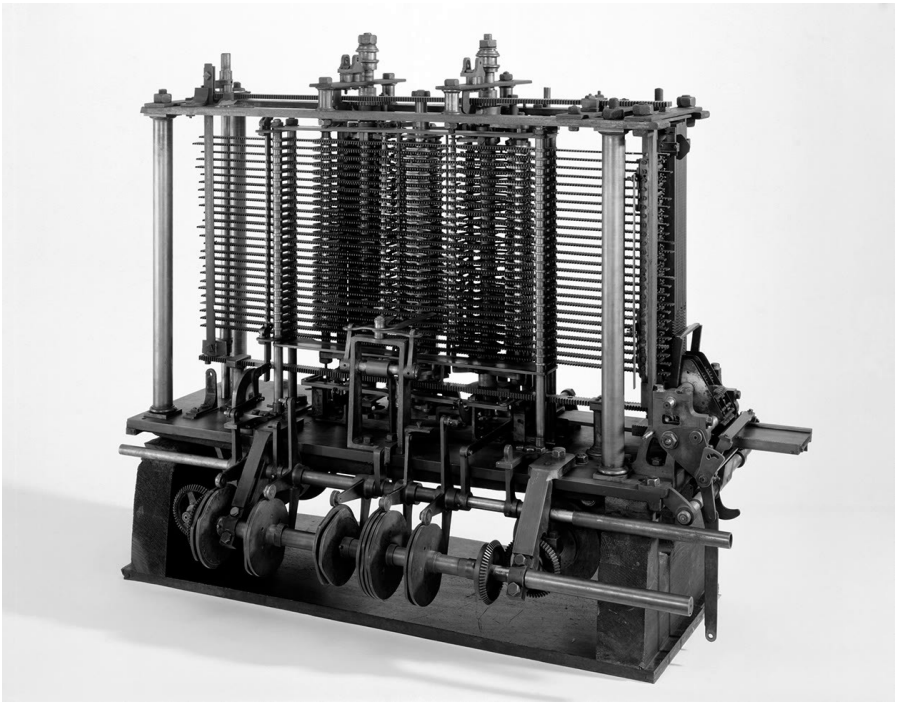


Figure 1.3 A trial portion of the mill of Babbage's Analytical Engine, still being assembled when he died in 1871. The full machine was never built. Science Museum, London.

The last of the notes, Note G, set out in a detailed table the sequence of operations by which the engine could calculate a series of values known as Bernoulli numbers. The table showed which numbers would be moved from the store to the mill at each step, what would be done to them, and where the results would go. It is often described as the first computer program to be published, and Lovelace is often described as the first programmer. Historians debate how much of the work was hers and how much Babbage's, who had sketched simpler sequences of operations for the engine some years earlier. What is not in doubt is that the notes are the first extended explanation of what such a machine might be used for, and that their author understood it unusually well.

Two of Lovelace's observations matter for this book. The first was that the engine was not limited to arithmetic. It worked on symbols, and numbers were only one kind of symbol. If other things, such as the relations between musical notes, could be expressed in symbols, then the engine might work on those too, and might, she suggested, compose elaborate pieces of music. A machine that manipulates symbols according to instructions can be put to any task whose content can be written down as symbols. That is the idea on which all general-purpose computing rests.

We may say most aptly, that the Analytical Engine weaves algebraical patterns just as the Jacquard-loom weaves flowers and leaves.

ADA LOVELACE, NOTE A, 1843

The second observation was a limit. "The Analytical Engine has no pretensions whatever to originate anything," she wrote. "It can do whatever we know how to order it to perform." The engine would follow its instructions faithfully, but the instructions, and therefore the ideas, would have to come from people. For most of the history this book describes, she was plainly right. Whether she is still right, now that a machine can write a program from a sentence of English, is a question we return to in the last chapter.

A CLOSER LOOK

The slip in Note G

Modern readers who have worked through Lovelace's table of operations have found a small error in the printed table, which may have been introduced in printing rather than in her working: in one step, two quantities are written the wrong way round, so that a division would be carried out upside down. Had the engine been built and the table punched into cards as printed, the calculation would have produced wrong answers from that step onwards.

The slip is not a mark against Lovelace. Errors of this kind are made by every programmer, every day. But it illustrates something that holds for all programs. A person reading the table would probably notice that the step makes no sense and quietly correct it. The machine would not. It would carry out the instruction as written, and the error would spread through every later step. Much of the history of software, from the first assemblers to today's automated tests, is a history of tools for catching such slips before the machine acts on them.

Cards that counted a nation

Babbage's engine stayed on paper, but punched cards found a practical use in the 1880s, in the United States census. The American constitution requires a count of the population every ten years, and by the late nineteenth century the count had become hard to manage. The 1880 census

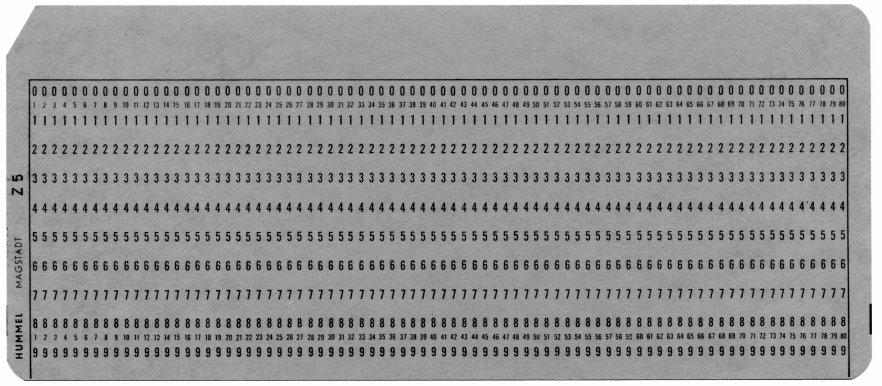


Figure 1.5 A punched card of the kind IBM standardised in 1928, with 80 columns. Cards like this carried data and programs into computers until the 1970s.

took most of a decade to tabulate by hand; there was a real prospect that the 1890 count would not be finished before the 1900 one was due.

Herman Hollerith, an engineer who had worked on the 1880 census, proposed recording each person's details as holes punched in a card: one position for male or female, others for age group, place of birth, occupation and so on. He later said the idea came partly from watching railway conductors punch tickets to record a passenger's description. The cards were then read by an electrical machine. Each card was placed in a press, and a set of spring-loaded pins came down on it; where a pin found a hole, it dipped into a small cup of mercury beneath, completed an electrical circuit, and advanced a counter on the dial in front of the operator.

With Hollerith's machines the 1890 census was tabulated far faster than the one before, and the headline population count was announced within weeks. The machines were soon used by railways, insurers and other governments. Hollerith's company, founded in 1896, merged with others in 1911, and in 1924 the combined firm took the name International Business Machines, or IBM. Punched cards remained the main way of feeding data and programs into computers until the 1970s, and many people who are now grandparents remember submitting university assignments as boxes of them.

THE BUSINESS VIEW

When software became a product

For most of their history, the instructions that ran machines were not sold separately. Jacquard's cards were a partial exception: pattern design and card punching became a specialist trade. In computing, though, software was long treated as part of the hardware. Into the 1960s, IBM supplied programs free to customers who leased its computers, as part of the service.

That changed in June 1969, when IBM announced that it would charge separately for much of its software and services, a decision known as unbundling. It was made partly under pressure from an American antitrust case, but its effect was to make software a product with a price. Independent software companies, which until then had competed against programs that IBM gave away, could now sell software for IBM machines without being undercut. Many historians date the software industry from that decision. Every software business since, from Microsoft to the app on your phone, sells the thing Jacquard's cards first made separable: the instructions, apart from the machine.

One machine, many programs

The loom, the engine and the tabulator share one idea: the instructions are separate from the machine. Change the cards and the same loom weaves a different cloth; punch different cards and the same tabulator counts a different population. The machine supplies the ability to act. The program decides what the action is.

But each of these machines was limited in what its instructions could ask for. Jacquard's cards could only choose which threads to lift. Hollerith's machine could only count holes, and rewiring it to count different things meant changing its connections by hand. Babbage's engine was meant to be general, able to carry out any calculation it was given, but it was never built. What was missing was a machine that could follow any set of instructions at all, and do so fast enough to be useful.

The theory of such a machine came in 1936, from the English mathematician Alan Turing. In a paper on a question in mathematical logic, Turing described an imaginary device that read and wrote symbols on a long tape, following a table of simple rules. He then showed that one such device, given the right table, could imitate any other. A **general-purpose computer** is the practical form of that idea: a single machine that becomes a calculator, a typewriter, a cinema or a map depending on the program it runs. It is the reason the same laptop can run a spreadsheet in the morning and a video call in the afternoon.

general-purpose computer A machine that can carry out any calculation, given the right program, rather than one fixed task.

Building such a machine took the electrical engineering of the 1940s, and the next three chapters describe how it works. Before we get there, it helps to see why writing instructions for it turned out to be so hard.

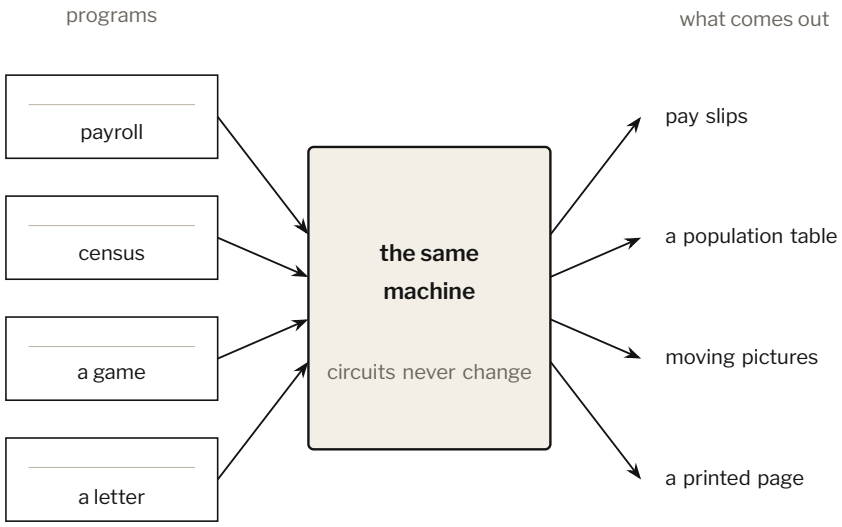


Figure 1.6 The idea that the loom, the engine and the tabulator have in common, and that every computer since has relied on: one machine, unchanged, does different work depending on the program it is given.

Why programs are hard to write

Consider again the task of adding up a list of numbers. Written in English, the steps seemed complete. But a machine following them would have questions that a person would not think to ask. What should happen if the list is empty: should it show zero, or show nothing, or complain? What if one of the items is not a number at all, but a word typed by mistake? Show the total where: on a screen, on paper, in a file? With how many decimal places? People answer such questions without noticing, from context and common sense. A program has to answer every one of them in advance, because the machine has neither.

That is the central difficulty of programming, and it has not changed since Lovelace's table. The hard part is rarely the arithmetic. It is the gap between what a person means and what they have actually said. Ordinary language is full of gaps that listeners fill without effort. "Book a table for two at eight" leaves out the day, the restaurant and whether eight means morning or evening, and no human waiter would be confused. A machine following instructions literally has nothing with which to fill the gaps, so the instructions must leave none.

Each layer in this book can be read as an attempt to make that gap easier to close. Better notations let people write instructions that are closer to how they think, and leave the translation into the machine's terms to other programs. The newest layer, described in Chapter 12, goes furthest: an AI system takes instructions in ordinary English and fills the gaps itself, with guesses based on what similar requests have usually meant.

That makes it much easier to get something working. It also means that the gaps are still there. They are filled by the machine's guesses instead of the writer's decisions, and someone still has to check whether the guesses were right.

AT THE KEYBOARD

The same gap, today

When you ask an AI coding agent to "build a website for my bakery", it will produce one. In doing so it will make dozens of decisions you did not mention: the colours, the pages, whether there is an order form, where the orders go, what happens if someone orders a cake for yesterday. Some of its choices will suit you and some will not, and you will only find out which by looking.

The skill that Volume 2 teaches is largely the skill of closing this gap on purpose: deciding in advance the things that matter, saying them clearly, and checking the result. Chapter 4 of Volume 2, on turning an idea into a specification, is the practical version of this section.

The ladder of abstraction

A modern computer is far too complicated for any one person to understand all at once. Its chip contains billions of switches; its operating system runs to tens of millions of lines of instructions; the programs on it rely on other programs written by thousands of people. It is possible to work with all of this only because it is built in layers, and each layer hides the detail of the one beneath. Someone writing a program in a modern language does not think about switches, any more than someone driving a car thinks about the timing of the fuel injection. The pedal is a simpler way of asking for the same result.

Hiding detail in this way is called **abstraction**. Every layer of software is an abstraction over the layer below: it offers a simpler way of saying what you want, and it translates that into the more detailed terms the lower layer understands. The translations stack up. An instruction typed by a person is translated by one program, then another, then another, until at the bottom it becomes a pattern of high and low voltages in a chip.

abstraction Hiding the details of how something works behind a simpler way of using it, so that people can work at a higher level.

This book climbs that stack from the bottom. Chapters 2 and 3 describe the machine itself: how voltages stand for numbers, and how simple circuits combine them into a processor that follows instructions. Chapters

4 to 8 describe how people learned to give the processor its instructions, first as raw numbers, then in shorthand, then in programming languages, and how the operating system came to organise it all. Chapters 9 to 12 describe how computers were joined together, how modern apps are assembled from many parts, how a different kind of program came to be learned from examples instead of written by hand, and how plain English became the newest layer on top. At the end of each chapter a small diagram of a ladder shows the rungs added so far.

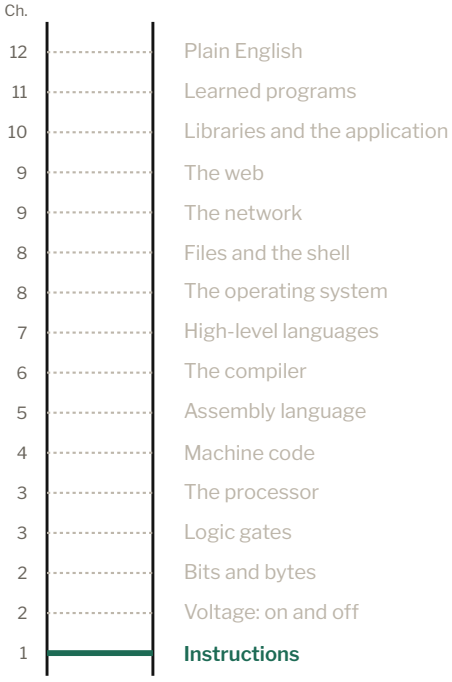
The course this book belongs to takes a particular view of what you need from all this. You are visiting the city of software, not moving there. A visitor does not need to know how every building was constructed, but a visitor who knows the layout of the streets gets lost less often, and knows roughly where to look when something goes wrong. The aim of the chapters that follow is to give you that layout.

Where this leaves us

We now have the idea at the heart of all software: a program is a set of instructions, written in advance, separate from the machine that carries them out. Jacquard's loom showed that instructions could be stored on cards and swapped. Babbage and Lovelace saw that a machine following such instructions could, in principle, do any calculation, and could work on any symbols at all. Hollerith showed that machines reading cards could do useful work on a national scale.

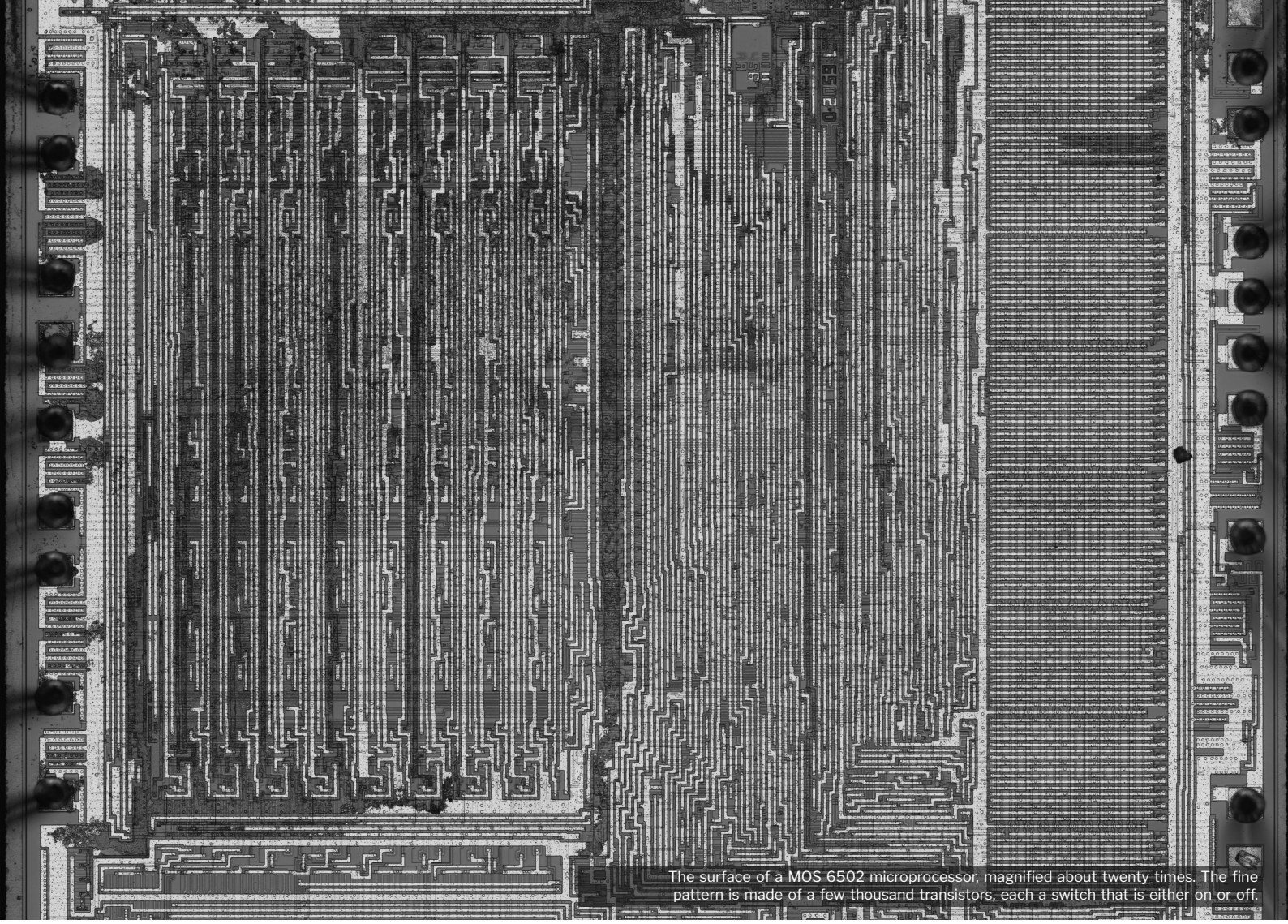
What none of them had was a machine that could hold any program and carry it out quickly. The loom's cards could only lift threads, the tabulator could only count, and the Analytical Engine's gears were too slow and too difficult to build. The material for such a machine turned out to be electricity: switches that are either on or off, and that can be flipped millions of times a second.

That is where the climb begins. Chapter 2 starts at the lowest layer of every computer, where there are no letters, no numbers and no instructions, only voltages that are high or low.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.



The surface of a MOS 6502 microprocessor, magnified about twenty times. The fine pattern is made of a few thousand transistors, each a switch that is either on or off.

CHAPTER TWO

From Voltage to Bits

Inside a computer there are no letters, no pictures and no numbers. There are only switches, each of which is either on or off. This chapter explains how that is enough to hold a word, a photograph or a song.

OPEN ANY COMPUTER, from a laptop to the phone in your pocket, and you will not find a letter of the alphabet anywhere inside it. Nor will you find a photograph, a song, or even a number in the sense that you would write one on paper. You will find a circuit board, some flat black packages soldered to it, and inside the largest of them a slice of silicon smaller than a postage stamp. On that slice are billions of tiny switches, connected by wires far finer than a hair.

Each switch does one thing: it lets electric current through, or it does not. Everything a computer appears to hold, from the text of an email to a video call, is stored and moved about as the positions of these switches. This chapter explains how switches can stand for numbers, how numbers can stand for letters, pictures and sound, and why the whole arrangement rests on a decision to use only two states. It is the bottom of the ladder. Nothing here is a program yet, but every program that runs on a computer, including the ones an AI agent will write for you, ends up as patterns of on and off at this level.

Only high and low

Electricity in a circuit can be described by its *voltage*, a measure of the electrical pressure that pushes current along a wire. A torch battery supplies about one and a half volts; a household socket in India supplies about 230. Inside a modern processor the working voltage is around one volt.

A chip does not care about the exact voltage on each of its wires. It sorts every voltage into one of two groups. A wire whose voltage is near the top of the range is *high*; one whose voltage is near zero is *low*. That is all a chip can see. It has no way of noticing that a wire is at 0.93 volts rather than 0.97. It only asks: high or low?

The switches that set those voltages are called **transistors**. A transistor has three connections. Current can flow between two of them, but only when a voltage is applied to the third. The important feature is that the thing doing the switching is itself a voltage. The output of one transistor can therefore control the input of another, and that one the next, with no person or moving part in between. A chip is a network of switches that set one another.

transistors Tiny electronic switches with no moving parts. A small voltage on one connection turns the flow of current between the other two on or off.

The two states go by several names: high and low, on and off, true and false. This book uses the names from arithmetic: `1` for high and `0` for low.

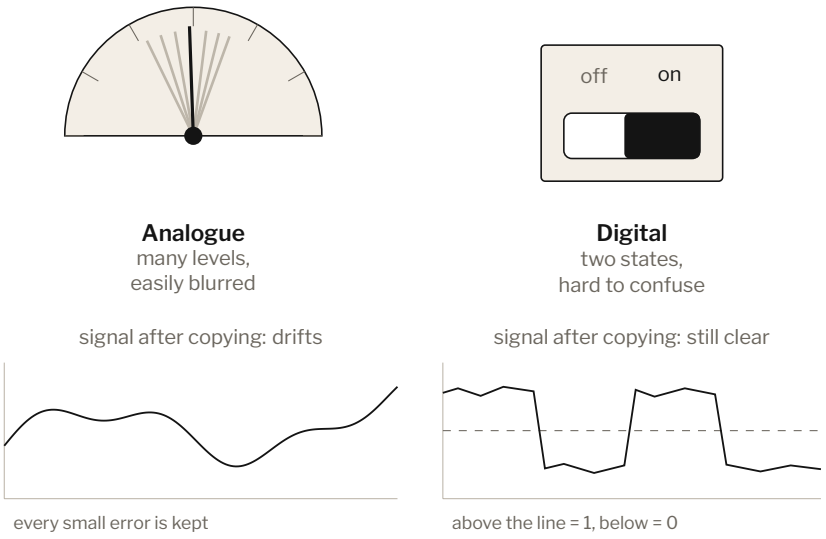


Figure 2.1 A dial can be set to many positions, but a small error in setting or reading it is never noticed and never removed. A switch has two positions, so a signal that has been disturbed in transit can still be read correctly, and copied again without loss.

Why only two states?

A wire can carry any voltage between zero and the maximum, so why not use ten levels and let each wire stand for a digit from 0 to 9? Several early machines, ENIAC among them, did their arithmetic in decimal. The reason two states won is noise.

No electrical signal is perfectly clean. Wires pick up interference from their neighbours, components heat up and drift, and no two transistors are made exactly alike. If a wire must distinguish ten levels packed into a single volt, a disturbance of a few hundredths of a volt can turn a 6 into a 7, and nobody would know. If it must distinguish only two levels, far apart, the same disturbance makes no difference. A voltage that should be high but has sagged a little is still much nearer high than low, so it is read correctly.

Think of the difference between a dimmer and a light switch. Ask a friend to set a dimmer to the position you had last night and they will get it roughly right, but probably not exactly. A light switch has two positions, and nobody gets it wrong. Machines that represent information by smoothly varying quantities, like the dimmer, are called *analogue*. Machines that use a small number of distinct states, like the switch, are called **digital**.

digital Representing information by a small number of distinct states (in computers, two) instead of a smoothly varying quantity.

There is a second advantage. Because each reading is rounded to high or low, a signal is cleaned up as it passes through each switch: a slightly weak high goes in, a full-strength high comes out. Errors do not accumulate. A copy of a copy of a digital file is identical to the original, which is

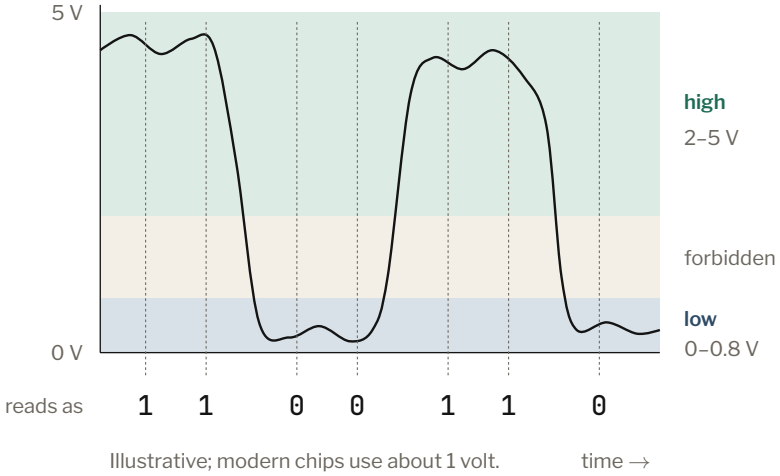


Figure 2.2 A real voltage is never perfectly steady. The chip reads anything in the upper band as 1 and anything in the lower band as 0, so small wobbles do not change the result. The middle band is a gap that a working circuit passes through quickly but never rests in.

why a song can be sent around the world a million times and still sound the same, while a cassette copied from a cassette loses a little at every step.

Engineers turn this into precise rules. In TTL, a family of chips that was standard from the 1960s onwards, the supply was five volts. Any input between 2 and 5 volts counted as high, anything between 0 and 0.8 volts counted as low, and the space between was a gap that a working circuit should never rest in. Modern processors work at around one volt, to save power and heat, but the principle is the same.

From relays to billions of transistors

Computing with switches is older than the transistor. The first switches used for calculation were *relays*: small electromagnets that pull a metal contact open or shut, as in the telephone exchanges of the time. In the late 1930s and early 1940s Konrad Zuse in Berlin, George Stibitz at Bell Telephone Laboratories in New York and Howard Aiken at Harvard all built calculating machines from relays. They worked, but a relay has a moving part, and it can click open and shut at best a few hundred times a second.

The next switch had no moving parts. A *vacuum tube*, a glass bulb with the air removed, could pass or block a current under the control of a voltage, and it switched roughly a thousand times faster than a relay. ENIAC, the American machine completed in 1945 and described in Chapter 4, used about 18,000 of them. But tubes were bulky, ran hot and burned out

like light bulbs, and a machine with thousands of them spent much of its time being repaired.

In December 1947, at Bell Telephone Laboratories in New Jersey, John Bardeen and Walter Brattain, working in a group led by William Shockley, demonstrated a switch made from a small piece of solid germanium, with no vacuum and no glowing filament. It was named the transistor, and the three men shared the Nobel Prize in Physics in 1956. Early transistors were made one at a time and wired together by hand, but they were small, cool and reliable, and through the 1950s they replaced tubes in radios, hearing aids and computers.

The step that made modern computers possible was to stop wiring transistors together at all. In 1958 Jack Kilby at Texas Instruments built several components on one piece of germanium, and in 1959 Robert Noyce at Fairchild Semiconductor in California described a way of making transistors and the connections between them together on a single flat slice of silicon. The result is called an **integrated circuit**, or chip. Because a chip is made by printing patterns onto silicon, rather as a photograph is printed, making the transistors smaller lets more of them fit on the same chip at little extra cost.

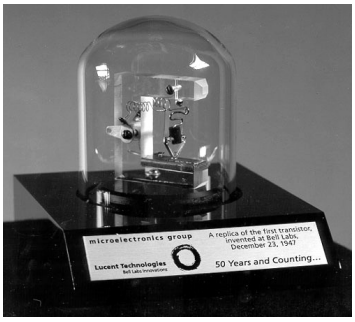


Figure 2.3 A replica of the first transistor, demonstrated at Bell Telephone Laboratories in December 1947. The working part is the small wedge pressed against a block of germanium.

integrated circuit Many transistors, and the wires between them, made together on one small slice of silicon. Also called a chip.

In 1965 Gordon Moore, then at Fairchild, wrote a short article for the magazine *Electronics* observing that the number of components that could economically be put on one chip had been doubling every year, and predicting that it would go on doing so for at least a decade. In 1975 he revised the rate to a doubling every two years. Moore's law, as the prediction came to be called, is not a law of nature. It became a target that the industry planned around, and for about half a century it held. Intel's first microprocessor, in 1971, had 2,300 transistors. Apple's M4 chip of 2024, used in tablets and laptops, has 28 billion.

None of this changes what a transistor does. A relay, a glass tube and a speck of silicon all do the same job: on or off. What changed was speed, size and price. A modern chip can flip its switches billions of times a second, which is why a design built from nothing but on and off is fast enough to play a film.

Counting with two digits

With only two states available, the machine needs a way to write numbers using only 0 and 1. The method is the one we already use for ordinary numbers, with fewer digits.

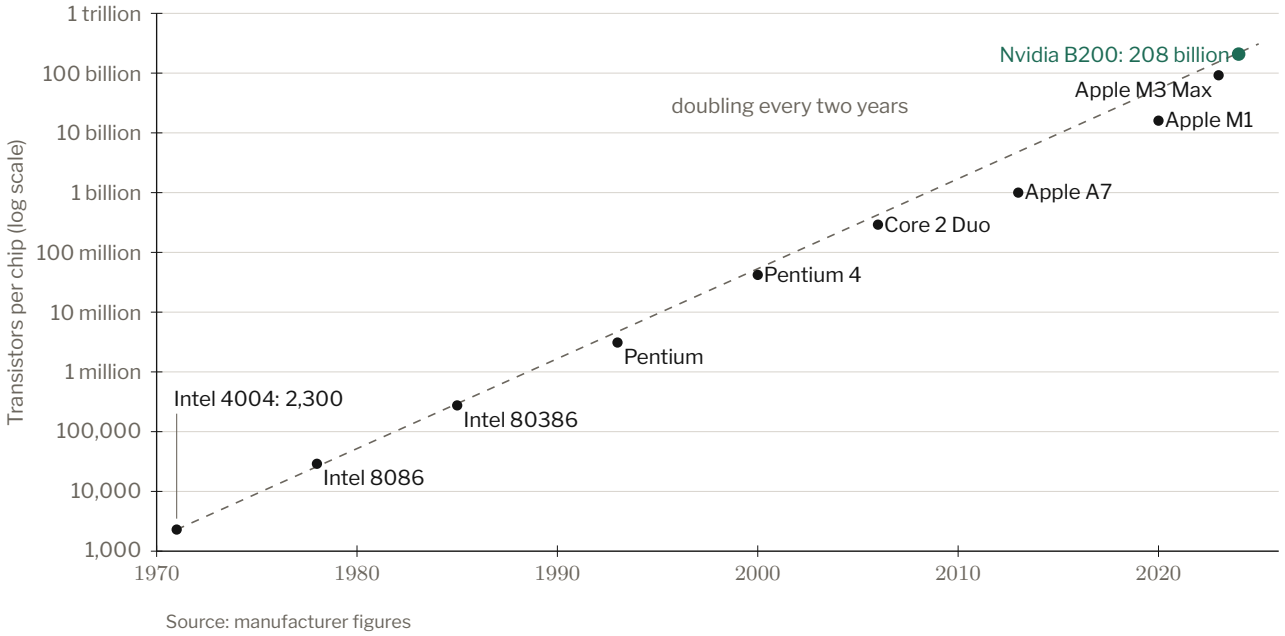


Figure 2.4 Transistors on a single chip, 1971 to 2024, on a scale where each line up is ten times the one below, so that steady doubling appears as a straight line. The industry kept close to Gordon Moore's revised prediction, a doubling about every two years, for half a century.

A car's mileage counter, the odometer, shows how decimal counting works. Each wheel turns from 0 to 9; when one wheel rolls over from 9 back to 0, it nudges the wheel to its left forward by one, so 9 becomes 10. Each column is worth ten times the column to its right: ones, tens, hundreds. Now imagine an odometer whose wheels carry only two digits, 0 and 1. Counting starts at 0, then 1. At the next step the right-hand wheel has nowhere to go, so it rolls back to 0 and nudges its neighbour: the reading is `10`. Then `11`; then both wheels roll over and nudge a third, giving `100`. Here each column is worth twice the one to its right: ones, twos, fours, eights. The reading `101` means one four, no twos and one one, which is five. This way of writing numbers is called **binary**.

binary A way of writing numbers using only two digits, 0 and 1, in which each column is worth twice the column to its right.

Binary numbers are long: 28 needs five binary digits, `11100`, where decimal needs two. But length costs a machine very little, because each extra digit is only one more switch. What matters is that every digit has one of two values, which is exactly what a switch can hold.

A single binary digit is called a **bit**. The word first appeared in print in 1948, in Claude Shannon's paper on the mathematics of communication, where he credited it to his colleague John Tukey. One bit answers a single yes-or-no question. Two bits give four possible patterns (`00`, `01`, `10`, `11`), three give eight, and each extra bit doubles the count again.

bit A single binary digit, 0 or 1: the smallest unit of information a computer stores.

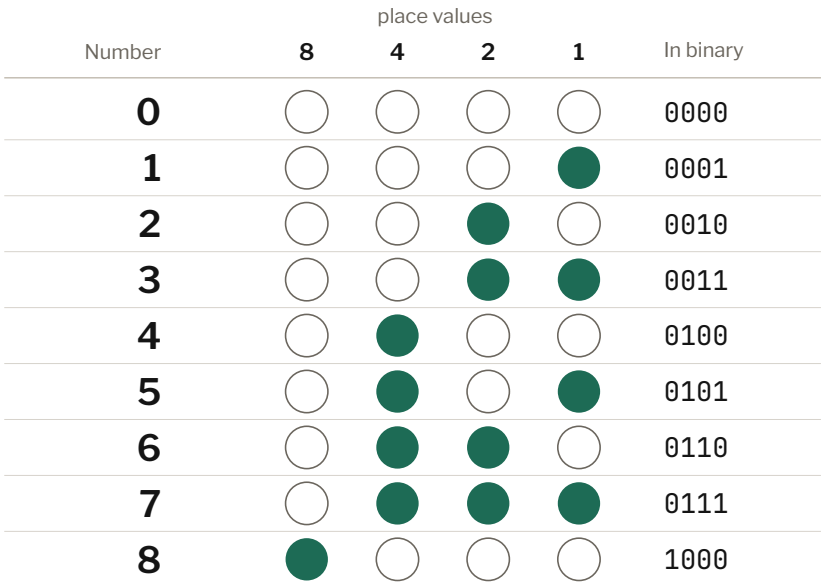


Figure 2.5 Counting from zero to eight in binary. Each column is worth twice the one to its right. When a column is full it goes back to zero and carries one to the next column, as a car's odometer rolls over from 9 to 10.

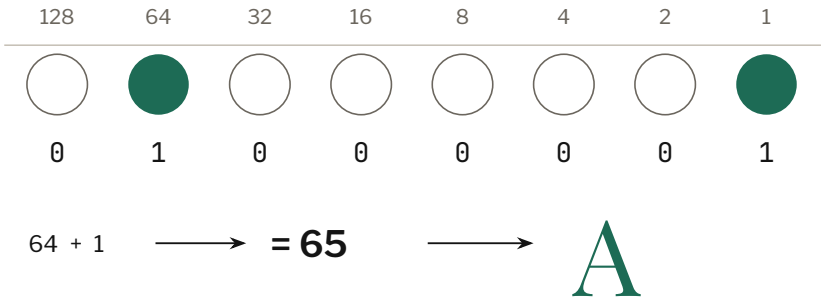
The running example of this book, adding up a list of numbers, can already be written at this level, though not yet carried out. Suppose the list is 4, 9 and 15. Inside the machine those are three groups of switches: `00000100`, `00001001` and `00001111`. Their total, 28, would be `00011100`. Nothing at this level knows that the patterns form a list, or that they are meant to be added. They are switches in particular positions.

A CLOSER LOOK

Leibniz, Boole and the arithmetic of two

Binary arithmetic is far older than the computer. In 1703 the German mathematician and philosopher Gottfried Wilhelm Leibniz published a paper with the Royal Academy of Sciences in Paris, "Explanation of binary arithmetic, which uses only the characters 1 and 0". A Jesuit missionary in China, Joachim Bouvet, had shown him that the hexagrams of the ancient Chinese *I Ching*, figures of broken and unbroken lines, could be read as binary numbers, and Leibniz discussed them in the paper. He was not the first European to count in twos (the English mathematician Thomas Harriot had done so in unpublished notes around 1600), but his paper made the idea widely known.

A century and a half later George Boole, a largely self-taught mathematician who had become a professor at Queen's College, Cork, came at the number two from another direction. His book *An Investigation of the Laws of Thought*, published in London in 1854, treated logic as a kind of algebra in which every statement has one of two values: true,



The same eight switches could mean 65, the letter A, or a shade of grey — it depends on how they are read.

Figure 2.6 Eight switches make one byte. Reading each column as a value (128, 64, 32 and so on down to 1) and adding up the columns that are on gives a number: here 64 plus 1, or 65. Under the code described in the next section, 65 stands for the letter A.

written 1, or false, written 0. Reasoning such as "if this and that are both true" could then be worked out with symbols, like a sum.

Neither man had any electrical machine in mind. Their work became important only when engineers needed a mathematics for circuits that were on or off, and found that it already existed. Chapter 3 describes how that connection was made.

Bits, bytes and how many

Single bits are too small to be convenient, so computers handle them in groups. The standard group is eight bits, called a **byte**. Eight bits can be arranged in 256 different patterns, from `00000000` to `11111111`, so one byte can hold any whole number from 0 to 255.

byte A group of eight bits. It can hold any of 256 patterns: a number from 0 to 255, or one letter of English text.

The word was coined in June 1956 by Werner Buchholz, an engineer at IBM, while he was helping to design a large scientific computer called Stretch. He spelled it with a "y" on purpose, so that it could not be mistaken for "bit". At first a byte could be any convenient small group of bits; the eight-bit byte became the standard after IBM announced its widely copied System/360 range of computers in April 1964.

Large quantities of bytes are counted with the prefixes used for metres and grams. A *kilobyte* is a thousand bytes, a *megabyte* a million, a *gigabyte* a billion and a *terabyte* a thousand billion. (Some software counts a kilobyte as 1,024 bytes instead, for historical reasons, which is why a new drive can seem slightly smaller than its label.) A phone with 128 gigabytes of storage has room for about a trillion bits. For a sense of scale, a page of plain text takes about three kilobytes, a minute of music on a compact

disc about ten megabytes, and a two-hour film in high definition several gigabytes.

Letters as numbers

A byte holds a number. To hold a letter, people must agree on a code: a table that says which number stands for which letter. By the early 1960s computer makers each had their own tables, and a machine from one manufacturer could not read text written by another's.

In 1963 the American Standards Association published a common table, the American Standard Code for Information Interchange, known as **ASCII**. It used seven bits, giving 128 positions, one for each capital letter, digit and common punctuation mark, plus control signals for teleprinters such as "start a new line". Capital A was given 65, B was 66, and so on. Written in binary, A is `01000001`. The first version had no small letters; they were added in a revision of 1967, in which small a is 97.

ASCII An early standard table (1963) that gives each English letter, digit and punctuation mark a number from 0 to 127.

One consequence is worth noticing. The digit "7" in a piece of text is not stored as the number 7. It is stored as the code for the character "7", which in ASCII is 55. A program that reads figures from a file must convert those characters into numbers before it can add them up; a spreadsheet that refuses to total a column of "numbers" that are really text has fallen into exactly this gap.

ASCII was designed for American English. It had no room for accented letters, let alone for Greek, Arabic, Chinese or any Indian script. Many local codes were invented to fill the gap, including ISCII, a standard for Indian scripts issued in India in the late 1980s. But each used the same numbers for different letters, so a document written under one code turned into nonsense when read under another.

The answer was one table large enough for every script in the world. The **Unicode** Consortium was founded in January 1991 by engineers from several computer companies, and the first version of the standard appeared that October with about 7,100 characters. Unicode 18.0, published in September 2026, contains 172,808. They include Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada and Malayalam among many other scripts, the Indian rupee sign ₹ (added in 2010), and nearly 4,000 emoji. Each character has a number, called its code point, written in a form such as `U+0041` for capital A.

Unicode A single international table that gives every character in the world's writing systems, and symbols such as emoji, its own number.

So large a table cannot fit in one byte. The usual way of storing Unicode text, called UTF-8, uses between one and four bytes per character.

character	name	code point	bytes in UTF-8 (hex)	bytes
A	Latin capital A	U+0041	41	1
é	Latin e with acute	U+00E9	C3 A9	2
अ	Devanagari a	U+0905	E0 A4 85	3
த	Tamil ta	U+0BA4	E0 AE A4	3
₹	Indian rupee sign	U+20B9	E2 82 B9	3
grinning face	Emoji	U+1F600	F0 9F 98 80	4

Figure 2.7 Six characters, their Unicode code points and the bytes that store them in UTF-8 (written here in a short form called hexadecimal). English letters keep their one-byte ASCII codes; Devanagari, Tamil and the rupee sign take three bytes; an emoji takes four.

It was designed in 1992 by Ken Thompson and Rob Pike, and it has a convenient property: the 128 ASCII characters are stored exactly as before, one byte each, so old English text is already valid UTF-8. Other characters take more. An accented é takes two bytes, most Indian letters three, and an emoji four. UTF-8 is now used by the great majority of web pages.

The bytes can be seen directly. On a Mac or Linux computer, a short command typed into the terminal (a window for typing commands, described in Chapter 8) prints the bits that store a piece of text. Here it is asked for the letter A, then é, then the euro sign:

```
Terminal

$ printf 'A' | xxd -b
00000000: 01000001                                 A
$ printf 'é' | xxd -b
00000000: 11000011 10101001                        ..
$ printf '€' | xxd -b
00000000: 11100010 10000010 10101100               ...
```

Each line beginning with `\$` is typed by a person; the line under it is the computer's reply. The column of zeros on the left is a position count and can be ignored. Then come the bytes: one for A, matching the pattern above; two for é; three for the euro sign. On the right the program repeats the text, but it can show only plain English characters there, so it prints a dot for each byte it cannot display.

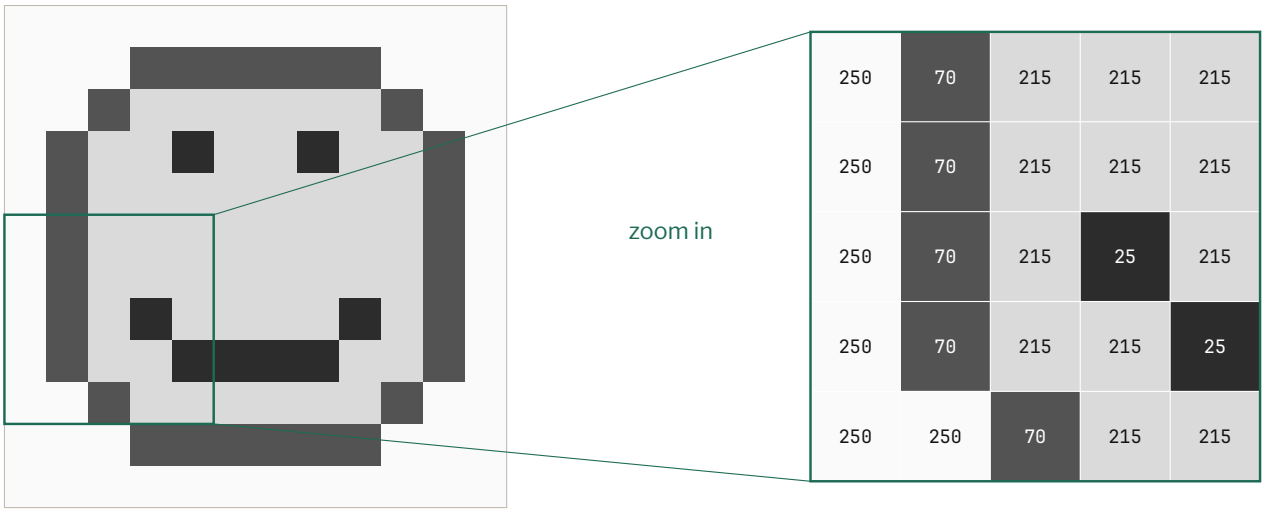


Figure 2.8 A small picture and the numbers that store it. Every square in the grid holds one number for its brightness, from 0 for black to 255 for white. A photograph works the same way, with millions of squares and three numbers per square for colour.

Pictures and sound

Once letters can be numbers, so can anything else that can be measured. A picture is turned into numbers by dividing it into a grid of tiny squares, each of a single colour, and recording the colour of each square. The squares are called **pixels**. In a greyscale image each pixel can be stored as one byte: 0 for black, 255 for white, and the numbers in between for shades of grey.

pixels The tiny squares, each a single colour, that make up a digital image. Short for "picture elements".

Colour needs three numbers per pixel. Screens make every colour by mixing red, green and blue light, so a colour image stores, for each pixel, how much of each to use, usually one byte apiece. Three bytes allow $256 \times 256 \times 256$, about 16.7 million, colours. A phone camera taking a 12-megapixel photograph records 12 million pixels with three numbers each: stored in full, 36 million bytes for one picture.

Sound is turned into numbers by a different kind of measurement. What we hear is a rapid rise and fall in air pressure. A microphone turns it into a voltage that rises and falls in the same pattern, and a circuit measures that voltage many times a second, recording each measurement, or *sample*, as a number. Played back, the numbers become a voltage again, which moves a loudspeaker, which moves the air. The audio compact disc, agreed by Sony and Philips in 1980 and on sale from 1982, takes 44,100 samples a second, each of 16 bits, for each of two stereo channels.

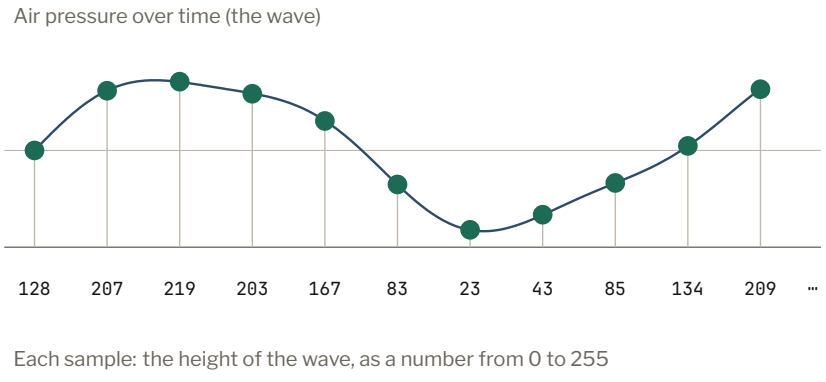


Figure 2.9 A sound wave measured at regular moments. Each dot is a sample, a single number recording the height of the wave at that instant. This example uses numbers from 0 to 255; a compact disc uses 16 bits per sample, which allows 65,536 levels, and takes 44,100 samples every second.

That rate is a little more than twice the highest pitch most people can hear, about 20,000 vibrations a second, and a sound must be measured at least twice per vibration to be captured faithfully.

Most pictures and music on a phone are not stored in full. Formats such as JPEG for photographs and MP3 for music use *compression*: mathematical methods that discard detail the eye or ear is unlikely to miss and store the rest more economically. A 36-megabyte photograph typically becomes a file of a few megabytes, and a minute of music about one megabyte. Compression is a program working on the numbers. The picture and the music are still numbers.

THE BUSINESS VIEW

Why a photo weighs more than a contract

A written contract of twenty pages is about 50,000 characters, or roughly 50 kilobytes as plain text. A single photograph from a phone is a few megabytes, about a hundred times larger, and a minute of video is larger again by a factor of ten or more.

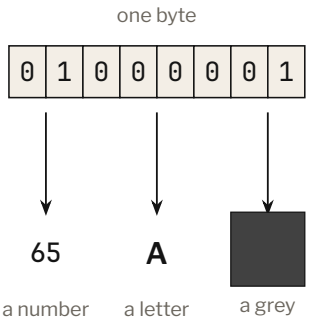
Every byte has to be stored, sent across a network and usually copied for safety, and cloud services charge for each of these by the gigabyte. An app that lets customers post text reviews costs very little to run; one that lets them upload photographs or video may cost a hundred times as much in storage and data transfer for the same number of customers, and will feel slower on a weak mobile connection. When you plan a product, it is worth asking early what kind of data it will hold, and how much. The question is cheap to answer on paper and expensive to discover from a bill.

A number means nothing on its own

We have now seen a byte stand for a number, a letter, the brightness of a pixel and the height of a sound wave. That leads to the most important idea in this chapter. The byte `01000001` is 65 if a program treats it as a number. It is the letter A if a program treats it as text. It is a dark grey if a program treats it as a pixel. In Chapter 4 we will see that a number of this kind can also be an instruction, telling the processor what to do next. The switches are the same in every case. The meaning is supplied entirely by the program that reads them.

A pattern of bits has no meaning of its own. It means whatever the program reading it takes it to mean.

This is why files have types, usually shown by the ending of the file's name. `report.txt` is a promise that the bytes are meant to be read as text, `photo.jpg` that they are a compressed photograph, `song.mp3` compressed sound. The computer uses that ending to choose which program should open the file. Rename `photo.jpg` to `photo.txt` and open it in a text editor, and you will see a screen of meaningless symbols: the bytes of a photograph, read as though they were letters. The bytes have not changed. Only the reading has, and the program does not stop to wonder whether it is reading them the right way.



Meaning depends on the program reading it.

Figure 2.10 One byte, three readings. Nothing in the byte itself says which reading is correct.

AT THE KEYBOARD

Garbled text

Sooner or later you will see `cafÃ©` where you expected `café`, or a row of question marks or empty boxes where a Hindi or Tamil name should be. This is almost always an encoding mistake. The text was saved as UTF-8, in which é is the two bytes shown above, but some program read it under an older one-byte code and turned each of those bytes into a separate character. It happens most often where data passes between programs: a spreadsheet exported as a CSV file and opened elsewhere, text copied from a web page, an old database.

When you build with an AI agent and see garbled characters, you do not need to fix the bytes yourself. You need to recognise the symptom and name it: "Names in Hindi show as question marks after the import.

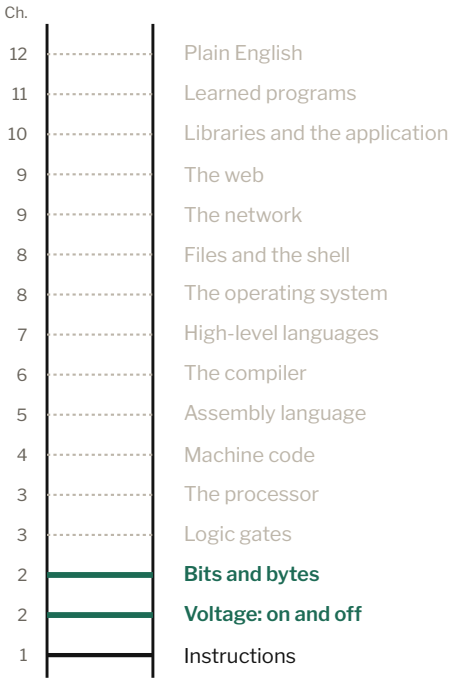
Please make sure the file is read and saved as UTF-8 throughout." Naming the likely cause saves several rounds of guessing.

Where this leaves us

We now have the lowest layer of the machine. Inside a chip there are only switches, each holding a high or a low voltage, and the decision to use two states rather than ten is what makes the machine reliable: a signal can be disturbed, cleaned up and copied without error. Read as 0 and 1, those states become bits; grouped in eights, they become bytes; and with agreed tables such as ASCII and Unicode, and agreed measurements such as pixels and samples, bytes can hold text in any language, photographs and sound.

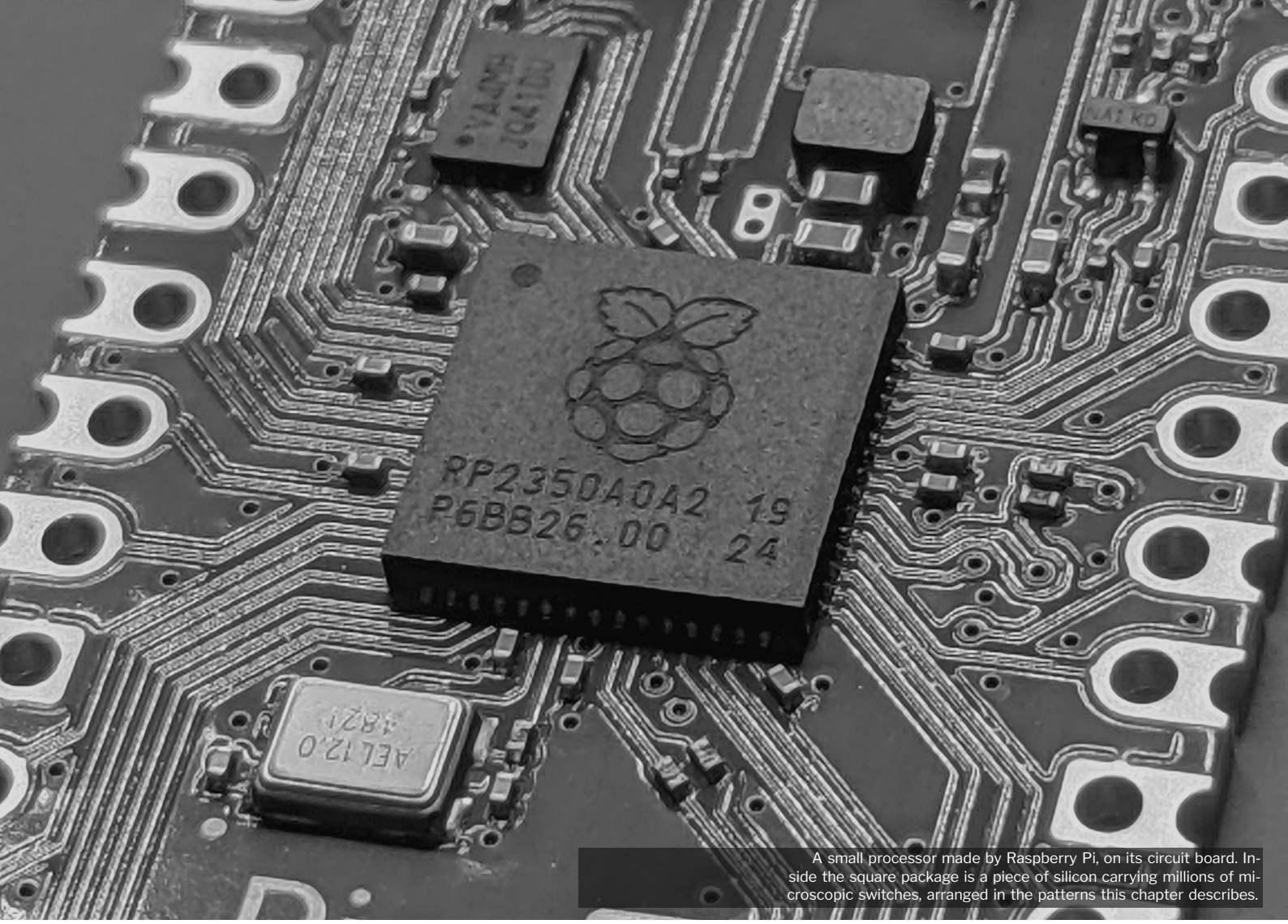
What we do not yet have is any way of doing something with them. A pattern of bits sitting in a group of switches is a record, not a calculation. The list 4, 9 and 15 can be stored as three bytes, but nothing at this level can add them up, and nothing here decides what a byte means; that is left to whatever reads it.

The next layer comes from combining switches so that the positions of some control the positions of others. Arranged in the right way, a handful of transistors can compare two bits, add them, or remember one. Chapter 3 shows how such small circuits, called logic gates, are built up into a processor.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.



A small processor made by Raspberry Pi, on its circuit board. Inside the square package is a piece of silicon carrying millions of microscopic switches, arranged in the patterns this chapter describes.

CHAPTER THREE

Gates and the Processor

A switch can only be on or off. Wire a few of them together and they can make a decision; wire a few thousand and they can add; wire billions and they become a processor that follows instructions faster than anyone can count. This chapter shows how.

CHAPTER 2 ENDED WITH a machine that holds nothing but bits. Every letter, photograph and song inside a computer is a pattern of high and low voltages, and each voltage is set by a switch. That explains how a computer stores things. It does not yet explain how it does anything with them: how two numbers stored as switches are added to give a third, how a machine decides between one course of action and another, or how it works through a list of instructions one after another.

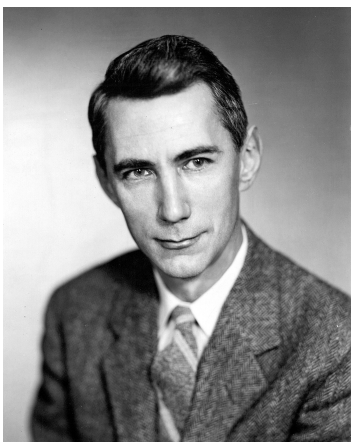
The answer comes in three steps, and this chapter takes them in order. First, switches can be wired so that they carry out simple rules of logic. Second, those rules, combined, can do arithmetic and can remember. Third, arithmetic and memory, kept in step by a clock, can be organised into a machine that reads an instruction, carries it out, and moves on to the next. That machine is the processor, the part of every computer that does the work. What makes a modern processor remarkable is not any clever part, but the number of simple parts and the speed at which they work.

Switches that do logic

In the summer of 1937 a 21-year-old graduate student at the Massachusetts Institute of Technology, Claude Shannon, finished a master's thesis with an unpromising title: "A Symbolic Analysis of Relay and Switching Circuits". He submitted it on 10 August 1937, and a version was published the following year in the *Transactions* of the American Institute of Electrical Engineers.

Shannon had two pieces of knowledge that few people then held together. As an undergraduate at the University of Michigan he had studied the algebra of logic devised by George Boole in the 1850s, in which statements are either true or false and are combined with words such as *and*, *or* and *not*. At MIT he had worked on Vannevar Bush's differential analyser, a large calculating machine whose operation was controlled by circuits of relays: switches that are opened and closed by an electromagnet. Such circuits were designed by experience and trial.

Shannon saw that the two subjects were the same. A switch is either closed or open, as a statement is either true or false. Two switches placed one after the other in a wire let current through only if the first *and* the second are closed. Two switches placed side by side, so that current can take either path, let it through if the first *or* the second is closed. Boole's



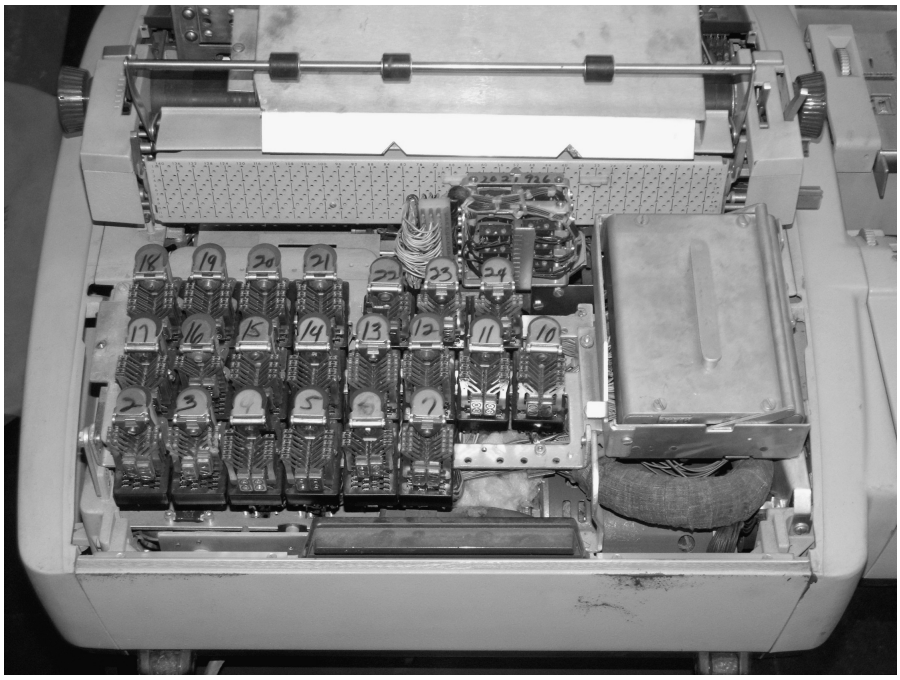


Figure 3.2 Relays inside a Friden Flexowriter, an automatic typewriter. Each numbered unit is a switch worked by an electromagnet, the kind of circuit Shannon analysed in 1937. A modern chip does the same job with transistors far too small to see.

algebra therefore described exactly what a circuit of switches would do, and could be used to simplify circuits and to design new ones on paper.

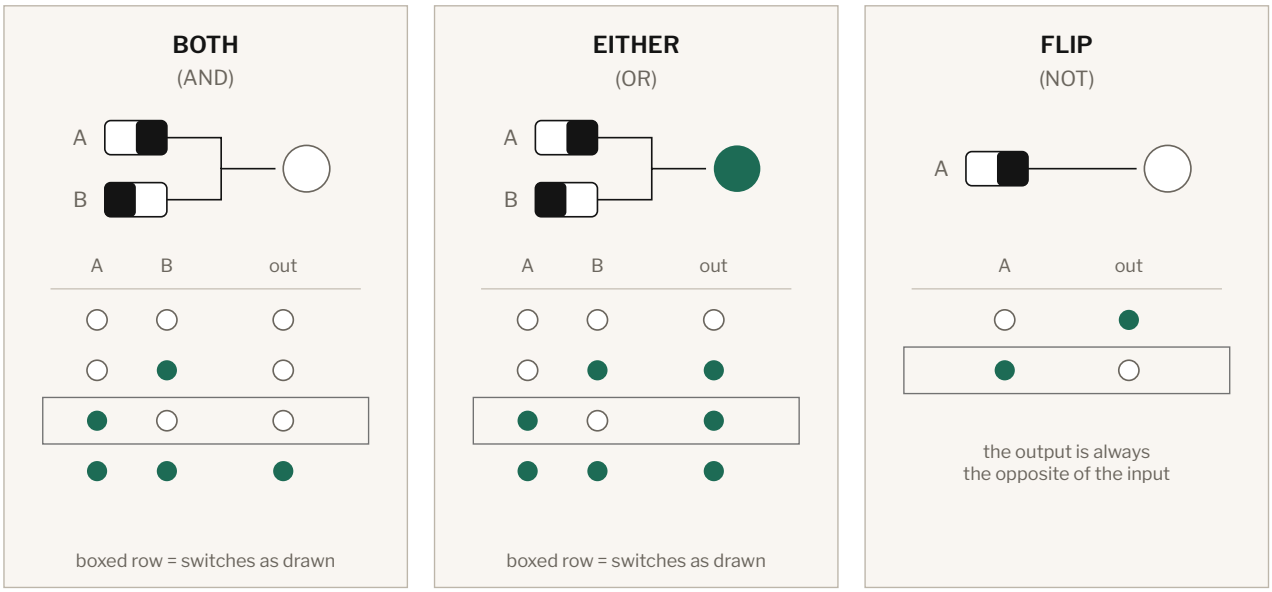
The thesis had a second consequence that mattered even more. If switches can carry out logic, and if numbers can be written in binary as patterns of ones and zeros, then switches can carry out arithmetic too, because the rules of binary arithmetic can be written as rules of logic. Shannon made the point himself, with a circuit for adding binary numbers. The psychologist Howard Gardner later described the work as possibly the most important master's thesis of the century. Engineers in Japan and the Soviet Union, among them Akira Nakashima, were reaching similar conclusions in the same decade, but it was Shannon's clear statement that the field took up.

Three simple rules

Shannon's relays have long since been replaced by transistors, which do the same job with no moving parts, billions of times faster and at a tiny fraction of the size. But the logic is unchanged, and it is built from small units that engineers call gates.

A **logic gate** is a circuit that takes one or two bits in and gives one bit out, according to a fixed rule. Three rules are enough to build everything else. An *AND* gate gives 1 only if both of its inputs are 1; in plain words, "both".

logic gate A small circuit, built from a few transistors, that takes one or more bits in and gives one bit out by a fixed rule.



Filled red circle = on; open circle = off. Switch knob to the right = on.

Figure 3.3 The three basic gates, pictured as switches and a lamp, with their truth tables. AND ("both") lights the lamp only when both switches are on. OR ("either") lights it when at least one is on. NOT ("flip") gives the opposite of its input. The boxed row in each table matches the switches as drawn.

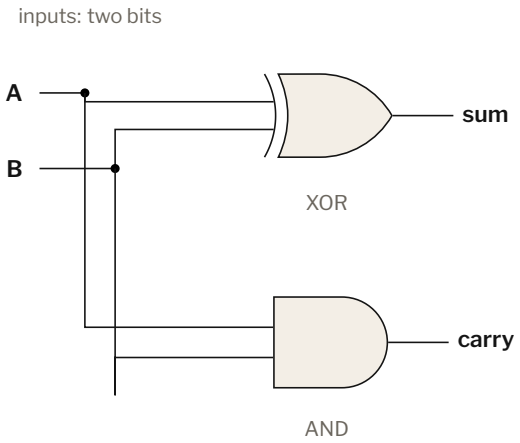
An *OR* gate gives 1 if either input is 1, or both; in plain words, "either". A *NOT* gate has a single input and gives the opposite: 1 becomes 0 and 0 becomes 1. Each takes a handful of transistors to build.

They are easiest to picture as switches and a lamp, as in Shannon's circuits: for AND the lamp lights only when both switches are on, for OR when either is, and for NOT the lamp is lit only when the switch is off. Because each input can only be 0 or 1, it is possible to list every combination and the result it produces. Such a list is called a **truth table**. A gate with two inputs has only four rows, and the table is the whole of its behaviour.

truth table A list of every possible combination of inputs to a logic circuit, with the output each one produces.

A single gate does very little. The power comes from connecting gates, so that the output of one gate becomes the input of another. A few gates wired together can express a rule such as "approve the loan if the applicant is employed *and* either has no existing debt *or* has a guarantor". Any rule whose inputs and output are ones and zeros, however complicated, can be built from these three kinds of gate. (In fact a single kind, called NAND, which is AND followed by NOT, is enough on its own.)

A gate does not know what its inputs mean, whether a 1 stands for "employed" or for part of a number. It applies its rule to whatever arrives, every time, in a tiny fraction of a billionth of a second.



A + B = carry sum

```
0 + 0 = 00

0 + 1 = 01

1 + 0 = 01

1 + 1 = 10
```

Left digit is the carry;
right digit is the sum.

Figure 3.4 A half adder. The XOR gate produces the sum digit; the AND gate produces the carry. The table on the right lists all four cases. Only one plus one produces a carry, which is the binary way of writing two.

Adding with wires

The step from logic to arithmetic is shorter than it looks. Consider the simplest possible sum: adding two single binary digits. There are only four cases. Nought plus nought is 0. Nought plus one is 1, and so is one plus nought. One plus one is two, which in binary is written 10: a 0 in this column and a 1 carried into the next, exactly as nine plus one in ordinary arithmetic gives 0 with a 1 carried.

Look at the answers as two separate columns. The digit that stays in this column, the *sum*, is 1 when exactly one of the inputs is 1, and 0 otherwise. The digit carried to the next column, the *carry*, is 1 only when both inputs are 1. That second rule is simply AND. The first is a rule called "exclusive or", XOR for short, which means "one or the other but not both", and it can be built from a few AND, OR and NOT gates. So two gates, side by side, add two bits. The circuit is called a **half adder**.

half adder A circuit of two gates that adds two single bits, giving a sum bit and a carry bit.

A half adder handles only the rightmost column of a sum, where nothing is carried in. Every other column has three things to add: a digit from each number and a carry from the column to its right. Two half adders and an OR gate together handle that case, and the result is called a full adder. Line up full adders side by side, with the carry from each one wired to the next, and you have a circuit that adds numbers of any length, column by column, carrying as it goes. It is the method of long addition taught in primary school, carried out by wiring instead of a pencil.

Take three plus two. In binary, three is 011 and two is 010. In the rightmost column, 1 plus 0 gives 1 with nothing to carry. In the middle column, 1 plus 1 gives 0 and carries 1. In the left column, 0 plus 0 plus the carried 1 gives 1. The answer is 101, which is five. A chain of three full adders does

this in a single step, as fast as the voltages can settle through the gates. A processor built to handle 64-bit numbers has, in effect, a chain of 64 (real chips use faster arrangements for the carries, but the principle is the same).

This is the moment at which arithmetic stops being something a person does and becomes something a circuit is. Nobody inside the chip is adding; the answer appears because of the way the wires are connected. Subtraction, comparison ("is this number bigger than that one?") and multiplication are built from the same parts. Gathered together in one place, these circuits make up the part of a processor called the arithmetic logic unit, or ALU.

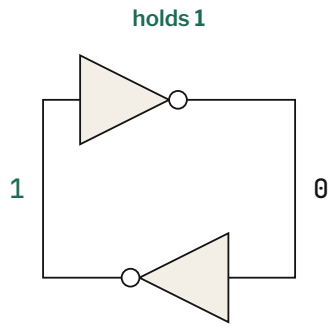
A circuit that remembers

Arithmetic is useless if the result vanishes the instant it appears. A computer must also keep numbers: the ones it is working on, the answers it has produced, and, as Chapter 4 explains, the instructions it is following. Here too the material is gates.

Take two NOT gates and connect them in a loop, so that the output of each is the input of the other. If the first gate's output is 1, the second gate turns it into 0, and that 0 goes back into the first gate, which turns it into 1 again. The loop holds itself in place. If it started the other way round, with 0, it would hold 0 just as firmly. Add a couple of gates to let a new value be written in when needed, and the result is a circuit that remembers one bit for as long as the power stays on. Engineers call it a latch or a flip-flop.

Put eight such circuits side by side and they hold a byte; put 64 side by side and they hold a large number. A row of this kind, inside the processor itself, is called a **register**. A processor has only a few dozen registers, but they are the fastest memory in the machine, and they are where numbers sit while the ALU works on them. One register, often called the accumulator, holds the running result of a calculation.

A computer needs far more storage than a few dozen registers, and that storage sits outside the processor, in the main memory. Picture it as a wall of pigeonholes in a sorting office. Each pigeonhole holds one number and has a number painted on it so that it can be found. The painted number is called an **address**. To read a value, the processor sends the address along a set of wires and the contents come back. A laptop with 8 gigabytes of memory has about eight billion pigeonholes, each holding one byte.

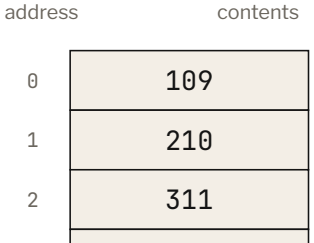


Two switches holding each other in place

Figure 3.5 Two NOT gates, each feeding the other. Whatever value the loop holds, it keeps, for as long as the power is on.

register A small, very fast storage place inside the processor that holds one number while it is being worked on.

address The number that identifies one cell of memory, so that the processor can find what is stored there.



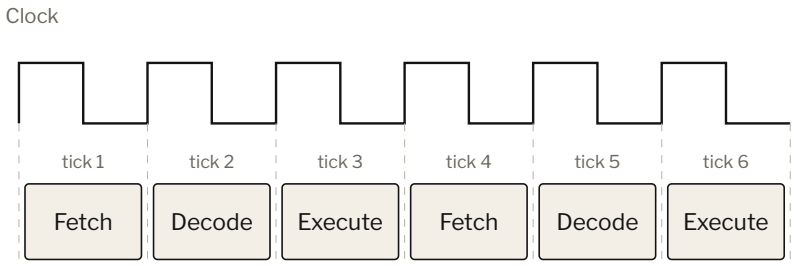


Figure 3.7 The clock. On every tick the registers take in their new values and the next piece of work begins. (Real processors overlap these steps, so that several instructions are under way at once.)

One step of the work happens on every tick.

1 GHz = one billion ticks a second

This kind of memory is called RAM, for *random-access memory*, because any pigeonhole can be reached as quickly as any other. (Most RAM today stores each bit as a tiny electrical charge rather than in a loop of gates, but the picture is the same.) RAM forgets everything when the power goes off. That is why unsaved work is lost when a laptop's battery dies, and why computers also need slower, permanent storage, such as a solid-state drive, to keep files, the subject of Chapter 8.

A pigeonhole holding 17 does not say whether 17 is a price, a letter, a shade of grey or part of an instruction. As Chapter 2 showed, a number means nothing until something decides how to read it.

Keeping time

Signals take time to pass through gates. When the inputs to an adder change, its output flickers for a moment while the new values work their way through the chain, and only then settles on the right answer. If the next part of the machine read the result too soon, it would read nonsense. Thousands of circuits working at once need some way of agreeing when a result is ready.

The solution is a **clock**: a circuit that sends a steady pulse, high and low and high again, to every part of the processor. It works like a metronome for a group of musicians. Between beats, the gates do their work and their outputs settle; on each beat, the registers capture the new values all at once, and the next round begins.

clock A circuit that sends out a steady stream of pulses, so that all parts of the processor change step at the same moment.

The speed of the clock is measured in hertz, or ticks per second. The first microprocessors in the early 1970s ran at a few hundred thousand ticks a second. A laptop or phone processor today runs at roughly three

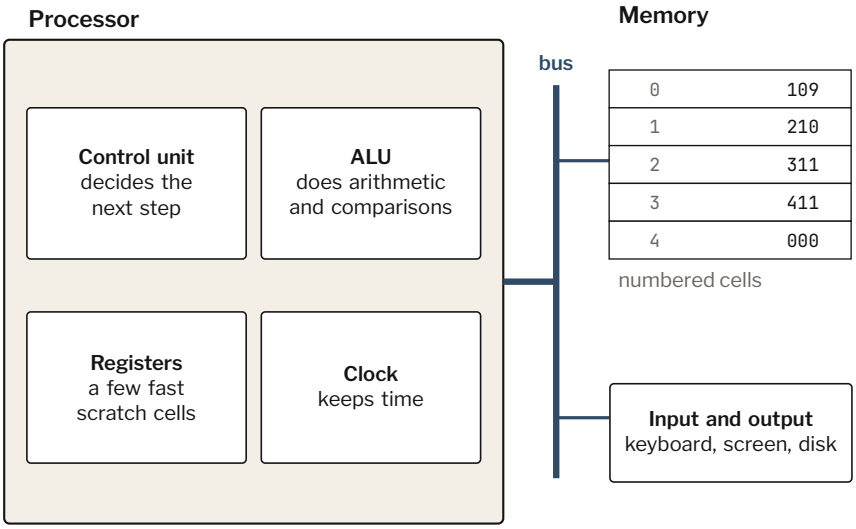


Figure 3.8 The parts of a processor. A bus connects it to memory, whose numbered cells here hold the first instructions of a small program written for the simple machine described in the text, and to input and output.

to six billion: three to six gigahertz. At four gigahertz a single tick lasts a quarter of a billionth of a second, in which time light travels about seven and a half centimetres.

The processor

We now have gates that apply rules, an ALU that does arithmetic and comparisons, registers, a main memory of numbered cells, and a clock. The last ingredient turns them from a calculator into a computer: a part that decides, at every moment, what the others should do next.

That part is called the control unit, and together with the ALU, the registers and the clock it makes up the **processor**, often called the central processing unit or CPU. Bundles of wires called buses connect it to the memory and to the outside world. Babbage's division of his Analytical Engine into a mill and a store has survived: the mill is now the processor, the store the memory.

processor The part of a computer that carries out instructions: it fetches each one from memory, works out what it means and does it.

What does the control unit decide? It follows the instructions of a program, which are kept in memory as numbers. In the simple machine used in the next few chapters, each instruction is a three-digit number. The first digit says what to do: 1 means load a number into the accumulator, 2 means add a number to it, 3 means store the result, 4 means print it, and 0 means stop. The other two digits give the address of the cell to use. So the number 210, read as an instruction, means "add the number in cell 10 to the accumulator". Real processors use longer numbers and many more kinds of instruction, but they work on the same principle.

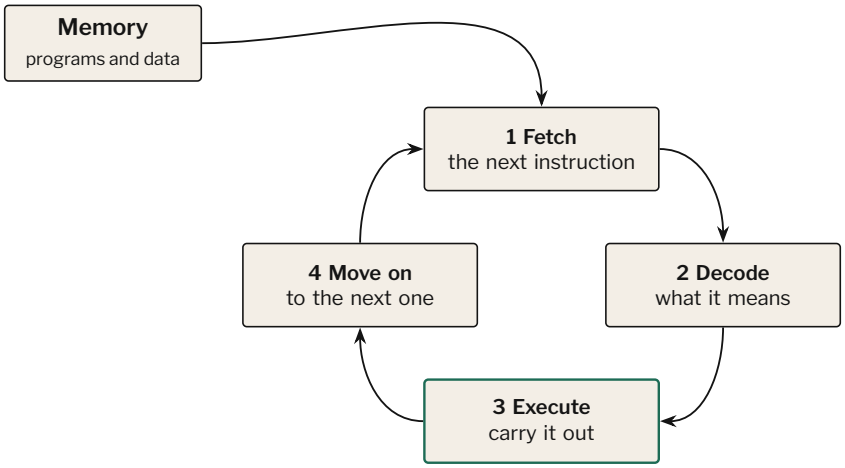


Figure 3.9 The fetch–decode–execute cycle. The processor fetches the next instruction from memory, decodes it, carries it out and moves on. Everything a computer does is this loop, repeated billions of times a second.

To keep its place, the processor uses one special register, the program counter, which holds the address of the next instruction. Then it repeats the same short cycle over and over. It *fetches* the number stored at that address and copies it into the processor. It *decodes* it: the control unit's gates examine the bits and switch on the paths that the instruction requires, connecting, say, a memory cell to the ALU and the ALU to the accumulator. It *executes* it, and the addition or copying happens. Finally it moves the program counter on to the next address, and the cycle begins again. This is the fetch–decode–execute cycle, and every program you have ever used has run as a long sequence of turns of this loop.

Two kinds of instruction give the loop its flexibility. A *compare* instruction checks something, such as whether two numbers are equal or one is larger, and records the result. A *jump* instruction changes the program counter, so that the next instruction is fetched from somewhere other than the following cell. A jump can be made conditional: "if the last comparison found the numbers equal, jump to cell 40". With compare and jump, a program can repeat steps and choose between them: the repetition and selection that Chapter 1 found in every recipe and every program, now built from wiring.

Here is the recurring example of this book, adding up a list of numbers, as a processor would carry it out. Set a register to zero to hold the total. Load the next number from memory. Add it to the total. Compare: was that the last number? If not, jump back and load the next one. If so, store the total in memory, where something else can display it. Each of those steps is one instruction, or close to it. A list of a million numbers takes

a few million turns of the loop, which a modern processor completes in well under a thousandth of a second.

The complete list of instructions that a processor understands is called its **instruction set**. A typical instruction set has a few hundred basic kinds of instruction, more if every variation is counted: load, store, add, subtract, multiply, compare, jump, and specialised operations for things such as encryption and graphics. That list is the whole of the processor's vocabulary. Everything a computer does has to be expressed, in the end, as a sequence of these simple verbs.

instruction set The full list of operations a processor can carry out. It is the only vocabulary the processor understands.

Different families of processor have different instruction sets, and a program written in one set's numbers will not run on another. Two families dominate today. The x86 family descends from Intel's 8086 of 1978 and runs most desktop PCs and the servers in data centres. The ARM family began at Acorn Computers in Cambridge, England, where Sophie Wilson and Steve Furber designed a processor whose first chips worked on 26 April 1985. ARM designs now run nearly every smartphone and Apple's recent Macs. The problem of programs that work on one family but not another reappears in Chapter 5.

The processor has no idea what a program is for. It only ever follows the next instruction.

That is the central fact of this chapter. The processor does not know that the numbers it is adding are a restaurant bill, or that the jump it has just made will charge a customer twice. It fetches, decodes, executes and moves on. All the meaning lies in the program, which is to say with the people who wrote it.

AT THE KEYBOARD

Exactly what it is told

When you ask an AI coding agent to build something, it can seem to understand you. Underneath, every part of that exchange is processors running the loop described above. The model that writes the code runs on processors, mostly specialised chips doing vast numbers of multiplications and additions. And the code it writes is eventually turned into instructions that some processor will follow, one at a time, exactly as written.

If the agent's code contains a mistake, such as a test that should say "greater than or equal to" but says "greater than", no part of the machine

will notice that it seems odd. The processor will carry out the mistake as faithfully as it carries out everything else. The only judgement in the system is the judgement that people bring to it, which is why Volume 2 spends as much time on checking an agent's work as on asking for it.

From 2,300 switches to billions

For the first 25 years of electronic computing, a processor was not a single object. It was cabinets, then boards, then handfuls of chips, each containing a few of the gates described above, wired together by hand or on circuit boards. The integrated circuit, described in Chapter 2, made it possible to put many transistors on one piece of silicon. By 1971 it had become possible to put a whole processor on one.

A CLOSER LOOK

The Intel 4004

In 1969 a Japanese calculator company, Busicom, asked a young American firm, Intel, to make the chips for a new range of desktop calculators. Busicom's own design needed a set of custom chips, each wired to do a particular job. An Intel engineer, Marcian "Ted" Hoff, working with Stanley Mazor, proposed something different: a single general-purpose processor that would do the calculator's work by following a program stored in a separate memory chip. Change the program and the same chip could serve a different calculator, or a machine that was not a calculator at all.

Federico Faggin, who joined Intel in April 1970, led the design of the chip itself, with Masatoshi Shima of Busicom working on the logic. Faggin's part was long underplayed; historians now credit the 4004 to the team. Intel later negotiated the right to sell the chip to other customers, and on 15 November 1971 it advertised the 4004 in the trade paper *Electronic News* under the heading "Announcing a new era of integrated electronics".

The 4004 is usually described as the first commercially available microprocessor; other chips of the period have competing claims to be first of some kind. It had about 2,300 transistors, handled numbers four bits at a time, ran at 740,000 ticks a second, and could carry out around 60,000 instructions a second. What mattered was the idea: a single

chip whose behaviour was set by a program, not by its wiring, which is Jacquard's separation of the pattern from the loom, made in silicon.

Since then, the number of transistors that can be placed on a chip has grown roughly as Gordon Moore predicted, doubling every couple of years for half a century. Apple's M4 processor, announced in May 2024 for its tablets and laptops, has 28 billion transistors, about twelve million times as many as the 4004. The largest chips built for artificial intelligence have more than 200 billion.

The extra transistors have been spent in several ways. A modern processor overlaps its work, starting the next instruction before the last has finished and often carrying out several at once. As a rough order of magnitude, one processor core today completes around ten billion instructions a second. And since the early 2000s, when faster clocks began to make chips too hot, manufacturers have put several complete processors, called cores, on one chip. A phone or laptop chip now typically has somewhere between six and two dozen, each running its own fetch–decode–execute loop, so that several programs can run at once.

None of this changes the basic picture. A core in a 2026 laptop does what the 4004 did: it applies simple logical rules to bits, following a program one instruction at a time. It does so more than a hundred thousand times faster.

THE BUSINESS VIEW

Who makes the chips

The chip industry is at least three kinds of business. Some firms design processors and also manufacture them in their own factories, as Intel has done since the 1970s. Some design chips but own no factories: Apple, Nvidia and Qualcomm design their own processors and pay others to make them. And ARM, based in Cambridge, makes no chips at all; it designs processor cores and instruction sets and licenses them to other companies, earning a fee on each chip sold.

Manufacturing the most advanced chips is now concentrated in very few places. The Taiwan Semiconductor Manufacturing Company (TSMC), founded in 1987 to make chips designed by others, produces most of the world's leading-edge processors, including Apple's. Such a factory costs tens of billions of dollars to build. The concentration became a public concern during the chip shortages of 2020–22, and governments in the United States, Europe, Japan and India have since offered large subsidies to bring chipmaking closer to home.

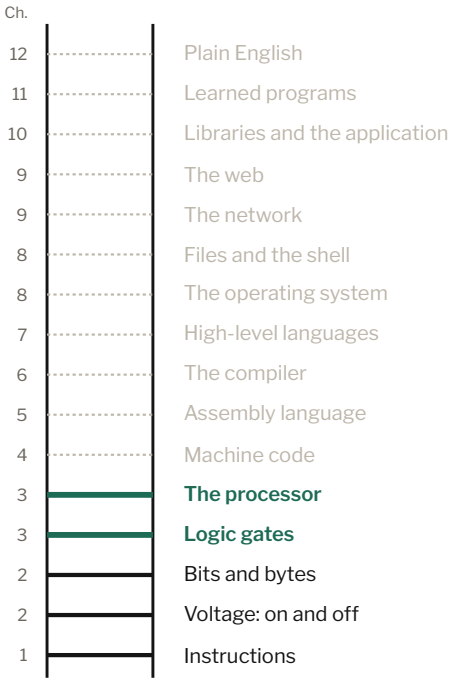
India's part has mostly been in design: many large chip companies run major design centres in Bengaluru, Hyderabad and elsewhere. Since the India Semiconductor Mission was launched in December 2021, the government has also backed factories in Gujarat, among them a chip assembly and testing plant by the American firm Micron in Sanand and a fabrication plant planned by Tata Electronics in Dholera. Whether these succeed will take years to judge. The processor at the bottom of every software layer is also a strategic product, with a supply chain that crosses many borders.

Where this leaves us

We now have a machine. Shannon showed that switches could carry out logic. Gates built from transistors apply simple rules to bits; chains of gates add; loops of gates remember; a clock keeps them in step; and a control unit drives the whole through the fetch–decode–execute cycle, one instruction at a time, billions of times a second. Every computer, from the 4004 to the phone in your pocket, is built this way.

What the processor lacks is any sense of purpose. It knows a few hundred verbs and nothing else. It will add, compare and jump, but only when an instruction tells it to, and the instructions must arrive as numbers in its memory, written in its own instruction set.

That leaves the question of how the numbers get there. The first electronic computers were built before anyone had settled it, and the earliest were instructed by plugging in cables and setting switches by hand. Chapter 4 describes that work, the people who did it, and the idea that replaced it: keeping the program in memory as numbers, alongside the data.



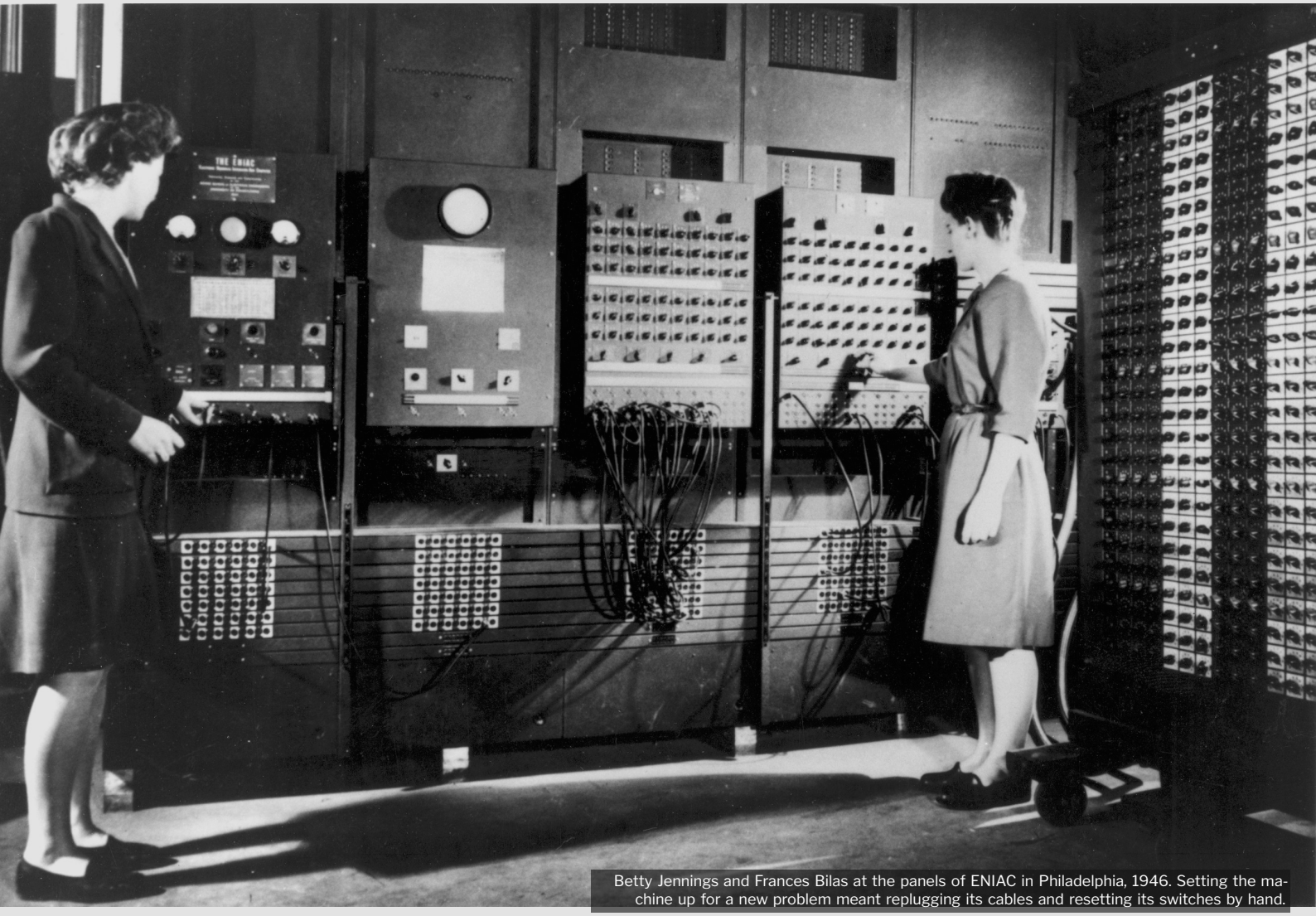
THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

PART TWO

Climbing the Ladder

How people taught the machine to accept instructions in forms that are easier for people to write: numbers, then names, then languages, then an operating system to run them.



Betty Jennings and Frances Bilas at the panels of ENIAC in Philadelphia, 1946. Setting the machine up for a new problem meant replugging its cables and resetting its switches by hand.

CHAPTER FOUR

From Rewiring to Remembering

The first electronic computers were programmed by moving cables, a job that took days. The remedy, worked out in 1945 and first made to work in 1948, was to keep the instructions inside the machine as numbers, in the same memory as the data. Those numbers are called machine code, and every program still ends up as them.

IN FEBRUARY 1946 the United States Army and the University of Pennsylvania showed the press a machine that filled a large room in Philadelphia. It was called ENIAC, and it could add numbers thousands of times faster than a person with a desk calculator. Some newspapers called it a giant brain. For the demonstration it worked out the flight of an artillery shell, the kind of calculation it had been built to do.

The program for that demonstration had not been typed in or loaded from anywhere. It had been built. Cables had been plugged between the machine's panels and hundreds of switches set by hand, following plans drawn up over weeks by a small group of young women, none of whom was named in the coverage. Within three years, machines in England were running programs that were held inside them as numbers, and could be changed in minutes.

Chapter 3 ended with a processor that obeys numbered instructions, one after another, very fast. This chapter is about how those numbers first got into a machine: the change from programs as wiring to programs as numbers held in memory, and what such numbers look like.

A machine set up by hand

ENIAC, the Electronic Numerical Integrator and Computer, was built at the Moore School of Electrical Engineering at the University of Pennsylvania. Its designers were J. Presper Eckert, an engineer in his twenties, and John Mauchly, a physicist, and its customer was the Army's Ballistic Research Laboratory. The Army needed firing tables: booklets that told gunners how to aim a particular gun with a particular shell at a target a given distance away. Each table required the paths of many shells to be calculated step by step, and the work was done by teams of people, most of them women, who were employed as "computers". In the 1940s that was a job title, not a machine. The war had made the backlog of tables impossible to clear by hand.

Work began in 1943 and the machine was running by the end of 1945. Its first real job, from December 1945, was a set of calculations for the scientists at Los Alamos who were studying the hydrogen bomb. It was announced to the public on 14 February 1946 and formally dedicated the next day. It was very large: some 40 panels arranged in a U about 24 metres long, nearly 18,000 vacuum tubes, about 30 tonnes in weight and around 150 kilowatts of power, enough to heat the room on its own.

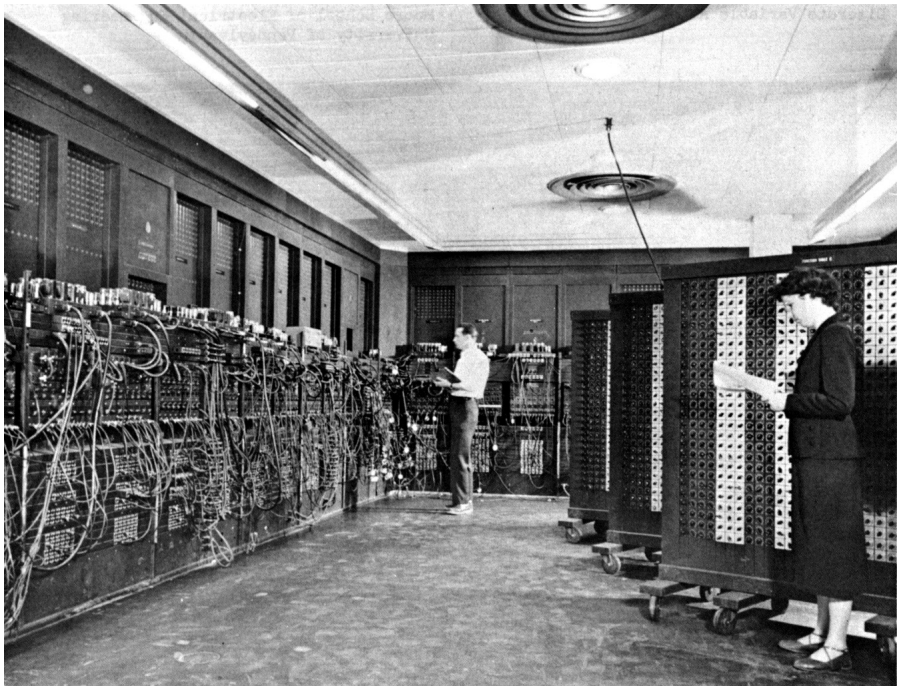


Figure 4.1 ENIAC in a US Army photograph. Each tall panel is one unit of the machine. The cables hanging from the panels on the left were part of the program: to give the machine a different job, they had to be unplugged and plugged in again elsewhere.

ENIAC was not one calculator but a collection of units that each did a single kind of work. There were 20 accumulators, each able to hold and add one ten-digit decimal number, along with units to multiply, divide and take square roots, and tables of switches for setting constants. To solve a problem, the units had to be told what to do and when. Cables carried numbers from one unit to another, other cables carried a pulse that told the next unit to start, and switches on every unit set what it would do when its turn came. The program was the pattern of those connections and settings.

Giving ENIAC a new problem was therefore less like handing a clerk new instructions than like rearranging the machines on a factory floor for a new product. Planning a problem could take weeks, and setting it up on the machine took a day or two. Once set up, it ran at electronic speed. For a long series of similar calculations, such as the many paths needed for a firing table, that was a good bargain. For anything that changed often, it was not.

There was a reason the instructions lived in the wiring. ENIAC's only working memory was its 20 accumulators, so it could hold just 20 numbers at once, and each accumulator needed hundreds of vacuum tubes. There was simply nowhere inside the machine to keep a list of instructions. Its builders knew this was a weakness before the machine was finished.

The first programmers

The people who worked out how to set ENIAC up were chosen in 1945 from among the Army's human computers. There were six of them: Kathleen McNulty, Betty Jean Jennings, Frances Elizabeth (Betty) Snyder, Marlyn Wescoff, Frances Bilas and Ruth Lichterman. They are better known today by their married names: Kay Mauchly Antonelli, Jean Bartik, Betty Holberton, Marlyn Meltzer, Fran Spence and Ruth Teitelbaum.

They had no manuals, because none existed, and no programming language, because the idea did not yet exist either. They learned the machine from its wiring diagrams and from questioning the engineers who had built it. Their work was to take a calculation, such as the path of a shell, and break it into steps small enough for ENIAC's units to carry out; to decide which unit would do each step and at which pulse; and to draw all of this up as diagrams showing which cables went where and how every switch should be set. Then they set the machine up, ran it, and when it went wrong, worked out whether the fault lay in their plan, in a cable, or in one of its thousands of tubes.

Jennings and Snyder led the work on the trajectory program shown to the press in February 1946. It ran correctly. At the dinner held for the dedication the programmers were not introduced, and photographs of them at the machine were published without their names, sometimes with captions that treated them as models posing beside it. In the 1940s the machine was seen as the achievement and the work of setting it up as a kind of clerical task. Historians have since argued that this view, and the fact that most of the people doing the work were women, kept their contribution out of sight for decades.

A CLOSER LOOK

Finding the ENIAC programmers

In 1986 Kathy Kleiman, then a student at Harvard, came across photographs of ENIAC with women standing at its panels. The photographs did not say who they were. She spent years tracking them down and recording their memories, and her work, with others', brought their story into the open.

Recognition came late. In 1997 all six were inducted into the Women in Technology International Hall of Fame, and in the same year Betty Holberton received the IEEE Computer Pioneer Award. Jean Bartik was made a Fellow of the Computer History Museum in 2008. Kleiman's

documentary *The Computers* appeared in 2014, and her book *Proving Ground* in 2022. Several of the six went on to long careers in computing. Holberton, for example, worked on the instructions for later commercial machines and on early programming languages.

Their story matters for this book for a second reason. What they did was programming in the full sense: turning a calculation into steps a machine could follow, and finding out why it did not. The tools were cables and switches, but the thinking was the same thinking that every later layer of software was built to support.

Keeping the program in memory

The obvious remedy for ENIAC's weakness was to give the next machine a memory large enough to hold its instructions as well as its numbers. Instructions could then be written down as numbers, fed in like any other data, and changed without touching a cable. Eckert and Mauchly had been discussing a cheaper kind of memory, and keeping instructions in it, since early 1944, and from that year the Moore School team began planning a successor to ENIAC, called EDVAC.

In 1944 the team was joined, as a consultant, by John von Neumann, a Hungarian-born mathematician at the Institute for Advanced Study in Princeton who also advised the atomic weapons project at Los Alamos. Von Neumann took part in the EDVAC discussions and wrote up the design. His *First Draft of a Report on the EDVAC*, dated 30 June 1945, ran to 101 pages and was never finished. It described the machine in five parts: a unit for arithmetic, a unit for control, a memory, and units for input and output. Its central idea was that one memory would hold both the numbers being worked on and the instructions for working on them, all written in the same code. To describe the circuits, von Neumann borrowed the notation of a 1943 paper on the nerve cells of the brain by Warren McCulloch and Walter Pitts, a link that returns in Chapter 11.

A machine built on this plan is called a **stored-program computer**. The general layout, a single memory holding both program and data and a processor that takes instructions from it one at a time, came to be called the **von Neumann architecture**, and almost every computer built since follows it. The name is not entirely fair. The draft carried only von Neumann's name, and Herman Goldstine, the Army's liaison officer to the project, circulated copies in the summer of 1945. Eckert and Mauchly felt that ideas they had developed before von Neumann arrived had gone

Figure 4.2 John von Neumann at Los Alamos. His report of June 1945 gave the first written description of

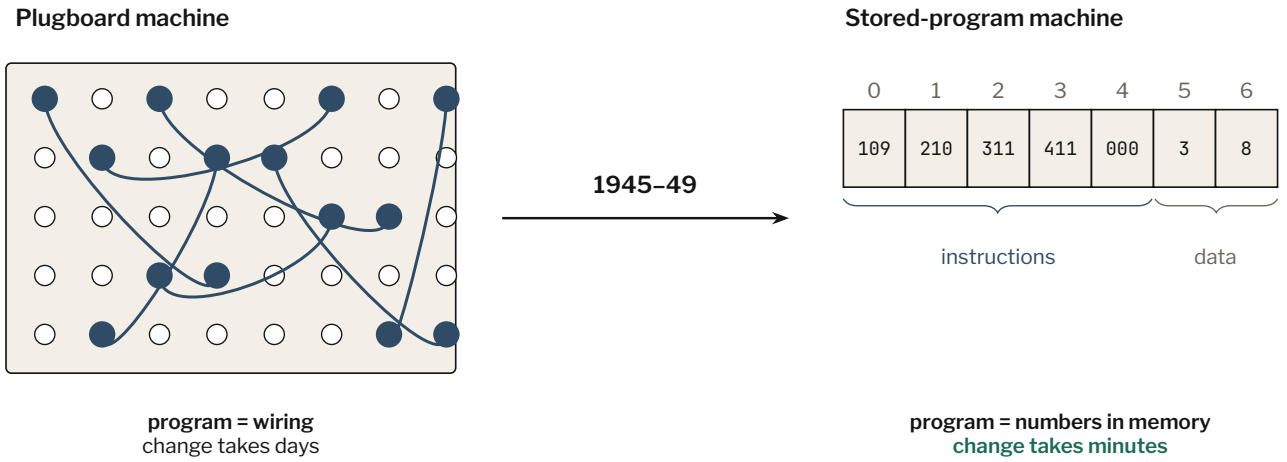


Figure 4.3 Two ways of giving a machine its program. On the left, a plugboard: the program is a pattern of cables, and changing it means rewiring, which took days. On the right, a stored-program machine: the program is a row of numbers in memory, next to the data, and changing it means loading different numbers. The change was made between 1945 and 1949.

out under someone else's name. Because the draft had circulated, it also counted as public disclosure, which weakened their later attempts to patent the design. Von Neumann gave the idea its clearest and most general form; it had grown out of the whole team's discussions. Many historians therefore prefer to speak of the stored-program architecture and to credit the Moore School group as a whole.

The idea also had a parallel in England. In his 1936 paper, mentioned in Chapter 1, Alan Turing had described a universal machine that read its table of rules from the same tape as its data, which is the stored-program idea in theoretical form. In late 1945, working at the National Physical Laboratory near London, Turing wrote his own detailed design for a stored-program machine, the Automatic Computing Engine, or ACE. Building it went slowly, and a smaller version, the Pilot ACE, did not run until 1950.

Storing the program in memory changed three things. A new job no longer meant rewiring: it meant loading a different set of numbers, which took minutes. The machine could fetch its instructions from memory as fast as it fetched its data, so the program ran at the full speed of the electronics. And because instructions were now numbers in memory, a program could read, change or even produce other instructions as if they were data. That last point looked like a curiosity in 1945. It turned out to be the basis of every layer that the rest of this book describes.

Building a memory

The idea was easy to state and hard to build. A useful memory had to hold hundreds of numbers, keep them reliably, and hand any one of them to the processor in a small fraction of a second. Vacuum tubes were far too expensive and too power-hungry to use for every digit. The engineers of the late 1940s found two cheaper ways to remember a number, both of which now look improbable.

The first, proposed by Eckert, was the mercury delay line. A number was turned into a train of pulses, the pulses were sent as sound waves along a tube filled with mercury, and when they arrived at the far end they were amplified and sent round again. The numbers stayed in memory for as long as they kept circulating. The second was developed at the University of Manchester by Frederic (Freddie) Williams and Tom Kilburn. It stored each bit as a small spot of electric charge on the face of a cathode-ray tube, the kind of glass screen used in early televisions and radar sets. The spots faded within a fraction of a second, so the machine read them and wrote them back many times a second.

To test their tube, Williams, Kilburn and Geoff Tootill built a small computer around it, the Small-Scale Experimental Machine, soon known as the Manchester Baby. Its memory held just 32 numbers of 32 bits each. Its only arithmetic was subtraction. Programs were entered one bit at a time with a row of switches, and the results appeared as a pattern of dots on a display tube.

On 21 June 1948 the Baby ran a program that Kilburn had written: 17 instructions to find the largest number, other than 2^{18} (262,144) itself, that divides exactly into it. The program tried every number downwards from 262,143, testing each one by repeated subtraction. The answer, 131,072, is obvious to anyone who notices that it is half of 262,144, and that was partly the point: the calculation was long and the answer easy to check, so it was a good test of whether the memory could be trusted. It took 52 minutes and some 3.5 million uses of the memory. It was the first time a program held in a computer's electronic memory had run.

The Baby was a test machine. The first stored-program computer to give a regular service to its users was EDSAC, built at the University of Cambridge's Mathematical Laboratory by a team led by Maurice Wilkes, who had read von Neumann's draft and visited the Moore School in 1946. EDSAC used mercury delay lines. Its first program ran on 6 May 1949, and the laboratory's log recorded the event in one line.

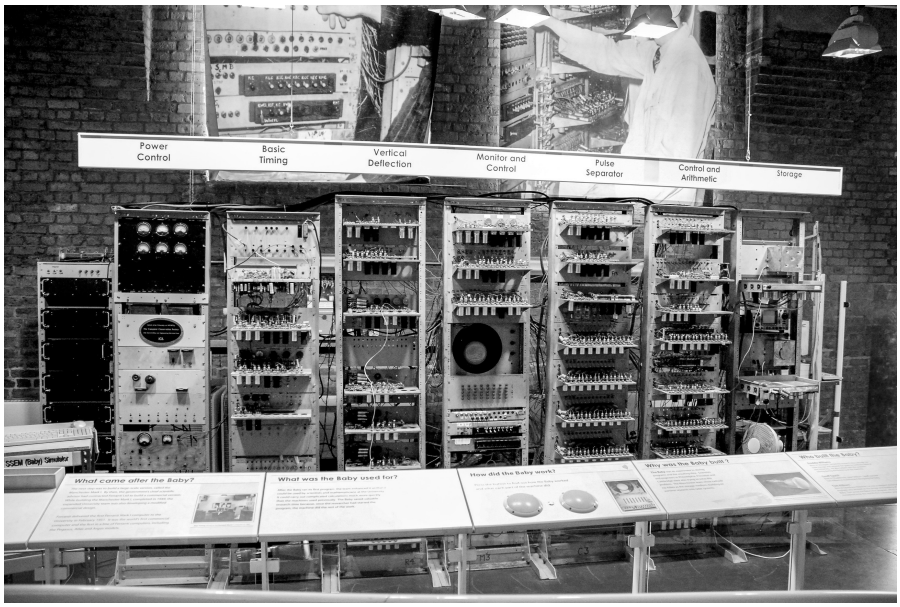


Figure 4.4 A working replica of the Manchester Baby in the Science and Industry Museum, Manchester. The original was a test machine with a memory of 32 numbers; on 21 June 1948 it ran the first program stored in an electronic memory.

Machine in operation for first time. Printed table of squares (0–99), time for programme 2 mins. 35 secs.

EDSAC LOG, CAMBRIDGE, 6 MAY 1949

Soon scientists across the university were bringing problems to it. Even ENIAC was adapted. From 1948 it was set up permanently to carry out a fixed set of instructions, which were selected by numbers held in its tables of switches. In effect it was turned into a simple stored-program machine, slower than before, but much quicker to give a new job. In the mid-1950s both delay lines and storage tubes were replaced by magnetic core memory, in which each bit was held by a tiny ring of magnetic material threaded on wires, and core memory was in turn replaced by the memory chips described in Chapter 3. The principle has not changed since 1948: program and data side by side, as numbers, in the same memory.

THE BUSINESS VIEW

A catering firm builds a computer

One of the first organisations to see what a stored-program computer could do for an ordinary business was J. Lyons and Company, a British firm best known for its tea shops, cakes and bread. Lyons already ran

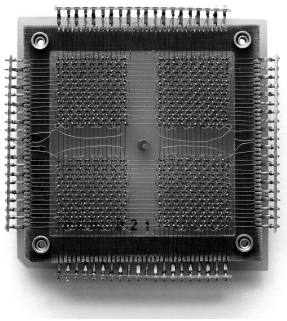


Figure 4.5 Magnetic core memory from 1961, holding 1,024 bits. Each bit is a tiny ring of magnetic material threaded on crossing wires.

large clerical departments and was interested in anything that would make office work cheaper. Two of its managers, visiting the United States in 1947, learned of the work at Cambridge. Lyons offered the EDSAC team money and staff in return for help, and then built its own machine on the same design. It called it LEO, for Lyons Electronic Office.

From 1951 LEO carried out routine office work for the company, starting with the valuation of the output of its bakeries and later including the payroll. It is usually described as the first computer used for the everyday administration of a business. Lyons later set up a company to build and sell LEO computers to other firms. The lesson was plain early on: once programs could be changed quickly, a single machine could do a company's sums one day and its wages the next, and the value lay as much in the programs as in the machine.

What an instruction looks like

A stored-program computer runs on numbers, and nothing else. Chapter 3 described its memory as a long row of numbered cells and its processor as a device that fetches an instruction from a cell, carries it out, and moves on to the next. The question this leaves is what the instruction in the cell actually is.

It is a number, and the number has parts. One part says which operation to carry out: load, add, store, compare, and so on. Each operation in the processor's repertoire has a number of its own, called its **operation code**. The other part usually says which cell of memory the operation should use, given as that cell's number, or address. An instruction is therefore an operation and an address packed into one number. A program written directly as such numbers is called **machine code**.

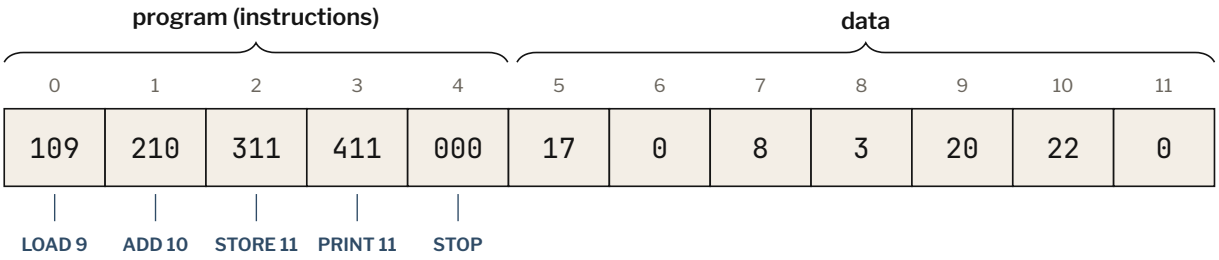
operation code The part of an instruction that says which operation to perform, such as add or store; often shortened to opcode.

machine code Instructions written as the numbers that a processor reads and carries out directly; the lowest layer of software.

To see how this works, it helps to use a toy machine, far simpler than any real one but built on the same plan. Its memory has 100 cells, numbered 0 to 99, and each cell holds a number. Its processor has a single working place for arithmetic, called the **accumulator**, which behaves like the display of a pocket calculator: it shows one number, and each operation changes it. The machine knows five instructions. Each is three digits long. The first digit is the operation code, and the last two digits are the address of a cell:

accumulator A working register in the processor in which a result is built up, one operation at a time.

- **1** (load): copy the number in the cell into the accumulator;



Instruction codes: first digit 1 = LOAD, 2 = ADD, 3 = STORE, 4 = PRINT, 0 = STOP; the last digits are an address (a cell number).

Every cell holds just a number. Only how it is used makes it an instruction or data.

Figure 4.6 Twelve cells of the toy machine's memory. Cells 0 to 4 hold a program and the rest hold data, but every cell holds only a number, and nothing in the memory marks which is which. Started at cell 0, this program adds the numbers in cells 9 and 10 (20 and 22), stores the result in cell 11 and prints 42.

- **2** (add): add the number in the cell to the accumulator;
- **3** (store): copy the accumulator into the cell;
- **4** (print): show the number in the cell on the screen;
- **0** (stop): stop.

So `109` means "load the number in cell 9", `210` means "add the number in cell 10", and `311` means "store the result in cell 11". The words load, add and store are for our benefit. The machine sees only the digits.

Figure 4.6 shows a short program in the toy machine's memory, and the most important thing about it is what it does not show. There is no label on any cell saying "instruction" or "number". Cell 1 holds `210`, which is an instruction because the processor will arrive there and carry it out. Cell 9 holds 20, which is data because an instruction tells the processor to load it. If the processor were sent to cell 9 instead, it would try to treat 20 as an instruction, and read it as "stop" with an address of 20. A number means what the program uses it to mean, which is the same lesson Chapter 2 drew about bits.

Figure 4.7 follows the toy machine through a smaller program, one that adds 3 and 8, step by step. At each step the processor fetches the number in the next cell, splits it into an operation and an address, and carries it out. The accumulator goes from 3 to 11, the result is stored, the screen shows 11, and the machine stops.

Real machines differ from the toy in size, not in kind. Their instructions are patterns of bits rather than decimal digits, as Chapter 2 would

Memory before the run

0	1	2	3	4	···	9	10	11
109	210	311	411	000		3	8	0

step	cell	number	meaning	accumulator
1	0	109	LOAD 9: copy cell 9 (3) into the accumulator	3
2	1	210	ADD 10: add cell 10 (8) to it	11
3	2	311	STORE 11: put it into cell 11	11
4	3	411	PRINT 11: show cell 11 on the screen	11
5	4	000	STOP	11

The screen shows **11**, the sum of 3 and 8.

Figure 4.7 The toy machine adding 3 and 8. Each row is one step: the processor fetches the number in the next cell, reads its first digit as the operation and the last two as an address, and carries it out. The accumulator column shows the running result.

lead you to expect: the Baby's were 32 bits long, and so are the instructions of the processors in most smartphones today. They also know many more operations. Most important among them is the jump, an instruction that tells the processor to take its next instruction from some other cell instead of the next one in line. A jump that depends on a test ("if the accumulator is negative, go to cell 7") lets a program choose between two courses, and a jump backwards lets it repeat a step. These are the selection and repetition of Chapter 1. Our toy machine leaves them out to stay small, so its programs can only run straight through from top to bottom.

Adding up a list in machine code

The task that recurs through this book is to add up a list of numbers and show the total. In the toy machine's code, for the list 3, 8 and 20, it looks like this. The left column gives the cell, the middle column the number stored in it, and the right column a note for the reader. Only the middle column is in the machine.

toy machine code (illustrative)

```
cell  number   meaning
  0     109    load cell 9 into the accumulator
  1     210    add cell 10
  2     211    add cell 11
  3     312    store the accumulator in cell 12
  4     412    print cell 12
  5     000    stop
  9       3    first number
 10       8    second number
```

```
11      20    third number
12       0    room for the total
```

The program begins at cell 0. The first instruction, `109`, puts 3 into the accumulator. The second, `210`, adds the 8 from cell 10, making 11. The third, `211`, adds the 20 from cell 11, making 31. The fourth, `312`, copies 31 into cell 12, and the fifth, `412`, shows the contents of cell 12 on the screen. The sixth, `000`, stops the machine. The screen shows 31. Cells 6 to 8 are left empty, which turns out to matter.

Six instructions to add three numbers is not bad. But the program can only add these three cells. A list of 50 numbers would need 50 instructions, one `2` for each. A real machine would use a jump to go round a loop instead, and on the earliest machines the usual way to step through a list was for the program to change one of its own instructions each time round: add 1 to the `210`, making it `211`, then `212`, and so on. The program treated its own instruction as a number and did arithmetic on it. Nothing could show more clearly that, in a stored-program computer, instructions and data are made of the same stuff.

Why it was hard

Writing the program above took a minute. Writing a real one in this way, hundreds or thousands of instructions long, was slow and exacting work, for reasons that the small example already shows.

Every address had to be worked out by hand. Suppose we want to add a fourth number to the list. It can go in cell 12, and the total can move to cell 13. But that means a new `212` must be inserted after cell 2, the store, print and stop instructions all move down one cell, and the store and print must be rewritten as `313` and `413`. In a long program, adding a single instruction near the beginning moved everything after it, and every instruction that referred to a moved cell had to be found and changed. If the program grew past cell 8, it would run into the data, and then the data would have to move, along with every address that pointed to it. Programmers left gaps, like the empty cells 6 to 8, to make room for changes, and kept careful written records of what was in every cell.

A single wrong digit could ruin the result without any warning. Suppose the second instruction is copied down as `201` instead of `210`, two digits swapped. The machine does not object. It obeys: it adds the contents of cell 1, which is the instruction `201` itself, treated as an ordinary

number. The accumulator goes from 3 to 204, then to 224 when the 20 is added, and the screen calmly shows 224. Nothing tells you that the answer is wrong except your own knowledge of what it should have been. In a program of a thousand numbers, the error could be anywhere, and finding it meant reading through sheets of digits and comparing them with what each one was supposed to mean. Maurice Wilkes later recalled realising, as EDSAC came into use in 1949, that a large part of the rest of his life would be spent finding mistakes in his own programs.

The work was also tied to one machine. Each design had its own operation codes, its own size of number and its own way of addressing memory, so a program for EDSAC was meaningless on the Baby. Programmers wrote their programs out on paper, often on ruled sheets with a column for each part of the instruction, then checked them, had them checked by a colleague, and converted them to the form the machine could read, such as holes in paper tape. Only a small number of people in the world could do it well, and they were in great demand.

AT THE KEYBOARD

Your prompt becomes numbers like these

When you ask an AI coding agent to build something in Volume 2, it will write code in a language such as Python or JavaScript, and you may never see anything lower. But that code does not run as written. It is translated, by programs that the next few chapters describe, until it becomes machine code: numbers in memory, each an operation and an address, fetched and carried out one after another by a processor, exactly as in the toy machine. The agent itself, the editor you type into and the browser that shows the result are all stored programs of the same kind, sitting in the same memory as your data.

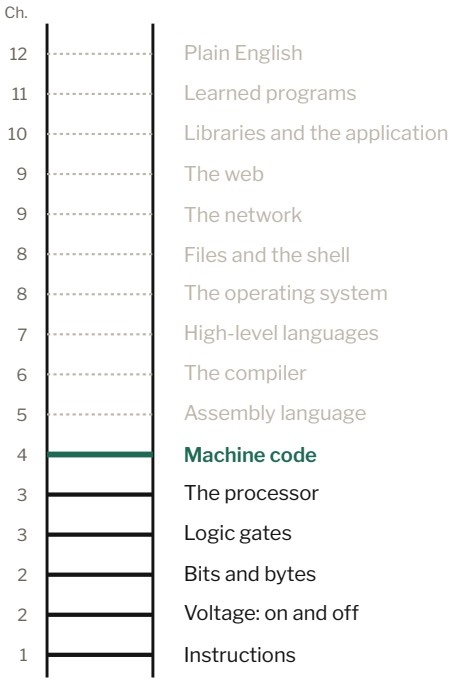
You will occasionally catch sight of this layer. An error message that mentions a “memory address”, or shows a long number such as `0x7ffee4b0c8`, is reporting a cell number like the toy machine’s two digits, written in hexadecimal and much longer. You will not need to read it. It is enough to know what kind of thing it is: a sign that something went wrong very close to the machine, which is usually a reason to give the whole message to the agent rather than guess.

Where this leaves us

We now have the machine that Babbage could not build and Turing described in theory: a computer that holds its program in memory, as numbers, beside the data. Changing its job means loading different numbers, and the same machine can do any task for which someone has written them. The program is machine code, each instruction an operation code and an address, and every program that runs today still reaches the processor in that form.

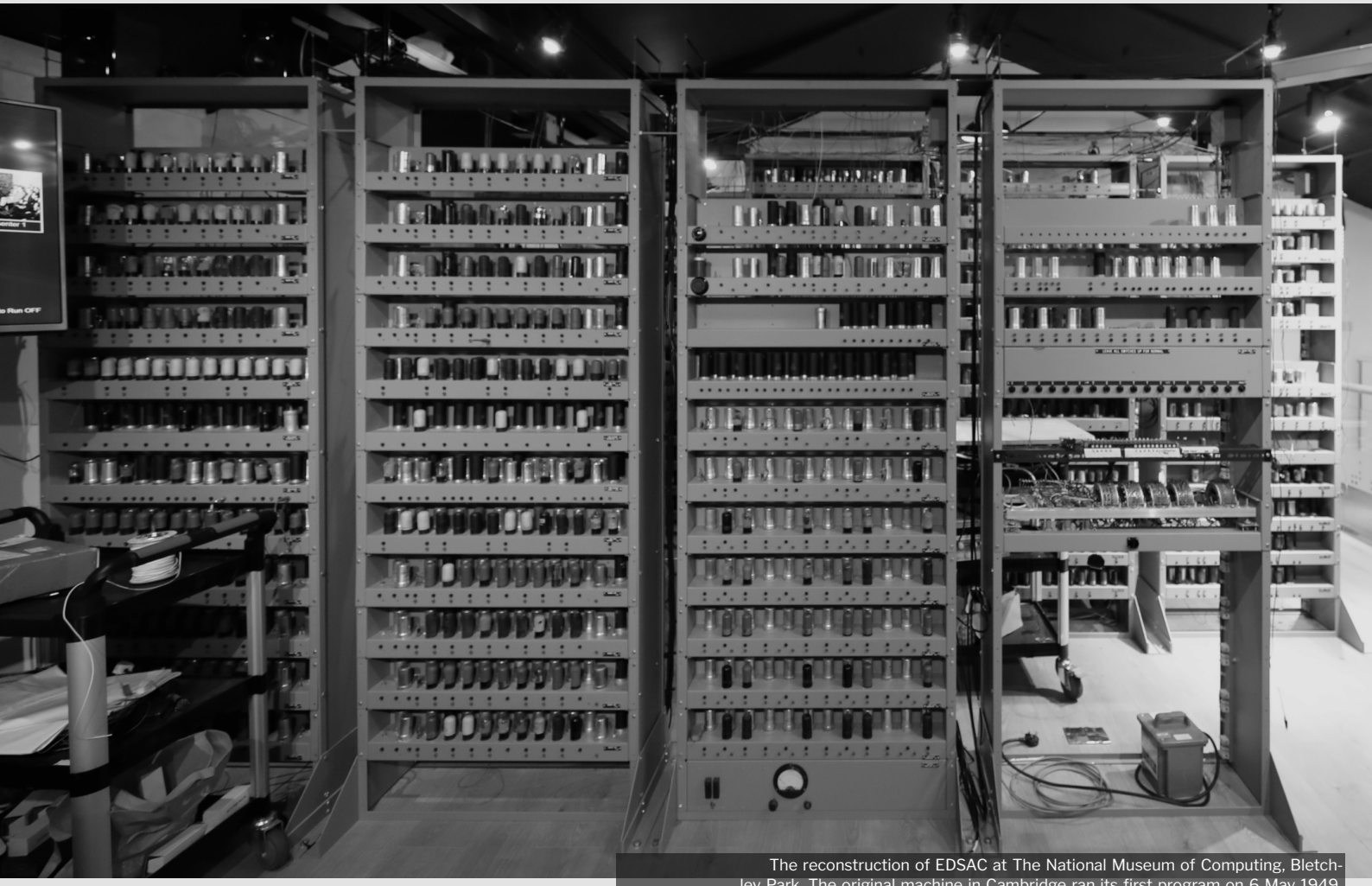
What we do not yet have is a comfortable way to write it. Programming in raw numbers works, as the Baby and EDSAC showed, but it is slow, every address must be tracked by hand, one wrong digit produces a wrong answer without complaint, and only a few specialists in the world could do it reliably. The ENIAC programmers had swapped cables for numbers; they had not yet been given anything easier than numbers.

The relief came from the stored-program idea itself. If instructions are numbers that a program can read and write, then a program can do the tedious parts of programming: look up the operation codes, keep track of the addresses, and produce the numbers from something easier for people to write. Chapter 5 describes the first programs that did this.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.



The reconstruction of EDSAC at The National Museum of Computing, Bletchley Park. The original machine in Cambridge ran its first program on 6 May 1949, and within months its users were writing programs in letters rather than binary.

CHAPTER FIVE

Assembly and the First Translators

Writing programs as raw numbers was slow and full of mistakes. The first relief came from giving each instruction a short name and letting the computer itself turn the names into numbers. With it came the first shared libraries of code, and the first patient hunt for errors.

BY THE SUMMER OF 1949 two machines in England could hold a program in their memory: the Manchester Baby, which had run its first program the year before, and EDSAC in Cambridge, which had started work in May. Chapter 4 described how such machines were programmed. Each instruction was a number. The person writing the program decided where in memory every instruction and every piece of data would go, worked out each number by hand, and punched the result onto paper tape. If a single digit was wrong, the machine did not object. It carried out a different instruction, or used a different cell, and produced a wrong answer or none at all.

This chapter describes the first layer that people built on top of those numbers. The idea was simple: let people write each instruction as a short name, and let the computer translate the names into numbers. It was a convenience, and a large one. It was also the first time a computer was used to prepare its own programs, and much of the rest of this book follows from that step. The same years produced two other habits that every programmer still depends on: reusing pieces of program that someone else has already written and tested, and searching methodically for the mistakes in one's own.

Names instead of numbers

Take the small program from Chapter 4 that adds 3 and 8 and shows the answer. To the toy machine it is five numbers: 109, 210, 311, 411 and 000. To read them, a person has to remember that a leading 1 means *load*, a 2 means *add*, a 3 means *store*, and so on, and that the last two digits are the number of a memory cell. With five instructions this is manageable. With five hundred it is not, and the real machines of the 1940s did not use tidy three-digit decimal codes. Their instructions were long strings of binary digits.

The obvious remedy is to write each operation as a short word that suggests what it does: `LOAD`, `ADD`, `STORE`, `PRINT`, `STOP`. A short name of this kind, chosen to be easy to remember, is called a **mnemonic** (from the Greek word for memory). A program written with mnemonics and ordinary numbers for addresses is said to be written in **assembly language**.

mnemonic A short, memorable name for a machine instruction, such as ADD or LOAD, used in place of its number.

assembly language A way of writing machine instructions with short names instead of numbers. Each line stands for one instruction.

Assembly language does not change what the machine does. Each line still stands for exactly one instruction, in the same order, with the same effect. It is a different way of writing down the same program, one that people can read. That turns out to matter a great deal. A programmer

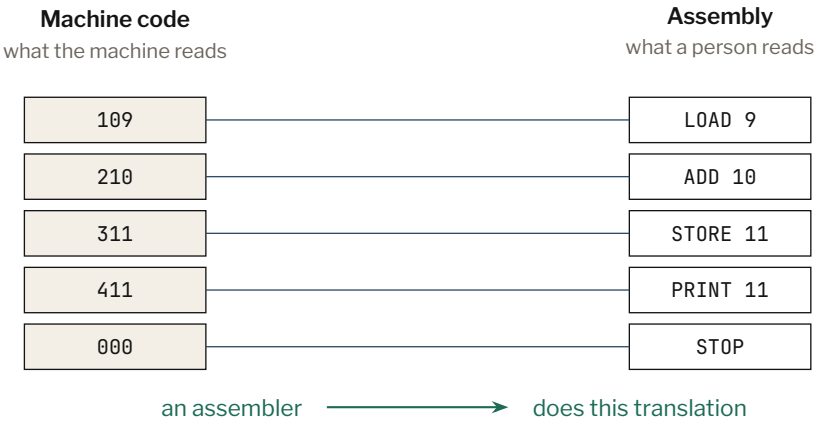


Figure 5.1 The program from Chapter 4 written two ways. On the left are the numbers the machine reads; on the right the same five instructions as a person would write them. Each line on the right stands for exactly one number on the left.

who sees `ADD 10` knows at a glance what the step is for; one who sees 210 has to decode it first, and is more likely to miss that it should have been 211.

One of the first people to set out such a notation was Kathleen Booth. Born Kathleen Britten in 1922, she worked with the engineer Andrew Booth at Birkbeck College, London, on an early computer called the ARC, the Automatic Relay Calculator. In 1947, during a visit to John von Neumann's group at the Institute for Advanced Study in Princeton, the two wrote a report called *Coding for A.R.C.* In it they described a compact symbolic way of writing the machine's instructions, which they called a contracted notation. It used symbols such as arrows to show numbers moving between memory and the machine's working registers. It was not written in words like `LOAD` or `MOV`, as some popular accounts suggest; those came later. But it is often cited as an early assembly language, and Booth as one of its first authors. She went on to a long career in computing and died in 2022, aged 100.

A notation on its own, though, solves only half the problem. Before the machine can run a program, somebody still has to turn the names back into numbers. If a person does it, looking up each name in a table and writing down its code, the notation helps with planning and reading, but the slow and error-prone copying remains. The real gain came when the translating was handed to the machine.

A program that translates programs

EDSAC, the Electronic Delay Storage Automatic Calculator, was built at the University of Cambridge Mathematical Laboratory by a team led by

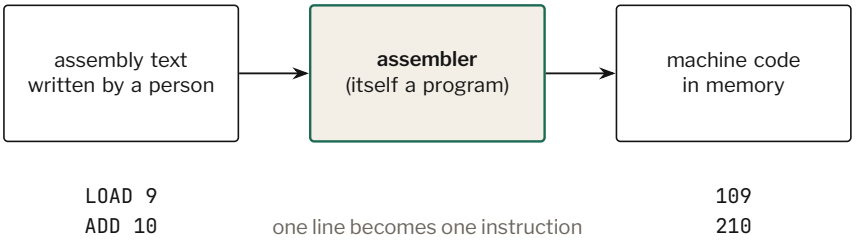


Figure 5.2 An assembler is itself a program. It reads text that a person has written and produces the numbers that the machine will run. Each line of assembly becomes one machine instruction.

Maurice Wilkes. On 6 May 1949 it printed a table of the squares of the numbers from 0 to 99, and from then on it was used to run other people's calculations. For that reason it is usually described as the first stored-program computer to give a regular computing service. The way its users wrote programs was shaped by a research student named David Wheeler.

Wheeler wrote what the Cambridge team called the *initial orders*: a short program built permanently into the machine and copied into its memory each time it was started. When an operator fed in a paper tape, the initial orders read it one character at a time. The tape did not hold binary. It held the program in a symbolic form: each instruction was a single letter for the operation, chosen to be easy to remember, followed by an address written in ordinary decimal digits. `A` meant add, `S` meant subtract and `T` meant transfer the result to memory. As each instruction came in, the initial orders worked out the binary number it stood for and placed it in the right cell. When the tape had been read, they handed control to the program they had just loaded.

The effect was that from 1949 EDSAC's users wrote their programs in letters and decimal numbers and never needed to write binary at all. A program that translates instructions written in this symbolic form into machine code is called an **assembler**. The word itself probably comes from a related job that such programs soon took on: gathering a complete program together, or assembling it, from a main part and pieces taken from a store of ready-made routines.

assembler A program that translates assembly language into machine code, usually one line into one instruction.

It is worth pausing on why this was possible at all. Chapter 4 showed that in a stored-program computer the program is simply numbers in memory, of the same kind as the data. One consequence is that a program can produce numbers that another program will then run. The assembler's output is the next program's instructions. For the first time, the computer was doing part of the work of preparing its own programs. The compilers described in Chapter 6, and the AI agents in Chapter 12, rest on exactly the same possibility: a program that writes or translates another program.

Wheeler soon improved his initial orders. A second version, introduced later in 1949, could also adjust the addresses in a piece of program

according to where in memory it was placed, so that the same piece could be loaded in different positions without being rewritten. Later assemblers, at Cambridge and elsewhere, took the idea further and allowed programmers to give names to memory cells as well as to operations. A name attached to a place in the program is called a label. Here is the program that adds 3, 8 and 20 written in that style for the toy machine:

toy machine (illustrative)

```
        LOAD  first     start with the first number
        ADD   second    add the second
        ADD   third     add the third
        STORE total     keep the result
        PRINT total     show it
        STOP
first:  3
second: 8
third:  20
total:  0
```

The programmer no longer says where anything goes. The assembler counts the lines, places the six instructions in cells 0 to 5 and the four numbers in cells 6 to 9, and then fills in the addresses: `LOAD first` becomes 106, `STORE total` becomes 309. The advantage shows when the program changes. If the programmer later inserts an extra instruction near the top, every cell below it moves down by one, and every address that referred to them must change. Done by hand, this meant renumbering the whole program and hoping nothing was missed, which is exactly the kind of silent error Chapter 4 described. With labels, the programmer inserts the line and runs the assembler again, and every address is recalculated correctly.

Write once, use many times

The programs that people brought to EDSAC had a lot in common. A physicist, a chemist and an economist might want quite different results, but all of them needed to print numbers in decimal, and many needed square roots, sines or logarithms along the way. Writing these pieces afresh for every program was wasteful, and each new version was a fresh chance to make a mistake. The Cambridge team's answer was to write each of them once, test it carefully, and let everyone use it.

A self-contained piece of program that does one job, and that a larger program can call on whenever it needs that job done, is called a **subroutine**. The idea is easy to state but raised a practical problem. A program

subroutine A self-contained piece of program that does one job and can be used from anywhere in a larger program.

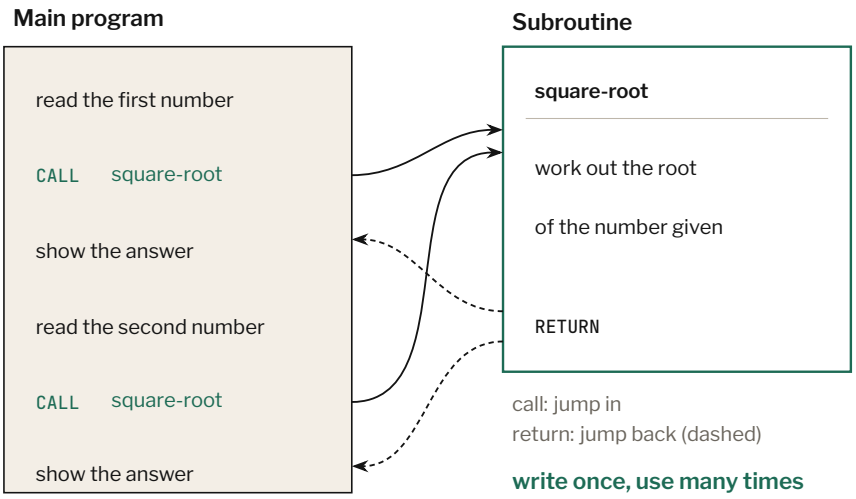


Figure 5.3 A subroutine is written once and called from wherever it is needed. Each call jumps into the subroutine; when it has finished, it jumps back to the place it was called from, which is different each time.

that wants a square root jumps to the subroutine, and the subroutine does its work. But then it has to jump back to the right place, which is different every time it is used. A square-root subroutine called from three places in a program has to return to three different places.

Wheeler's solution became known as the Wheeler jump. Just before jumping to the subroutine, the main program placed its own address in the machine's working register, as a note of where it had come from. The first thing the subroutine did was to take that note and use it to complete the last instruction of the subroutine itself, so that this final instruction became a jump back to the instruction after the call. The subroutine, in effect, rewrote its own ending each time it was used. It was only possible because instructions were numbers in memory that a program could change. Modern processors have special instructions for calling and returning, so programmers no longer need the trick, but the pattern it served is unchanged.

Over the following months the Cambridge team built up a collection of tested subroutines for common tasks, each kept on its own length of paper tape with a written description of what it did and how to use it. A programmer who needed one copied its tape onto the end of their own program, and the initial orders took care of fitting it in. A collection of ready-made, tested pieces of program that others can use is called a **library**, and EDSAC's was among the first.

library A collection of ready-made, tested pieces of program that other programmers can use in their own work.

In 1951 Wilkes, Wheeler and their colleague Stanley Gill described the whole system in a book, *The Preparation of Programs for an Electronic Digital Computer*. It is usually counted as the first textbook on programming, and it included listings of many of the library's subroutines so that people with other machines could learn from them. The same year Wheeler

was awarded a doctorate by Cambridge for his work, in what is often described as the first PhD in computer science.

The library mattered for more than convenience. It meant that a programmer could rely on work done by somebody else, without reading it line by line, because it had been tested and described. That is the beginning of a habit on which all modern software depends. Chapter 10 shows that a small app written today may draw on hundreds of libraries written by thousands of people. Each of them is a descendant of a length of paper tape in a Cambridge laboratory.

The same task in a modern assembly language

Assembly language is not a museum piece. Every processor made today has one, and the processors in phones and recent Apple computers, which follow a design called ARM, have their own. Below is the recurring task of this book, adding up a list of numbers, written in ARM assembly language for a modern Mac. Unlike the toy machine, a real processor can jump back to an earlier instruction, so this version uses a loop, as the recipe in Chapter 1 did. Everything after `//` on a line is a comment for human readers, which the assembler ignores.

sum.s

```
        .global _main
        .p2align 2
_main:  adrp  x1, numbers@PAGE         // find the list in memory
        add   x1, x1, numbers@PAGEOFF
        mov   w2, #3                   // three numbers to add
        mov   w0, #0                   // total = 0
loop:   ldr   w3, [x1], #4             // take the next number
        add   w0, w0, w3               // add it to the total
        subs  w2, w2, #1               // one fewer left
        b.ne  loop                     // if any remain, go back
        ret                            // hand back the total
        .data
numbers: .long 3, 8, 20
```

The names `x1`, `w0`, `w2` and `w3` refer to registers, the small, fast storage cells inside the processor described in Chapter 3. The first two working lines find where the list of numbers is in memory and keep that position in register `x1`. Register `w2` counts how many numbers are left, and `w0` holds the running total, which starts at zero. The four lines from the label `loop` are the heart of the program: take the next number from the list and

move along to the one after it, add it to the total, take one off the count, and, if the count has not yet reached zero, go back to `loop`. The instruction `b.ne` means "branch if not equal": jump if the last result was not zero. When the count reaches zero the jump is not taken, and `ret` ends the program, handing back whatever is in `w0`. The last line sets aside three cells of memory holding 3, 8 and 20, labelled `numbers`.

Here is the program being assembled and run in a terminal on a Mac. The first command assembles it and prepares it to run; the second runs it. This small program does not print anything itself. It hands its total back to the system when it finishes, and the third command asks for that number.

Terminal

```
$ cc sum.s -o sum
$ ./sum
$ echo $?
31
```

The answer is 31, the sum of 3, 8 and 20. Behind the first command, an assembler did what Wheeler's initial orders did in 1949: it turned each line into one instruction. The line `add w0, w0, w3`, for instance, became the 32-bit number 00001011000000110000000000000000 in binary, or 184,745,984 in decimal. The processor never sees the words. It sees only numbers like that one.

It is worth counting. Nine instructions were needed to add three numbers, and each one deals with registers, memory positions and jumps, the processor's own concerns rather than the problem's. In Chapter 6 the same task takes two lines.

Why programs go wrong

EDSAC's users soon discovered that writing a program was one thing and getting it right was another. Wilkes later recalled the moment, around June 1949, when this became clear to him. EDSAC was on the top floor of the laboratory and the equipment for punching and checking tapes was on the floor below, and it was while climbing the stairs between them that he realised how much of his time would go into correcting his own programs.

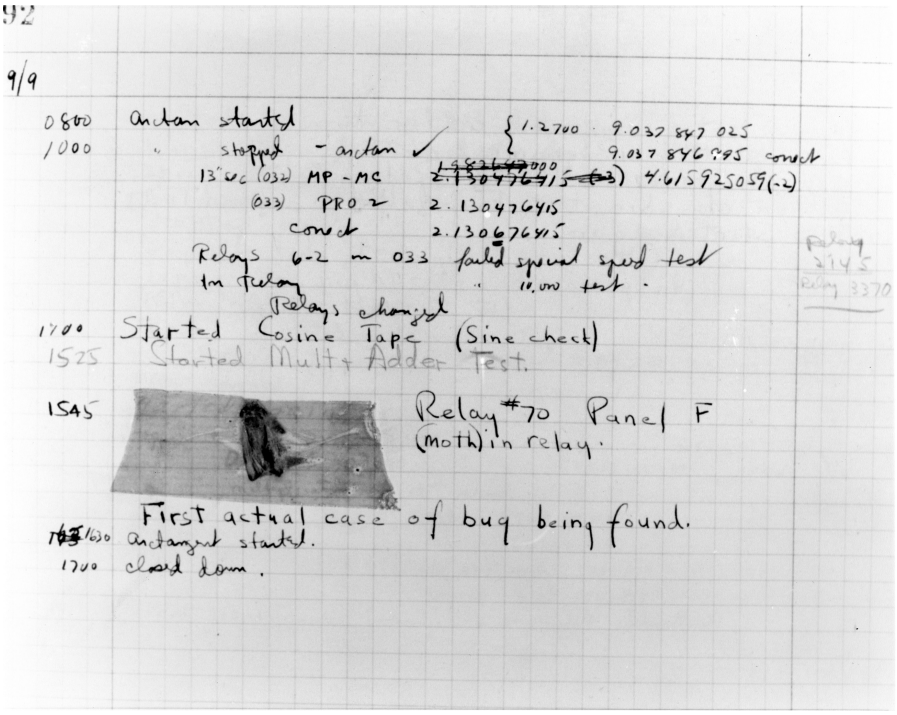


Figure 5.4 The page of the Harvard Mark II logbook for 9 September 1947, with the moth taped beside the entry for 15:45: "Relay #70 Panel F (moth) in relay. First actual case of bug being found." National Museum of American History, Washington.

The realization came over me with full force that a good part of the remainder of my life was going to be spent in finding errors in my own programs.

MAURICE WILKES, MEMOIRS OF A COMPUTER PIONEER, 1985

A mistake in a program that makes it behave wrongly is called a **bug**, and the work of finding and removing such mistakes is called **debugging**. Bugs have many sources. Some are slips: a wrong letter or digit punched into the tape. Some are errors of bookkeeping: an address that pointed to the right cell until the program was changed. Some are errors of logic: a loop that runs once too often, or a test that checks for the wrong thing. And some come from misunderstanding the problem itself, so that the program does correctly something that was not wanted.

bug A mistake in a program that makes it behave differently from what its author intended.

debugging Finding and removing the mistakes in a program, usually by observing what it actually does and comparing that with what was intended.

What they have in common is that the machine does exactly what it was told. A bug is not the computer misbehaving. It is the computer behaving precisely according to instructions that say something other than what their author meant, the gap between meaning and saying that Chapter 1 described. That is why bugs are hard to find. The program looks right to the person who wrote it, because they read it as what they intended,

and the only reliable way to find the difference is to watch what the program actually does.

The Cambridge team began to build tools for that purpose. Stanley Gill, for example, wrote programs that helped diagnose mistakes, including one that ran a faulty program step by step and printed a record of each instruction as it was carried out. Programmers still use the same approach today, with better tools: run the program, observe what it actually does at each step, and find the first point at which that departs from what was meant.

A CLOSER LOOK

The moth in Relay 70

The best-known bug in computing history was a real insect. On 9 September 1947 the operators of the Harvard Mark II, a large calculator built from electromechanical relays for the United States Navy, traced a fault to Relay 70 on Panel F. Caught between its contacts was a moth. They removed it, taped it into the machine's logbook beside the entry for 15:45, and wrote underneath: "First actual case of bug being found."

The logbook page survives in the Smithsonian's National Museum of American History in Washington. It is often said that Grace Hopper, who worked on the Mark II and later led the development of the first compilers (Chapter 6), found the moth, and that the episode gave us the word *bug*. Neither is right. The Smithsonian notes that the logbook was probably not Hopper's, and it is not recorded with certainty who removed the insect. And the joke in the entry, the word *actual*, only works because "bug" already meant a fault. Engineers had used it for decades. In 1878 Thomas Edison wrote in a letter that, with any new invention, " 'Bugs'—as such little faults and difficulties are called—show themselves", and that months of work were needed to deal with them. The word *debugging* was also in use among engineers by the mid-1940s.

What Hopper did was tell the story, often and well, in talks over many years, and it became part of computing folklore through her. Some later versions also gave the year as 1945; the logbook itself shows 1947.

AT THE KEYBOARD

Debugging is still the job

When you build software with an AI agent, the agent will write most of the code. It will also write bugs, for the same reasons people do: it fills gaps with guesses, misreads what was wanted, or makes a small slip that the computer then carries out faithfully. Some of these it will find and fix itself, by running the program and reading the error messages. Others only you will notice, because only you know what the program was meant to do.

The method has not changed since Cambridge in 1949. Say what you expected, observe what actually happened, and narrow down where the two part company. You will not be reading machine instructions, but you will be describing symptoms precisely, copying error messages in full, and checking one change at a time. Volume 2, Chapter 8, “When It Breaks”, turns this into a practical routine.

One line per step, one machine at a time

Assembly language and the assembler made programming faster and far less error-prone. Programmers no longer worked out binary codes or kept track of addresses by hand, and with libraries they no longer wrote every routine themselves. But two limits remained, and they were serious.

The first was that assembly still followed the machine one step at a time. Each line was one instruction, so the programmer still had to think in the processor’s terms: which register holds what, where in memory the data lies, when to jump and where to. A scientist who wanted to calculate a formula had to break it down by hand into a sequence of loads, multiplications, additions and stores, and keep track of every intermediate result. The ARM listing above needed nine instructions to add three numbers. A program that did something useful ran to thousands of lines, and writing it remained skilled, slow work for a small number of specialists.

The second limit was that assembly language belonged to one kind of machine. Its names were names for one processor’s instructions, and another processor had different instructions, different registers and different names. EDSAC’s letters meant nothing to the computers being built at Manchester. The ARM listing in this chapter will not assemble for the Intel and AMD processors found in most Windows computers, which have their own assembly language with instructions such as `mov eax, 0`. A pro-

gram written for one machine had to be rewritten, more or less from the beginning, to run on another. Every time an organisation bought a new computer, its programs had to be written again.

THE BUSINESS VIEW

The cost of starting again

In the 1950s and early 1960s, a firm that moved to a newer or larger computer usually found that its existing programs would not run on it. Payroll, accounts and stock control had to be rewritten for the new machine, at considerable cost in time and skilled staff. Customers came to see their programs as an investment that a change of hardware could wipe out.

That experience shaped the industry. When IBM announced its System/360 in April 1964, its central promise was a family of computers, from small to large, that all ran the same instructions, so that a customer could move to a bigger model without rewriting its programs. Keeping old software working on new hardware has been a commercial priority ever since. It is one reason why the processors in most of today's personal computers can still run instructions designed in the 1970s.

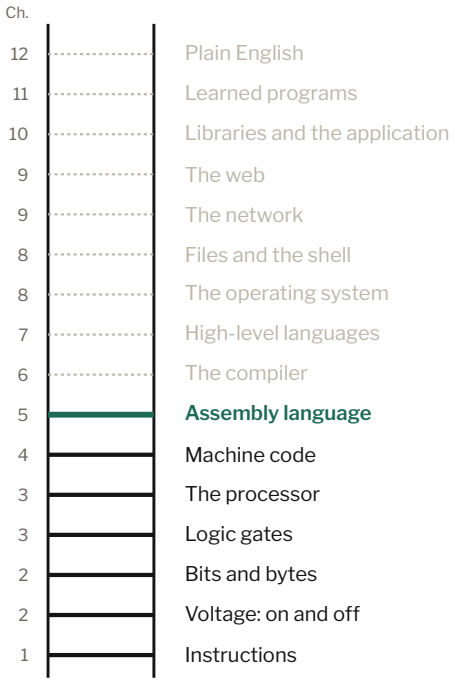
Assembly language is still written today, but only where its control over each instruction is needed: in the first instructions a computer runs when it is switched on, in small parts of operating systems, and in a few places where speed matters above everything, such as the inner loops of video software. Most of the assembly language that exists now is not written by people at all. It is produced by other programs, the compilers of the next chapter, and people read it mainly when they need to know exactly what the processor is being asked to do.

Where this leaves us

We now have the first layer in which a program, rather than a person, does part of the translating. Assembly language gives each machine instruction a name and each place in memory a label, and an assembler turns those names into the numbers the processor runs. From EDSAC onwards, programmers also had subroutines and libraries, so that a tested piece of work could be written once and used many times, and they had begun to treat the finding of mistakes as a regular part of the job.

What assembly does not change is the level at which a person has to think. It is still one line for every step the machine takes, written in terms of registers, addresses and jumps, and it is still tied to one kind of processor. A program written for one machine has to be written again for the next, and only specialists can write it.

The assembler did show that a program could translate another program. If a program could turn `ADD` into a number, perhaps a cleverer one could take something much closer to ordinary mathematical notation, a formula or even a sentence, and work out for itself the many machine instructions needed to carry it out. In the early 1950s a few people began to try. Chapter 6 describes what they built.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

Grace Hopper in January 1984, then a commodore in the US Navy. Three decades earlier she had written one of the first programs that turned other programs into machine code.

CHAPTER SIX

Compilers

In the early 1950s a few programmers asked whether the computer could do its own translating. The programs they wrote, called compilers, let people write formulas and words instead of the machine's numbers. Almost all software since has been written that way.

BY THE EARLY 1950S assembly language had made programming bearable, but only just. A programmer could write `ADD` instead of a number and let the machine look up the code, and could borrow tested subroutines from a library instead of writing everything afresh. But each line of assembly was still one step of the machine. Adding up a column of figures, working out a payroll or solving an engineering formula meant spelling out every load, every addition and every store, in the order the processor needed them, with every memory cell accounted for.

Two other problems grew as more computers were sold. The first was cost. Computers were expensive, but so were the people who programmed them, and a large program could take months to write and longer to get right. The second was that every program was tied to one model of machine. Assembly for a UNIVAC meant nothing to an IBM computer, and a company that changed suppliers, or even upgraded to a newer model from the same supplier, often had to write its programs again.

A computer already translated the names of instructions into numbers. The answer to both problems was to ask it to translate something much further from the machine, such as a formula or a sentence of English, into the many small steps the machine needed. At the time, the idea seemed odd to most people in the field.

A program that writes programs

Grace Hopper joined the Eckert–Mauchly Computer Corporation in Philadelphia in 1949. The company, founded by the two engineers who had built ENIAC, was making the UNIVAC, one of the first computers built to be sold. It was bought by the typewriter and office-equipment firm Remington Rand in 1950, and the first UNIVAC was delivered to the United States Census Bureau in 1951: the same customer that had first used Hollerith's punched-card machines sixty years earlier.

Hopper had been programming since 1944, when the US Navy sent her to work on the Mark I calculator at Harvard. Over those years she noticed how much of the work was copying. Programmers kept libraries of subroutines for common tasks, such as working out a square root or a sine, and built new programs largely by copying the right subroutines from the library, adjusting their memory addresses by hand so that they fitted together, and writing a little new code to join them up. The copying was

dull, slow and full of mistakes. It was also exactly the kind of mechanical, rule-following work that a computer does well.

In 1951 and 1952 she wrote a program for the UNIVAC that did the copying itself. A programmer gave it a list of the subroutines wanted, identified by call numbers, together with the values each should work on. The program found the subroutines on tape, fitted their addresses together and produced a complete program in machine code, ready to run. Hopper called it the A-0 system, and she described it in a paper, "The Education of a Computer", at a meeting of the Association for Computing Machinery in Pittsburgh in May 1952. Because it gathered material from a library into a finished whole, the way an editor compiles an anthology, she called it a compiler. A-0 did less than the word means today: by modern standards it was closer to a tool that links prepared pieces of code together. Its successors, A-1 and A-2, did more, and A-2 was given to UNIVAC customers in 1953.

The name stuck, and its meaning grew. A **compiler** is now any program that takes a program written in one language and translates the whole of it into machine code, which can then be run as often as needed. What made Hopper's work important was less the particular system than the principle behind it: that a program could take other programs as its material, and that much of the drudgery of programming could be handed to the machine.

compiler A program that translates a whole program from a programming language into machine code before it runs.

Not everyone was persuaded. Many programmers and managers of the time thought of a computer as a fast arithmetic machine. Letting it write its own instructions seemed either impossible or wasteful: surely a program written by a machine would be clumsy and slow compared with one written by a skilled person. Hopper recalled the reaction many years later.

I had a running compiler and nobody would touch it. They told me computers could only do arithmetic.

GRACE HOPPER, RECALLING THE EARLY 1950S, IN 1987

Others were trying similar ideas at about the same time, at the University of Manchester and at MIT, but none of these early systems was widely used. The one that settled the argument came from IBM.

A CLOSER LOOK

Grace Hopper

Grace Murray Hopper was born in New York in 1906. She studied mathematics and physics at Vassar College, earned a doctorate in mathematics at Yale in 1934, and taught at Vassar until the Second World War. In 1943 she joined the US Naval Reserve, and in 1944 she was sent to Harvard to work with Howard Aiken on the Mark I, a large electromechanical calculator built for the Navy. She became one of its first programmers and helped write its manual.

She was also on the Mark II team in September 1947, when its operators taped a moth into the log book. As Chapter 5 explained, she did not find it, but she told the story often and made it famous.

In 1949 she moved into industry, where she did the work described in this chapter. She stayed in the Naval Reserve, was recalled to active duty in 1967 to help the Navy standardise its programming languages, and finally retired in 1986, at 79, as a rear admiral. She died in 1992.

In her later years she lectured widely, often handing out pieces of wire about 30 centimetres long: the distance light travels in a billionth of a second. It was her way of showing why signals take time to cross a machine. Her larger argument, repeated for decades, was that computers should be made to fit the people who use them.

Formulas instead of instructions

In late 1953 John Backus, a programmer at IBM, proposed to his managers that the company build a system to let scientists and engineers write programs for its new IBM 704 computer in something close to ordinary mathematical notation. Backus and his small team spent more than three years on it. They called the language FORTRAN, short for *formula translation*, and the compiler was first delivered to IBM 704 customers in April 1957.

The idea was that a physicist or engineer should be able to write a calculation more or less as it appears on paper. The formula for one solution of a quadratic equation, which every engineering student learns, could be written in FORTRAN as `X = (-B + SQRT(B**2 - 4.0*A*C)) / (2.0*A)`, with `**` meaning “to the power of” and `*` meaning “multiply”. The compiler would work out the dozens of loads, multiplications, subtractions and stores that the machine needed, and in what order, and

which memory cells to use for the half-finished results along the way. The person writing the formula did not need to know.

A language of this kind, in which people write in terms of their problem rather than the machine's steps, is called a **high-level language**. "High" refers to the distance from the hardware, not to difficulty. The text a programmer writes in it is called **source code**; it is the source from which the machine code is produced.

high-level language A programming language that lets people write in terms of their problem, such as formulas or words, rather than the machine's steps.

source code The text of a program as a person writes it, in a programming language, before it is translated into machine code.

Here is the recurring task of this book, adding up a list of numbers, written in the style of early FORTRAN. It has been compiled and run with a modern FORTRAN compiler.

sum.f

```
C     ADD UP THREE NUMBERS AND PRINT THE TOTAL
      DIMENSION N(3)
      N(1) = 3
      N(2) = 8
      N(3) = 20
      ITOTAL = 0
      DO 10 I = 1, 3
   10 ITOTAL = ITOTAL + N(I)
      PRINT 20, ITOTAL
   20 FORMAT (I6)
      STOP
      END
```

The first line, beginning with `C`, is a comment for human readers, which the compiler ignores. `DIMENSION N(3)` sets aside room for a list of three numbers called `N`, and the next three lines fill it. The line beginning `DO 10` is the loop: it says "repeat the statement labelled 10 for `I` equal to 1, 2 and 3", and statement 10 adds the next number to the running total. `PRINT` shows the total, laid out according to the `FORMAT` statement labelled 20, which asks for a whole number in a space six characters wide.

The look of the program comes from the punched card. Each line was one card, and the language reserved fixed columns: the first five for statement labels such as `10` and `20`, the seventh onwards for the statement itself. That is why the lines are indented as they are. Everything is in capitals because the card punches and printers of the time had no lower-case letters.

Terminal

```
$ gfortran -std=legacy sum.f -o sumf
$ ./sumf
    31
```

The first command runs the compiler, `gfortran`, on the source file `sum.f` and asks it to write the machine code to a new file called `sumf`. The

second runs that file. The answer, 31, appears indented, in its space of six characters, as the `FORMAT` statement asked. Nothing in the program says which memory cells to use, which register to load or how the loop jumps back. The compiler decided all of that.

Backus's team knew that convenience alone would not be enough. The programmers who were expected to use FORTRAN were proud of the tight, fast code they wrote by hand, and the computers were so expensive that wasting their time was a serious matter. Backus later wrote that the team believed that if a compiled program ran "only half as fast as its hand coded counterpart, then acceptance of our system would be in serious danger". So most of their effort went into making the compiler's output efficient. It analysed the program, rearranged calculations, avoided repeating work and made careful use of the machine's few registers. A compiler that improves the code in this way as it translates is called an **optimising compiler**, and FORTRAN's was the first serious one.

optimising compiler A compiler that rearranges and simplifies a program as it translates it, so that the machine code runs faster.

It worked. In many cases the compiled programs ran nearly as fast as code written by hand, and they took a fraction of the time to write. That was the answer to the sceptics, and FORTRAN spread quickly among scientists and engineers. Updated many times, it is still used for weather forecasting, climate models and physics simulations, where large calculations written decades ago are still being run and improved.

Words for business

Formulas suited scientists and engineers. Most of the computers being sold in the 1950s, though, went to businesses and governments, and their work was different: payrolls, invoices, stock records, insurance policies. Such work involves little advanced mathematics and a great deal of reading records, comparing them, sorting them and printing them. Hopper believed that the people who did it would rather write in words than in symbols.

From 1955 her group at Remington Rand built a language on that principle. It was first called B-0, for "business language version 0", and was sold as FLOW-MATIC. Its statements were English words and phrases: a program might say `COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B)` to check two records against each other. FLOW-MATIC ran only on UNIVAC machines, but it showed that a compiler could take something resembling English and produce a working business program.

By 1959 there were several such languages, each tied to one manufacturer's computers. For large customers, above all the United States government, that was costly: every change of machine meant rewriting programs. In April 1959 Mary Hawes, a programmer at the Burroughs Corporation, brought together a small group at the University of Pennsylvania to propose a common business language that would work on any manufacturer's machines. The US Department of Defense agreed to sponsor the effort, and on 28 and 29 May 1959 about 40 people from computer makers, users and government met at the Pentagon. They formed a body called the Conference on Data Systems Languages, or CODASYL.

A short-range committee was asked to produce a language quickly. Its members included Jean Sammet of Sylvania, Gertrude Tierney of IBM and Hawes herself, and they drew heavily on FLOW-MATIC, with Hopper as a technical adviser. By December 1959 they had a specification, published in 1960. The language was named COBOL, for *common business-oriented language*. Hopper is often called its inventor; it is fairer to call her, as some do, its grandmother. The language was the work of a committee, and its main model was hers.

COBOL was meant to be read by people who were not programmers, such as the managers and auditors responsible for the work it did. A typical statement reads `MULTIPLY HOURS-WORKED BY HOURLY-RATE GIVING GROSS-PAY.` Anyone can follow that, and anyone who knows the business can check that it is right. The cost was length: COBOL programs are wordy, and critics have long complained that the English-like surface makes them look simpler than they are. But the design achieved its main aims. In December 1960 the same COBOL program was compiled and run on two different makes of computer, from RCA and from Remington Rand, with the same results.

That demonstration showed what high-level languages offered business beyond convenience. Source code written for one machine could be compiled for another, so long as someone had written a compiler for it. A program that can be moved between different kinds of computer in this way is called **portable**. The investment in a program no longer had to be thrown away when the hardware changed. With the US government, then the largest buyer of computers, pressing its suppliers to offer COBOL, it became the standard language of business data processing within a few years.

portable Able to run on different kinds of computer with little or no change to its source code.

THE BUSINESS VIEW

The COBOL that still runs

COBOL programs written decades ago still run much of the world's routine financial and administrative work: bank accounts, insurance policies, pensions, tax and benefit payments. Nobody knows exactly how much. Estimates of the amount of COBOL in use run into the hundreds of billions of lines. Some widely repeated figures, such as a 2017 Reuters estimate that COBOL handles 95 per cent of cash-machine transactions, rest on old and thin evidence, and the higher totals come from surveys paid for by companies that sell COBOL tools. What is not in doubt is that it remains central to many banks, insurers and government agencies.

Such systems survive because they work, and because replacing them is risky. A bank's core accounting program may hold decades of accumulated rules, many recorded nowhere else; rewriting it means rediscovering every one, and a mistake can stop payments to millions of customers. So the old programs keep running, while the people who can maintain them retire.

The risks show up in a crisis. In April 2020, when the pandemic sent unemployment claims soaring, several American states found that their benefit systems, some written in COBOL, could not be changed quickly, and the governor of New Jersey publicly appealed for volunteers who knew the language. Two decades earlier, checking old programs that stored years as two digits before the year 2000 arrived was a large business, much of it done by Indian IT services firms. The lesson for a manager is that software is an asset with a long life and a maintenance bill. Code written today, by hand or with an AI agent, may still be running long after its authors have moved on.

Translating in advance, or as you go

A compiler translates a whole program before it runs. There is another way to bring a high-level language to the machine. A program can read the source code a statement at a time and carry out each statement as it reads it, without ever producing a separate machine-code file. Such a program is called an **interpreter**.

interpreter A program that reads another program and carries out its statements directly, one at a time, each time it runs.

The difference resembles that between translating a book and interpreting at a meeting. A translator works through the whole book before it is published; readers then get the finished text and read it at their own speed, as often as they like. An interpreter at a meeting translates each

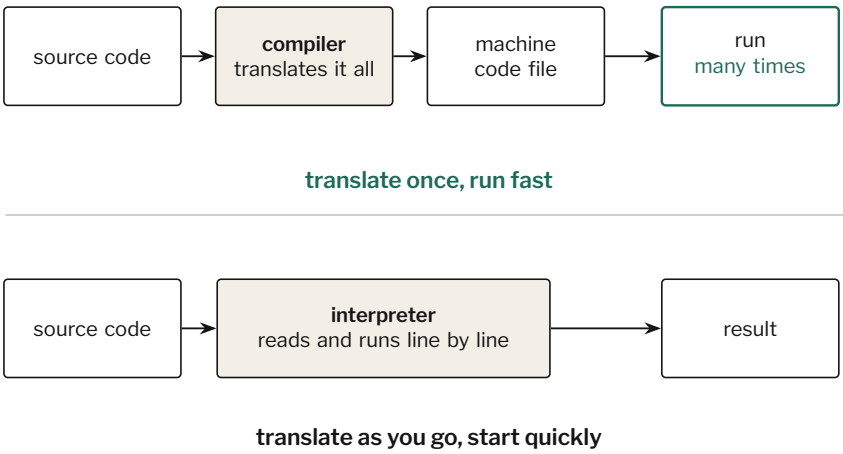


Figure 6.1 Two ways to run a high-level program. A compiler translates the whole program once into a machine-code file that can then be run many times without it. An interpreter reads the program and carries it out as it goes, so it is needed every time the program runs.

sentence as it is spoken. Nothing has to be prepared in advance and the conversation can change direction at any moment, but every sentence costs a little extra time, and the next meeting needs the interpreter again.

One of the first interpreters came out of a language designed for a very different purpose from FORTRAN or COBOL. In 1958 John McCarthy, at MIT, began work on LISP, a language for research in artificial intelligence, which works on lists of words and symbols rather than tables of numbers. In describing it, McCarthy wrote down on paper a definition of how any LISP expression should be evaluated. It was meant as a piece of theory. His student Steve Russell realised that the definition could be programmed directly, and did so, producing the first LISP interpreter. LISP introduced several ideas that later languages adopted, including the automatic recovery of memory that a program no longer needs.

The two approaches can be seen on a modern laptop. Here the add-up task is written in C, a compiled language, and in Python, which is usually run by an interpreter. (The two programs are shown in full in Figure 6.2, in the next section.)

Terminal

```
$ cc sum.c -o sum
$ ./sum
31
$ file sum
sum: Mach-O 64-bit executable arm64
$ python3 sum.py
31
```

The first command runs the C compiler, `cc`, which reads `sum.c` and writes the machine code into a new file called `sum`. The second runs that file. The third asks the computer what kind of file `sum` is; the answer says

that it is a program for the operating system of a Mac, in the machine code of an `arm64` processor, the kind in this laptop. That file would not run on a Windows PC with an Intel processor. The source file `sum.c` would, however: it could be compiled again there, by a compiler for that machine, which is what makes it portable. Python works differently. The last command gives the source file `sum.py` straight to the Python interpreter, which reads it and prints the answer. No machine-code file is left behind.

Compiled programs usually run faster, because the translation was done in advance, and the compiler can catch many mistakes before the program runs at all. Interpreted programs are quicker to try out: change a line and run it again at once. In practice most modern languages mix the two. Python quietly compiles each program into a simpler intermediate form, which its interpreter then runs; the JavaScript engines in web browsers start by interpreting a program and compile the busiest parts while it runs. The distinction remains a useful way to ask when the translation happens, and what it costs.

The same task at five levels

The task of adding up a list of numbers has now appeared in several forms in this book. Figure 6.2 puts five of them side by side, from the machine's numbers at the bottom to plain English at the top. All of them do the same thing: they add 3, 8 and 20, and show 31.

At the bottom is machine code for the toy machine used in Chapters 4 and 5. Cells 0 to 5 hold the program, cells 9 to 11 hold the numbers, and cell 12 has been left empty for the total. Every detail is explicit: which cell holds which number, the order of the loads and additions, where the answer goes. One level up, assembly language replaces each number with a name, `LOAD 9` instead of `109`, but it is still one line for each step of the machine, and the programmer still keeps track of the cells.

The C program is the first high-level version. It gives the list and the total names, `n` and `total`, and the compiler chooses where in memory they go. It describes the repetition as a loop ("for each `i` from 0 up to 3, add the `i`th number to the total") and leaves the compiler to work out the jumps. Python goes further. Adding up a list is so common that the language has a built-in way of doing it, `sum`, so the program only has to say what the numbers are and ask for their sum to be printed. At the top is the sentence a person would say to another person.

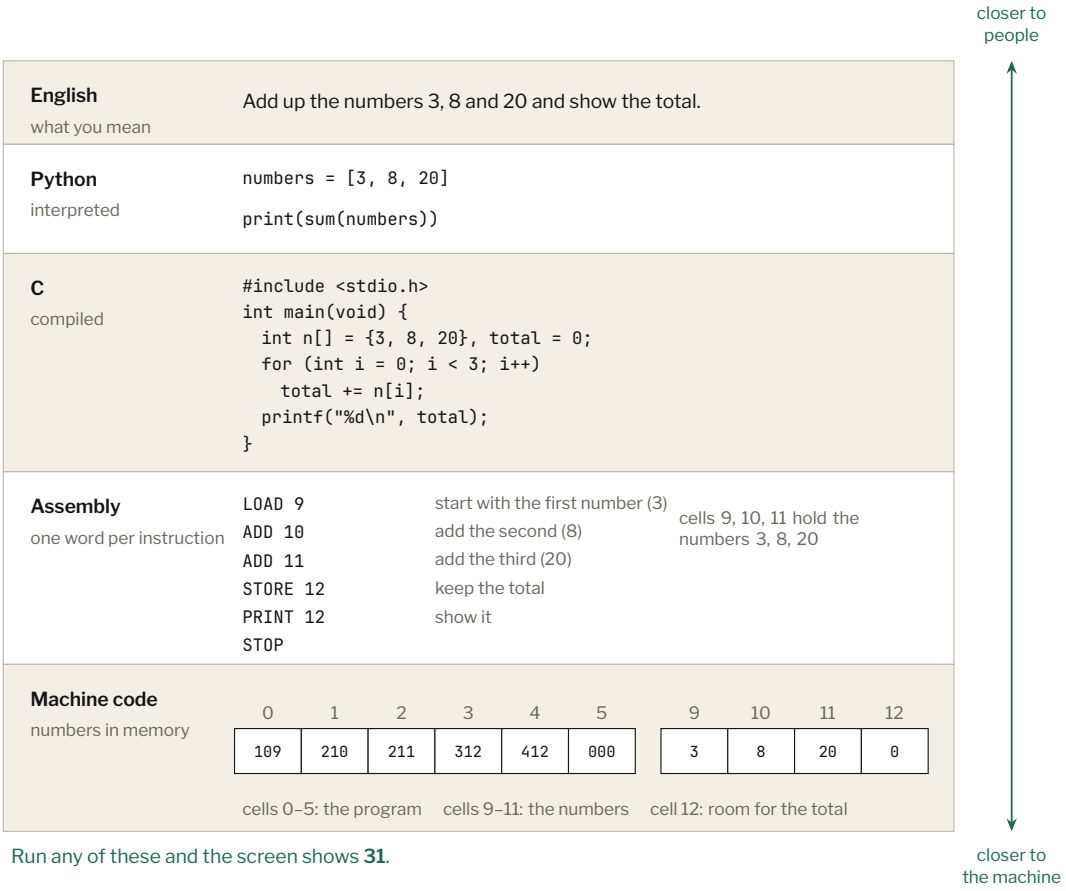


Figure 6.2 The same task at five levels. At the bottom, machine code for the toy machine of Chapter 4: numbers in numbered memory cells. Above it, the same program in assembly language, one named instruction per step. Then C and Python, two high-level languages, and at the top the request in English. Each level is shorter and closer to how a person would put it; each needs a program to translate it into the level below.

Two things change as the eye moves up the figure. The writing gets shorter and closer to how people think about the task, and more of the decisions are handed to another program. The assembler decides only the number for each name. The C compiler decides where values are kept and how the loop works. Python, and the people who wrote its `sum`, decide how a list is stored and added up. An AI agent given the English sentence would decide even which language to use. Each level is an abstraction in the sense of Chapter 1: a simpler way of saying the same thing, resting on a translator that supplies the detail.

The toy machine flatters the lower levels. Real processors need more instructions, and real programs carry bookkeeping for the operating system. On the laptop used for this book, the seven lines of C in the figure become about 80 lines of assembly language when the compiler is asked to show its work without improving it.

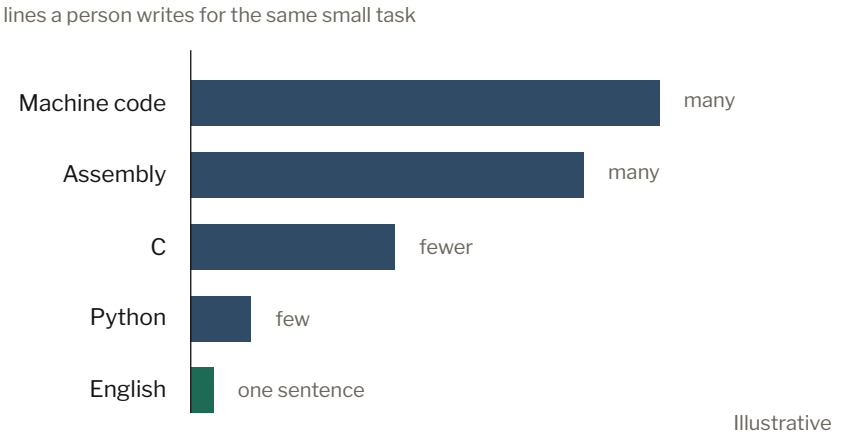


Figure 6.3 Each step up the ladder lets a person write less for the same task. The price is paid in speed and control: the more the translator decides, the less the writer knows about what the machine actually does. The chart is illustrative.

What the translation costs

Every layer in this book hides detail and charges something for doing so. The compiler's charges are of three kinds.

The first is speed. A program written by an expert directly in assembly language can sometimes be made faster than anything a compiler produces, because the expert knows things about the problem that the compiler cannot work out. Interpreted languages pay more: a simple loop in Python can run many times slower than the same loop in C. The second is control. A person writing assembly decides exactly what the machine does at each step; a person writing Python mostly does not know, and cannot easily find out. The third is distance. When something goes wrong deep inside, in the way memory is used or in how a number is rounded, the cause lies in a layer the programmer never wrote and may not understand.

Against this, the gains are large, and they have grown. Compilers improved steadily after FORTRAN, and a modern optimising compiler will often find shortcuts no person would bother with. Asked to translate the C program with its optimisations turned on, the compiler on this laptop noticed that the three numbers never change, did the addition itself, and left the machine code nothing to do but print the answer:

Terminal

```
$ cc -O2 -S sum.c
$ grep "#31" sum.s
        mov     w8, #31                         ; =0x1f
```

The option `-O2` asks for optimisation, and `-S` asks the compiler to stop at assembly language and save it in a file, `sum.s`, so that people can read it. The `grep` command searches that file for the number 31. The line it finds is an `arm64` instruction that puts 31 straight into a register. The loop, the additions and the list have all disappeared, because the compiler worked out the answer while translating.

The larger reason the trade was worth making lies in what became scarce. Computers grew faster and cheaper every year, as Chapter 2 described, while programmers did not. An hour of a person's time soon cost far more than an hour of the machine's, and a language that let one person write in an afternoon what had taken a week was worth a good deal of lost speed. High-level languages also let many more people program, including scientists, engineers and business analysts who had no wish to become specialists in one machine, and their programs could be read by colleagues and moved to new computers. Where every fraction of a second matters, as in parts of an operating system or a video game, programmers can still drop to a lower level. For almost everything else, the compiler's version is good enough, and far cheaper.

AT THE KEYBOARD

English as source code

When you ask an AI coding agent for a program, you are doing something Hopper's first users would recognise. You write in a language close to how you think, and a program translates it into a language closer to the machine, usually a high-level language such as Python or JavaScript, which a compiler or interpreter then translates further down.

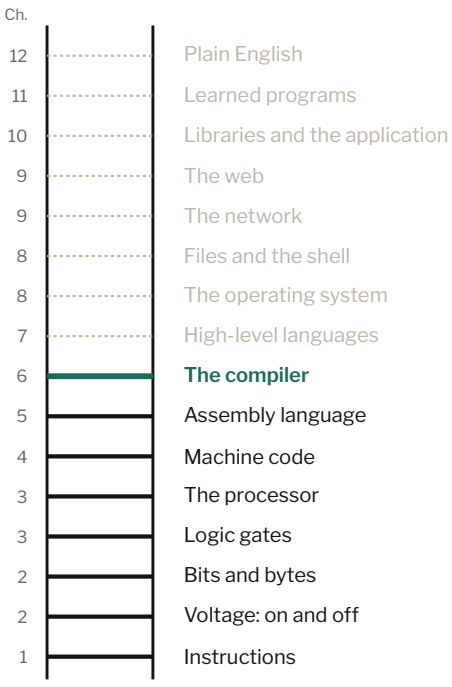
The resemblance has limits. A compiler follows fixed rules: the same source code always produces the same result, and if the source is not valid the compiler refuses it and says where the problem is. An AI agent works from patterns rather than rules. It accepts any English at all, fills the gaps with its best guess, may produce a different program if asked twice, and rarely refuses. So some of the checking a compiler once did for you, you now do yourself, by reading what the agent produced and testing whether it works. Chapters 11 and 12 explain why; Volume 2 is about doing it well.

Where this leaves us

We now have the layer that most programmers work at: high-level languages, and the compilers and interpreters that translate them into machine code. Hopper showed that a program could build other programs and argued, against much doubt, that the machine should do the clerical work of programming. Backus's team showed with FORTRAN that compiled programs could be nearly as fast as hand-written ones. COBOL showed that one program could run on different makes of computer, and that business logic could be written in something resembling English.

By 1960 there were already several such languages, each shaped by its purpose. FORTRAN served science and engineering, COBOL business records, and LISP research into artificial intelligence. An international committee had produced ALGOL, a carefully designed language that was influential mainly through the languages it inspired. Each made different trade-offs between speed, readability and convenience, and each suited some jobs better than others.

So the question for programmers changed. It was no longer whether to use a high-level language, but which one, and for what. The answers have multiplied ever since, and Chapter 7 looks at how the main families of languages arose and what each is good for.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

A Digital Equipment Corporation PDP-11/40. Unix and the C language grew up in the early 1970s on minicomputers of this family, small and cheap enough for one research group to have a machine of its own.

CHAPTER SEVEN

A Family Tree of Languages

By 1960 there were already several programming languages. Today there are hundreds. They differ because they were built for different jobs, by different people, at different times, and each gives up something to gain something else.

BY 1960 A PROGRAMMER had a choice of languages. FORTRAN served scientists and engineers who wanted to write formulas. COBOL served businesses that kept records of customers, accounts and stock. LISP served a small group of researchers working on artificial intelligence. A committee of European and American computer scientists had also just published ALGOL 60, a language designed to describe algorithms clearly. Few businesses used it, but its ideas shaped nearly every language that followed.

Since then the number has grown into the hundreds, and into the thousands if one counts languages built for a single company or a single product. To an outsider this can look wasteful. Why not settle on one good language and teach it to everyone? The reason is much the same as the reason there is more than one kind of vehicle. A bicycle, a lorry and an aircraft all move things from one place to another, but each is built around a different balance of cost, speed, load and the skill needed to drive it. Programming languages strike similar balances: between how fast the finished program runs and how fast it can be written; between control over the machine and protection from mistakes; between what suits an expert and what suits a beginner.

Languages also differ by era, because each was shaped by the machines and the economics of its time. In 1957 an hour of computer time cost far more than an hour of a programmer's time, so FORTRAN's designers worried above all that compiled programs should run nearly as fast as hand-written ones. By the 1990s the balance had reversed. Computers were cheap and programmers were expensive, and new languages willingly accepted slower programs in return for quicker writing and fewer mistakes.

On the surface languages differ most in their **syntax**: where the brackets go, how a line ends, which words have special meanings. Programmers argue about these choices with some passion. Underneath, though, nearly every language offers the three ingredients met in Chapter 1: sequence, repetition and selection. And languages borrow freely from one another. A good idea introduced in one tends to reappear in its successors, sometimes decades later, which is why their history is usually drawn as a family tree.

syntax The rules of spelling, punctuation and word order that a program must follow for a language's translator to accept it.

This chapter follows the main lines of descent, from the 1960s to the languages an AI agent is most likely to write for you today. It is not a complete history. It picks out the languages that introduced an idea still in use, and it shows the same two small tasks written in four of them, so that you can see what changes from one language to the next and what does not.

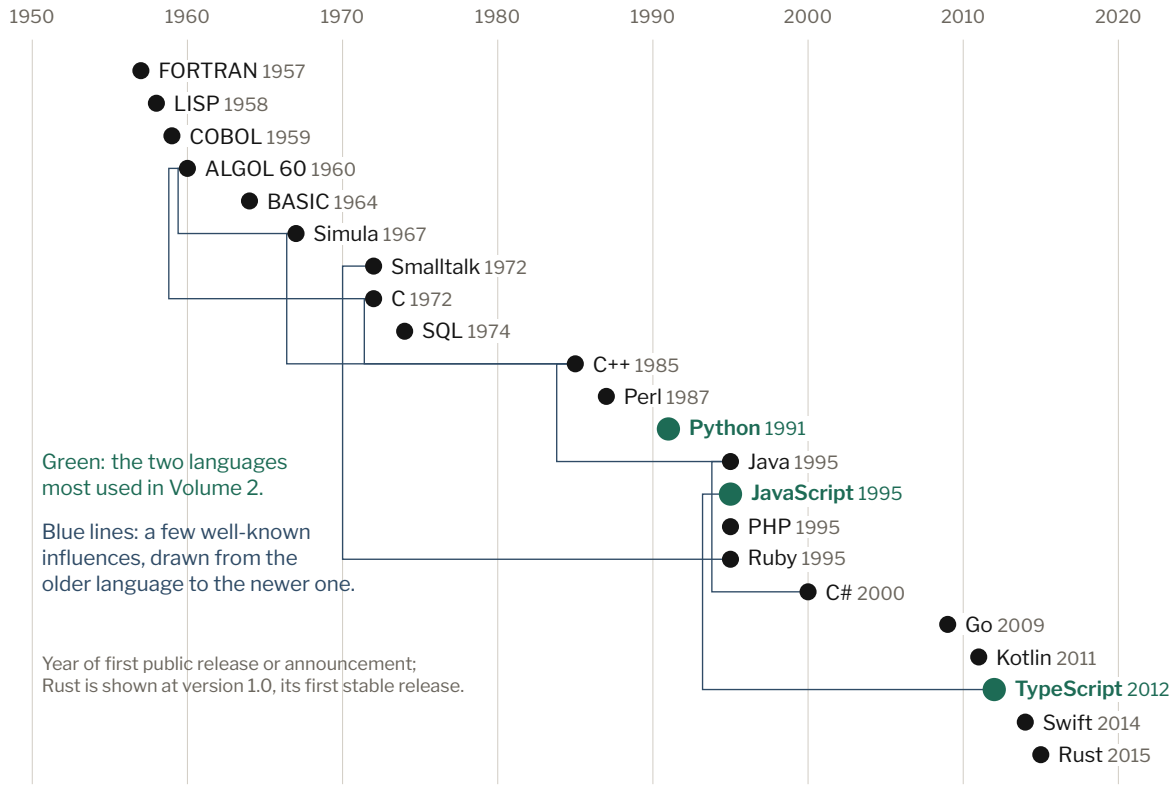


Figure 7.1 A family tree of programming languages, placed by year of first public release. Most new languages borrow from older ones: C took ideas from ALGOL, C++ from C and Simula, Java from C++, TypeScript from JavaScript. Python and JavaScript, with its typed relative TypeScript, are highlighted because they are the languages you are most likely to meet in Volume 2.

BASIC: programming for everyone

In the early 1960s using a computer usually meant handing a deck of punched cards to an operator and coming back hours later for the printed result, often to find that a typing error had stopped the program at the first line. John Kemeny and Thomas Kurtz, two mathematicians at Dartmouth College in New Hampshire, wanted every student at the college to be able to use a computer, not only those studying science. That needed two things. One was a way for many people to share one computer at the same time, each typing at a terminal and getting an answer in seconds. The other was a language simple enough to learn in an afternoon.

Both arrived in 1964. The Dartmouth Time-Sharing System let many users take turns on a General Electric computer so quickly that each seemed to have it to themselves. The language was BASIC, short for Beginner's All-purpose Symbolic Instruction Code. The first BASIC programs ran on 1 May 1964, early in the morning; college accounts put it

at about four o'clock. The first version had only 14 statements, most of them plain English words such as `PRINT`, `LET`, `IF`, `GOTO` and `END`, and every line of a program began with a number that fixed its place in the order.

BASIC's second life came with the small computers of the late 1970s and early 1980s. Almost every home computer of that period came with BASIC built in, so that switching the machine on led straight to a place where you could type a program. One of the first products of a small company called Microsoft, founded in 1975, was a version of BASIC for a hobbyists' computer, the Altair 8800. A generation of programmers wrote their first lines in BASIC, usually something like `10 PRINT "HELLO"`.

BASIC's lasting contribution was less a feature than an idea: that programming was not reserved for specialists. Professional programmers often looked down on it, and later languages replaced it. But every later attempt to make programming easier for non-specialists, from the spreadsheet to the AI agents described in Volume 2, belongs to the same line.

The software crisis

Through the 1960s computers grew more powerful, and the programs written for them grew much larger. An airline's booking system, a bank's accounts or the operating system for a large IBM computer could run to hundreds of thousands of instructions, written by hundreds of people over several years. Many such projects were delivered late, cost far more than planned, and were full of errors when they arrived. Frederick Brooks, who managed the development of IBM's System/360 computers and their operating system, later summed up one lesson from that experience in a sentence now known as Brooks's law: "Adding manpower to a late software project makes it later."

In October 1968 the NATO Science Committee brought about 50 experts from 11 countries to Garmisch, in southern Germany, to discuss the problem. The conference's title used a phrase that was still unusual: **software engineering**. Its report, edited by Peter Naur and Brian Randell, explained that the phrase had been chosen on purpose to provoke. The participants did not claim that programming already was a branch of engineering. They argued that it needed to become one, with the kind of methods, standards and accountability that bridge-builders and electrical engineers took for granted. The state of affairs they described came to be called the *software crisis*, a phrase that Edsger Dijkstra helped to popularise in a lecture in 1972.

software engineering The application of disciplined, measurable methods to the design, building, testing and maintenance of software.

The phrase 'software engineering' was deliberately chosen as being provocative ...

REPORT OF THE NATO SOFTWARE ENGINEERING CONFERENCE, 1968

Dijkstra, a Dutch computer scientist, had already made a narrower argument that turned out to be just as influential. In March 1968 he published a short letter in *Communications of the ACM* against the `GO TO` statement, which let a program jump from any line to any other. Programs full of such jumps were hard to follow, because the order in which their lines ran bore little relation to the order in which they were written. He began by observing that "the quality of programmers is a decreasing function of the density of go to statements in the programs they produce". His own title was "A Case against the GO TO Statement"; the journal's editor, Niklaus Wirth, gave it the one by which it is remembered, "Go To Statement Considered Harmful".

The alternative Dijkstra and others proposed became known as **structured programming**: build every program from a few well-behaved shapes, a run of steps in sequence, a loop that repeats, a choice between branches, each with one way in and one way out. Nearly every language designed since works this way, and the listings later in this chapter use nothing else.

structured programming Building programs only from blocks of steps, loops and choices, each with one way in and one way out, instead of free jumps.

A CLOSER LOOK

Margaret Hamilton and the Apollo software

Margaret Hamilton joined the Instrumentation Laboratory at the Massachusetts Institute of Technology in 1965. There she came to lead the team that wrote the on-board flight software for the Apollo command module and lunar module: the programs that guided the spacecraft to the Moon, down to its surface and back. The software ran on a small computer with very little memory, and a fault in it could cost the astronauts their lives. Hamilton's team put great effort into making it check itself and recover from trouble.

During the Apollo 11 landing on 20 July 1969, the guidance computer was given more work than it could handle and raised a series of alarms. The software had been designed to drop less important tasks in such a case and keep the essential ones running, and the landing went ahead.

Hamilton said she began calling the work "software engineering" in the mid-1960s, to win it the same respect as the hardware engineering around it. She was among the first to use and champion the term, though not the only one: Anthony Oettinger, the president of the Asso-

Figure 7.2 Margaret Hamilton at MIT in 1969, beside printed listings of the Apollo guidance software. The stack holds the programs for several missions, written by the team she led.

ciation for Computing Machinery, used it in print in 1966, and the NATO conference two years later made it standard.

The best-known photograph of her, taken at MIT in 1969 for publicity, shows her beside a stack of paper as tall as she is. The stack is printed listings of the guidance programs for several missions, not a single program for Apollo 11, and the code was written by a large team. Hamilton received the Presidential Medal of Freedom in 2016. She died in Cambridge, Massachusetts, on 30 September 2026, aged 90.

C and Unix

In 1969 Bell Labs, the research laboratory of the American telephone company AT&T, withdrew from Multics, a large joint project to build an ambitious system for sharing one computer among many users. Two of its researchers who had worked on Multics, Ken Thompson and Dennis Ritchie, wanted to keep some of its ideas on a smaller scale. On a little-

used Digital Equipment Corporation PDP-7, Thompson began to write a simpler system of his own, with Ritchie and others joining in. It became Unix, an operating system: the program that manages a computer on behalf of all the others, and the subject of Chapter 8.

Like nearly all operating systems at the time, Unix was first written in assembly language, which tied it to the PDP-7. When the group moved to a newer machine, the PDP-11, in 1970 and 1971, much of it had to be written again. To avoid doing that every time, Thompson and Ritchie wanted a language high-level enough to write the whole system in, yet close enough to the machine to do what an operating system must: move bytes around in memory, talk to disks and terminals, and run fast. Thompson first wrote a language called B, adapted from BCPL, which Martin Richards had designed at Cambridge in 1967. Ritchie then extended B into a new language, C. In his own account, "C came into being in the years 1969–1973"; the most creative period, he wrote, was 1972. In the summer of 1973 they rewrote the core of Unix in C.

That decision made Unix **portable**. Moving it to a different kind of computer no longer meant rewriting it. Someone had to write a C compiler for the new machine and adjust the small parts that depended on its hardware; the rest came across largely unchanged. Unix spread through universities and companies during the 1970s, and C spread with it, helped by a short and clear book, *The C Programming Language*, by Brian Kernighan and Ritchie, published in 1978. Its first example was a program that printed the words "hello, world". The habit of making that the first program anyone writes in a new language has lasted ever since.

portable Able to run on different kinds of computer with little or no change, usually by translating it again with a different compiler.

Both have long families. Linux, which runs most web servers and provides the core of Android phones, is a descendant of Unix in design, and Apple's macOS and iOS descend from a branch of it. Much of the software on which other software depends is still written in C or its close relatives: the cores of operating systems, databases, and the interpreter that runs Python programs. C gives the programmer close control and very little protection. A C program can read or overwrite any part of its own memory, and mistakes of that kind have caused a large share of the security flaws found in software. Newer languages such as Rust, whose first stable version appeared in 2015, try to keep C's speed while ruling such mistakes out.

Objects

As programs grew, a different question arose: how to organise them. Structured programming brought order to the flow of steps inside a program. It said less about how to divide a program of a million lines into parts that different people could build and understand separately.

One answer came from Oslo. At the Norwegian Computing Center in the 1960s, Ole-Johan Dahl and Kristen Nygaard were writing programs to simulate systems with many interacting parts. The natural way to describe such a system is in terms of the things in it, each with its own state and its own behaviour. Their language Simula 67, presented in 1967, let a programmer write a *class*, a description of one kind of thing, and then create as many **objects** of that class as were needed.

objects Bundles of data together with the actions that can be performed on it, handled as one unit in a program.

A bank shows the idea. Every account has an owner and a balance, and a few things can be done to it: pay money in, take money out, report the balance. In an object-oriented program the programmer writes one class, called `Account`, that says what every account holds and what can be done with it. Each real account is an object made from that description. The rest of the program never changes a balance directly. It asks an account to accept a deposit, and the account's own code checks that the amount makes sense before updating the balance. If the bank introduces a fee on withdrawals, the rule changes in one place. A savings account can then be defined as an account with one extra rule for interest, *inheriting* everything else.

In the 1970s, at Xerox's Palo Alto Research Center in California, Alan Kay, Dan Ingalls, Adele Goldberg and their colleagues took the idea further in Smalltalk, a language in which everything was an object. Kay coined the phrase "object-oriented programming" for this style. Smalltalk ran on the Xerox Alto, one of the first computers with windows and a mouse, and the two ideas grew up together. In 1979 Bjarne Stroustrup, a Danish computer scientist at Bell Labs, began adding Simula's classes to C. He called the result "C with Classes", renamed it C++ in 1983, and released it commercially in October 1985. C++ offered objects without giving up C's speed, and it became a main language for large programs in which speed matters: web browsers, video games, design software. Dahl and Nygaard received the Turing Award, computing's highest honour, in 2001.

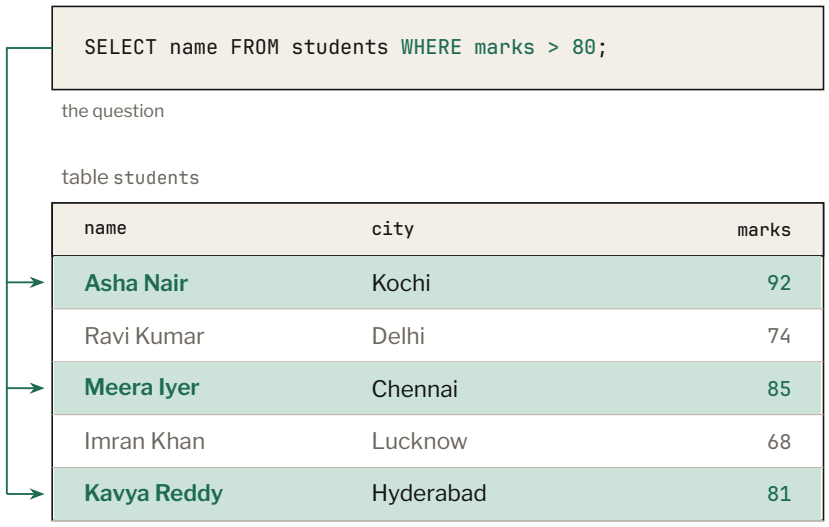


Figure 7.3 An SQL query and the table it asks about. The query says what is wanted (the names of students with more than 80 marks) and nothing about how to find them; the database works out the search itself and returns three names.

Asking a database

Not every language describes steps. In June 1970 Edgar F. Codd, an English-born mathematician working at IBM's research laboratory in San Jose, California, published a paper proposing a new way to store a business's data. Until then, getting information out of a company's records usually meant writing a program that walked through them one by one, following links from one record to the next in the order in which they happened to be stored. Codd proposed keeping data in simple tables of rows and columns, and letting people ask for what they wanted without saying how the computer should find it. A system that stores and retrieves data in this way is called a **database**.

database An organised store of data, managed by a program that lets people and other programs add, change and ask for it.

Codd's paper was mathematical. Turning it into something people could type was the work of two of his IBM colleagues, Donald Chamberlin and Raymond Boyce, who in 1974 described SEQUEL, a "Structured English Query Language". It was later renamed SQL because the name SEQUEL was already a trademark in Britain. The first commercial database to use SQL, Oracle Version 2, came out in 1979 from a small company in California that later took the product's name.

The query in the figure reads almost like an English sentence: select the name from the students table where the marks are above 80. There is no loop and no list of steps. The writer says *what* is wanted, and the database decides *how* to get it, which may be by checking every row or by

using an index, much as a reader uses the index of a book. A question of this kind is called a **query**. The recurring task of this book looks like this in SQL, once the numbers are in a table:

query A request for data, written in a language such as SQL, that describes what is wanted rather than how to find it.

sum.sql

```
CREATE TABLE numbers (n INTEGER);
INSERT INTO numbers VALUES (12), (7), (30), (5);
SELECT SUM(n) FROM numbers;
```

The first line creates a table with one column, the second puts four numbers into it, and the third asks for their total. Run through SQLite, a small database program found on most computers and phones, it prints `54`. More than 50 years after Codd's paper, SQL remains the standard way to ask questions of stored data. Most apps that remember anything, from a bank's systems to a food-delivery app, keep their data in a database that understands it.

Three languages of the 1990s

By the 1990s personal computers were in offices and homes, the internet was opening to the public, and programmers' time cost far more than the machines they worked on. Three languages born in that decade now do much of the world's everyday programming.

The first began as a holiday project. "In December 1989," Guido van Rossum later wrote, "I was looking for a 'hobby' programming project that would keep me occupied during the week around Christmas." He was working at CWI, the national research institute for mathematics and computer science in Amsterdam, where he had helped to build a teaching language called ABC. His new language, Python, was first released in February 1991. He named it after the British comedy series *Monty Python's Flying Circus*, not the snake. Python's guiding idea was that programs are read far more often than they are written, so they should be easy to read. It uses few symbols, and the way lines are indented is part of the language: steps that belong together are simply set in by the same amount. Python grew slowly, then quickly. In the 2010s it became the main language of data analysis and machine learning, and most of the research behind today's AI systems is written in it.

The second came from Sun Microsystems, a Californian maker of computers. In 1991 a team led by James Gosling began work on a language, first called Oak, for consumer devices such as television set-top boxes. Renamed Java, it was announced in May 1995, as the web was taking off,

under the slogan “write once, run anywhere”. A Java compiler does not translate a program into the machine code of any real processor. It translates it into the instructions of an imaginary computer, the Java **virtual machine**. Any real computer that has a program imitating that machine can then run the result unchanged. Java became a standard language for large business systems in banks, insurers and governments, and for many years it was the main language for Android apps.

virtual machine A program that imitates a computer, so that code written for the imaginary machine can run on any real one that has the imitator.

The third was written in a hurry. In May 1995 Brendan Eich, newly hired by Netscape, the company behind the most popular web browser of the day, wrote the first version of a small language for making web pages respond to the reader, in about ten days. Called Mocha and then LiveScript, it was renamed JavaScript in December 1995 as part of a marketing arrangement with Sun. Despite the name it is not a version of Java; the two are quite different languages. JavaScript became the one language that every web browser can run, which means that every interactive web page uses it. It has since spread to the servers behind websites as well. In 2012 Microsoft released TypeScript, which adds to JavaScript a way of declaring what kind of value each name will hold, so that many mistakes are caught before the program runs. Much of the code that AI agents write for the web today is TypeScript.

The same task in four languages

Here is the smallest program in each of two languages: one that does nothing but print a greeting. On the left is C, on the right Python.

hello.c

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world\n");
    return 0;
}
```

hello.py

```python
print("Hello, world")
```

The C version needs five lines of code for one action. The first asks for C’s standard collection of input and output tools; the third announces the part of the program, called `main`, where every C program starts; `printf` prints the text, with `\n` marking the end of the line; and `return 0` tells the operating system that the program finished without trouble. The Python version says only what is wanted. Both print the same words. The difference is the amount of the machine’s business that the language makes the programmer state.

The recurring task of this book, adding up a list of numbers, shows more. Here it is in C:

sum.c

```c
#include <stdio.h>

int main(void) {
    int numbers[] = {12, 7, 30, 5};
    int total = 0;
    for (int i = 0; i < 4; i++) {
        total = total + numbers[i];
    }
    printf("%d\n", total);
    return 0;
}
```

The word `int`, short for integer, appears four times: C insists on being told what kind of value each name holds, so that it can set aside exactly the right amount of memory. The `for` line keeps a counter, `i`, which runs from 0 up to 3 and is used to pick out each number in turn. The programmer has also had to write down that the list holds four numbers. If the `4` were mistyped as `5`, C would not object. It would add in whatever happened to be stored in memory just after the list, and print a wrong total. This is the lack of protection described earlier, in miniature.

Java needs a little more ceremony. Every Java program lives inside a class, here called `Sum`, and the line that begins `public static void main` plays the same part as C's `main`. But Java's loop no longer needs a counter or the length of the list: `for (int n : numbers)` means "for each number `n` in the list".

Sum.java

```java
public class Sum {
    public static void main(String[] args) {
        int[] numbers = {12, 7, 30, 5};
        int total = 0;
        for (int n : numbers) {
            total = total + n;
        }
        System.out.println(total);
    }
}
```

JavaScript and Python drop the declarations of type. In JavaScript the words `const` and `let` introduce a name, the first for one that will not change and the second for one that will. Python drops the braces and semicolons as well, and marks the repeated step simply by indenting it.

sum.js

```
const numbers = [12, 7, 30, 5];

let total = 0;

for (const n of numbers) {

  total = total + n;

}

console.log(total);
```

sum.py

```
numbers = [12, 7, 30, 5]

total = 0

for n in numbers:

    total = total + n

print(total)
```

All four produce the same answer. The C program must first be translated by a compiler into a separate file of machine code, here called `sum`, which is then run. The other three are handed straight to a program that translates and runs them in one go, as Chapter 6 described:

terminal

```
$ cc sum.c -o sum
$ ./sum
54
$ java Sum.java
54
$ node sum.js
54
$ python3 sum.py
54
```

Set side by side, the four listings show how much has changed since Chapter 4, where the same task took a page of numbered instructions, and how little. Each has a list, a running total that starts at zero, a loop that visits every number, and a final step that shows the result: the algorithm from Chapter 1, in four spellings. What the languages differ in is how much they ask the programmer to say about the machine. Python also comes with a great many ready-made pieces, and one of them does the whole job: `print(sum([12, 7, 30, 5]))` prints `54` too. Standing on code that others have already written is the subject of Chapter 10.

Choosing a language today

The table below lists the languages a business student is most likely to hear about, with what each is mostly used for now. The years are those of first public announcement or release.

LANGUAGES AT A GLANCE

Language	Year	Typical use today	Where you will meet it
COBOL	1959	Core systems of banks, insurers and governments	Behind your bank account; rarely written new
C	1972	Operating systems, devices, the insides of other languages	Underneath almost everything you use
SQL	1974	Storing data and asking questions of it	Any app that remembers anything
C++	1985	Browsers, games, fast financial and design software	Rarely, unless you build games
Python	1991	Data analysis, AI, automation, the server side of web apps	Data work and back ends in Volume 2
Java	1995	Large business systems; older Android apps	Corporate IT departments
JavaScript	1995	Everything that runs inside a web page	Every website you build
C#	2000	Business software on Microsoft systems; games	Firms built on Microsoft tools
Kotlin	2011	Android apps	If you build for Android
TypeScript	2012	JavaScript with declared types, for larger web apps	Most web apps an agent builds for you
Swift	2014	iPhone and Mac apps	If you build for Apple devices

Which languages are most popular depends on what is measured, and every measure changes from year to year. The TIOBE index, which counts how often languages are searched for on the web, placed Python first in September 2026, followed by C, C++, Java, C# and JavaScript. In Stack Overflow's survey of developers published in July 2025, the language the most respondents said they had used heavily in the past year was JavaScript, at 66 per cent; SQL and Python each stood at about 58 per cent, with Python up seven points on the year before. GitHub, which hosts much of the world's shared code, reported that in August 2025 TypeScript overtook both Python and JavaScript as the language used by the most contributors on its site. The lists disagree on order, but the same names appear near the top of all of them: Python, JavaScript and TypeScript, Java, C and C++, C# and SQL.

THE BUSINESS VIEW

Choosing a language is a business decision

For a company, a programming language is less a matter of taste than of long-term cost. A system written today may still be running in 20 or 30 years, as many COBOL systems written in the 1970s still are, and someone will have to maintain it. The questions that matter are practical. How many people can we hire who know this language, and at what salary? How many ready-made libraries and tools exist for it, so that we do not have to write everything ourselves? Will it still be supported in ten years? Popular languages win on all three, which is one reason popular languages stay popular.

AI agents add a new consideration. They write best in the languages for which they have seen the most examples, which are the most widely used ones. Choosing an unusual language may mean an agent that makes more mistakes, and fewer people able to check its work.

AT THE KEYBOARD

Which language should my app use?

When you start a project in Volume 2, you will seldom pick a language by taste. You will describe what you want, and the agent will propose a set of languages and tools. For a web app this is usually TypeScript or JavaScript for the part that runs in the browser, Python or TypeScript for the part that runs on the server, and SQL for the data.

Your job is to ask the agent to explain its choice in plain terms, to prefer common, well-supported options over fashionable ones, and to stay consistent once a choice is made. Chapter 5 of Volume 2 covers how to make that decision.

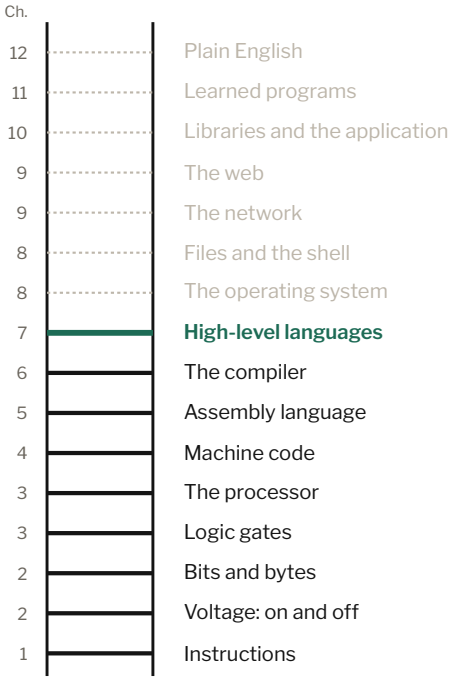
Where this leaves us

We now have the layer that most programmers work in: high-level languages, built on the compilers and interpreters of Chapter 6. There are hundreds of them because they serve different jobs, from C's close control of the machine to SQL's questions about data and Python's readable scripts, and because each was shaped by the computers and the costs of its time. They share an ancestry and the same three basic shapes of

sequence, repetition and selection, and a person who has seen one can usually recognise what another is doing.

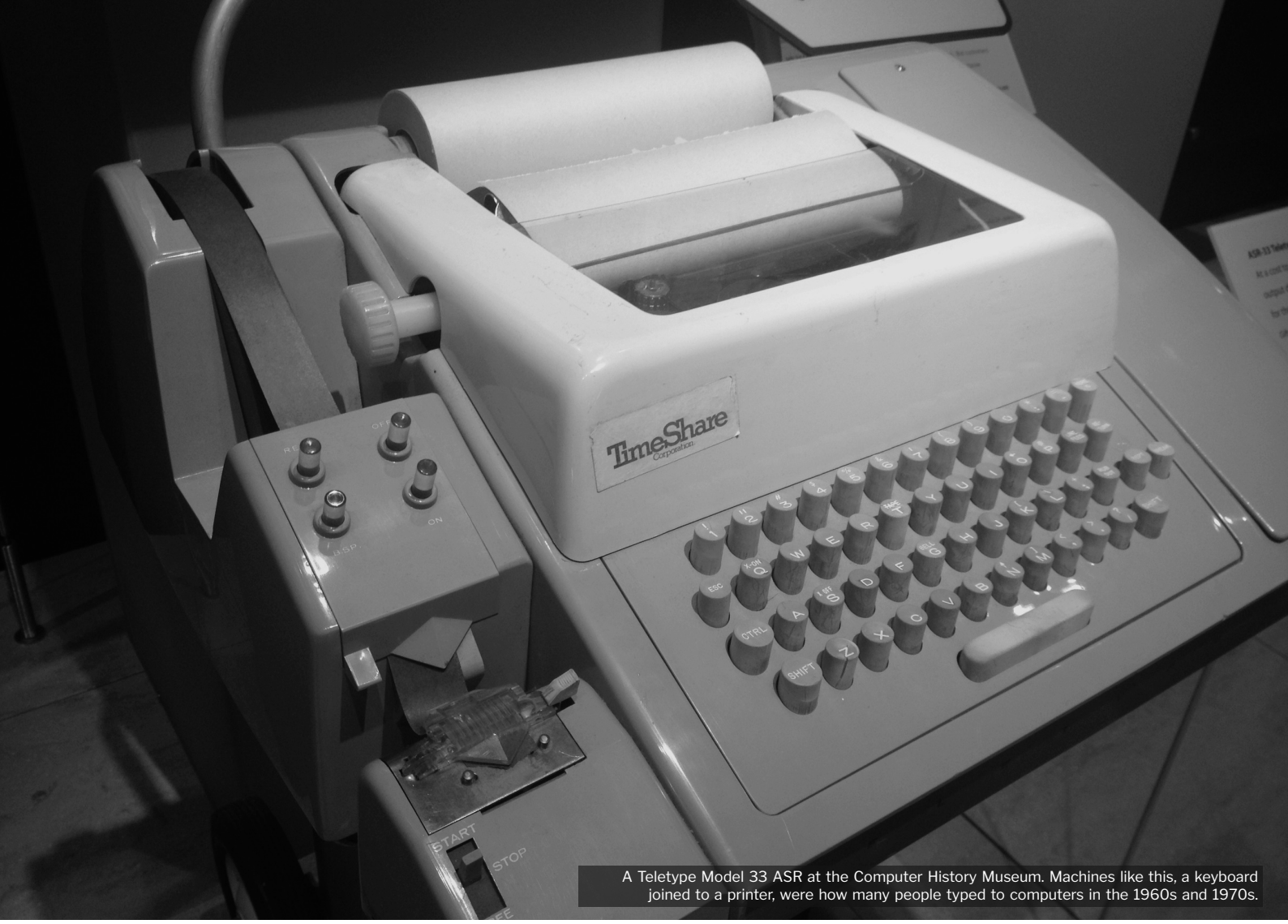
Languages are tools for people. The machine still sees only the numbered instructions of Chapter 4; what a language changes is how much the programmer must say, and how many mistakes the translator can catch before the program runs.

What a program written in any of these languages still needs is a home on the machine. It has to be stored somewhere, as a file with a name. Something has to start it, give it memory and a share of the processor, and stop it from interfering with the other programs running at the same time. On Unix that work was done by the operating system that C was invented to write, and every computer since has had one. Chapter 8 describes it, together with the files it keeps and the terminal through which people, and now AI agents, give it commands.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.



A Teletype Model 33 ASR at the Computer History Museum. Machines like this, a keyboard joined to a printer, were how many people typed to computers in the 1960s and 1970s.

CHAPTER EIGHT

The Operating System, Files and the Terminal

A program needs somewhere to live and someone to start it. That is the job of the operating system, the program that manages the machine on behalf of all the others. For decades people spoke to it by typing, first on paper and then on a screen, and many still do.

WHEN YOU DOUBLE-CLICK A SPREADSHEET on a laptop, several things happen that you never see. Something finds the file on the disk, among hundreds of thousands of others. Something works out which program should open it, loads that program into memory and starts it running. And while the spreadsheet opens, music keeps playing in another window, messages keep arriving, and the clock in the corner keeps ticking. One processor, as Chapter 3 described, follows one stream of instructions at a time. Something must be sharing it out.

That something is the operating system. The previous chapters were about how instructions are written: in numbers, in assembly, in languages that a compiler or interpreter translates. This chapter is about where those instructions live once they are written, and how they get to run. It covers the operating system itself, the files and folders it keeps, and the oldest way of talking to it that is still in daily use: typing commands into a terminal. That last one matters for this course, because it is where you will meet an AI coding agent in Volume 2.

The manager of the machine

The first electronic computers ran one program at a time, and people did the managing. An operator loaded a program, often from a deck of punched cards, waited for it to finish, collected the printed results and loaded the next. If the program needed to read a tape or print a line, it had to contain its own instructions for driving the tape unit or the printer. Every programmer wrote, again and again, the same tedious code for the same devices.

During the 1950s and 1960s much of this routine work was handed to a program that stayed in the machine permanently and looked after the others. It loaded the next job, supplied ready-made routines for reading and printing, and kept a log of what each job had used so that departments could be billed. In the 1960s such programs learned to switch the processor rapidly between several users, each at their own keyboard, so that each of them seemed to have the machine to themselves. This was called time-sharing, and it changed how people worked with computers: instead of handing in a deck of cards and coming back the next day, they typed and the machine answered.

A program that runs all the time and manages the computer on behalf of every other program is called an **operating system.** Windows, macOS,

operating system The program that runs all the time and manages the processor, memory, files and devices on behalf of other programs.

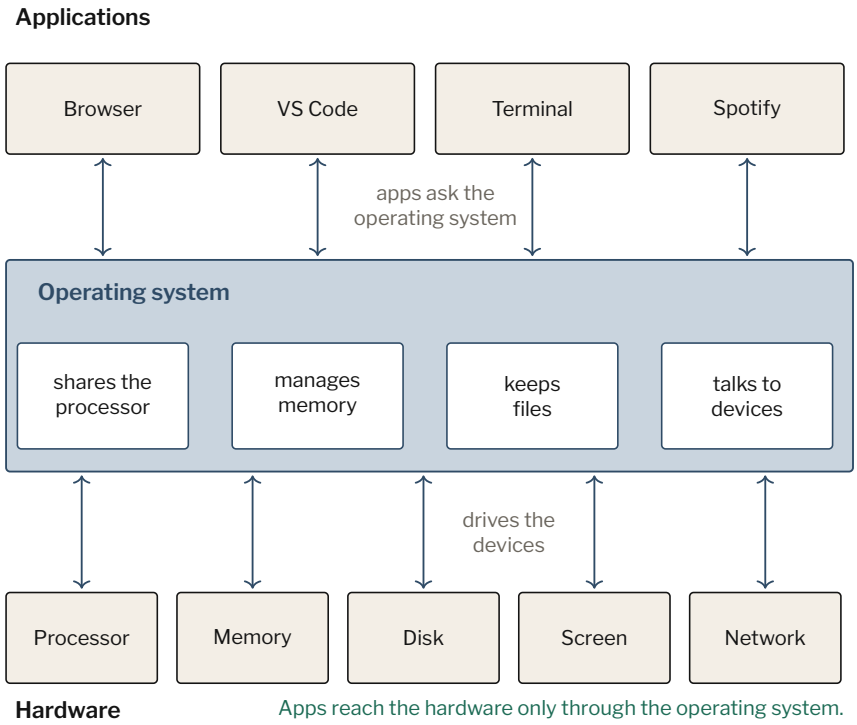


Figure 8.1 The operating system sits between the applications and the hardware. A browser or a music player never drives the disk or the screen directly; it asks the operating system, which shares the hardware fairly among all the programs that want it.

Linux, Android and iOS are operating systems. Their designs differ, but they do the same four main jobs.

The first is sharing the processor. A laptop may have dozens of programs open, and only a handful of processor cores to run them. The operating system gives each program a short turn, many times a second, and switches between them so quickly that they all appear to run at once. The second is managing memory. Each program is given its own area of memory and is stopped if it tries to read or write anyone else's, so that a fault in one program cannot quietly corrupt another. The third is keeping files: turning a disk, which is only a very large number of numbered blocks, into the named documents and folders that people recognise. The fourth is talking to devices. Every keyboard, screen, printer and network card works differently, and the operating system contains small programs, called drivers, that know the details of each one.

Programs reach these services by asking. When a word processor wants to save a document, it does not instruct the disk to move its heads or the flash memory to store particular bits. It asks the operating system to write these bytes to a file with this name, and the operating system does the rest. Such a request is called a system call, and an operating system offers a few hundred of them: open a file, read from it, start a program, send data over the network. This is the layer that lets one line

of code stand on millions written by others. A student writing a ten-line program in Python can print, save and connect to the internet because the operating system has already written the code that does those things, for every device the computer might have.

The best-known operating system in the history of software is Unix, begun in 1969 at Bell Labs in New Jersey by Ken Thompson and Dennis Ritchie, with others. Chapter 7 described how they rewrote it in C in 1973, which let it be moved to other makes of computer. Its ideas spread so widely that most of the computers in the world now run one of its descendants or imitations. Linux, which runs most web servers and, inside Android, most phones, follows its design; so does macOS, which grew from a branch of Unix, and so does iOS. Much of what this chapter shows comes from Unix, and works the same way on a Mac or a Linux machine today.

Files, folders and paths

A disk, whether a spinning magnetic one or the flash memory in a phone, knows nothing about documents. It stores bytes in numbered blocks, and it will hand back the bytes in block 4,172,905 if asked. Remembering that a particular report begins at that block, carries on in three others, is called `report.pdf` and was last changed on Tuesday is the operating system's work.

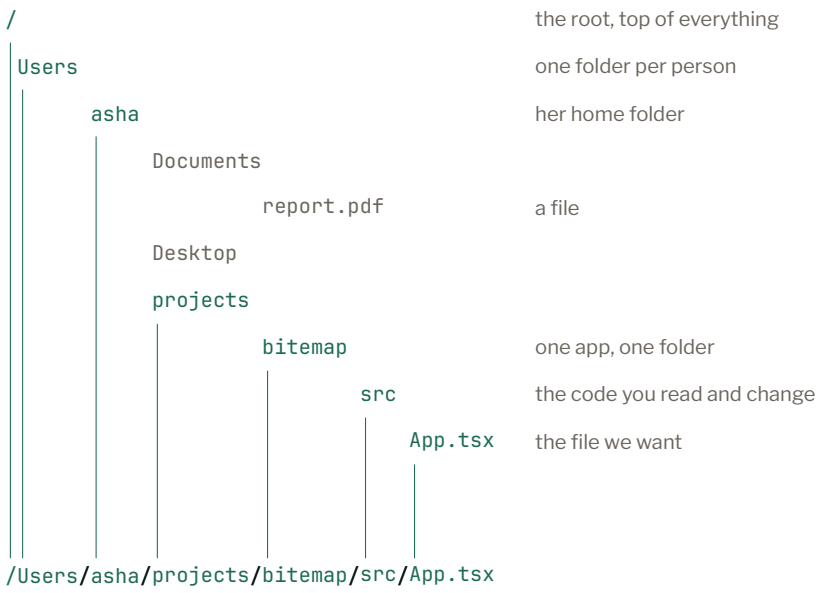
A named collection of bytes that the operating system keeps on a disk is called a **file**. The part of the operating system that keeps track of files is called the file system. It is in effect an index, stored on the disk alongside everything else, that records the name of each file, where its bytes are, how large it is, when it was changed and who is allowed to read it.

file A named collection of bytes stored on a disk, which the operating system keeps track of and can find again.

Folders, which Unix calls directories, are part of the same convention. A folder holds no bytes of the files inside it; it is a list of names, kept by the file system, and it may include the names of other folders. That allows folders inside folders, as deep as anyone likes. On a Mac or a Linux machine the whole arrangement hangs from a single top folder called the root, written as a single slash, `/`. Inside it is a folder called `Users`, inside that a folder for each person who uses the machine, and inside that person's folder the familiar `Documents`, `Desktop` and `Downloads`. That personal folder is called the home folder.

To name one file among all the others, you list the folders you pass through on the way down to it, separated by slashes. This is called a **path**.

path The address of a file: the folders you pass through to reach it, written in order and separated by slashes.



The path is the same location written on one line. Each slash steps one folder deeper.

Figure 8.2 The folders on a laptop form a tree, with the root at the top. Each folder can hold files and other folders. A path names one location by listing the folders you pass through to reach it from the root, separated by slashes.

The path `/Users/asha/Documents/report.pdf` means: start at the root, go into `Users`, then `asha`, then `Documents`, and there find `report.pdf`. Windows writes the same idea differently. Each disk has a letter, and the folders are separated by backslashes, so the same file might be `C:\Users\asha\Documents\report.pdf`. Paths matter in this course because AI agents and error messages use them constantly. When an agent reports that it changed `src/App.tsx`, it is giving a path, starting from the project's own folder.

The end of a file's name, after the last full stop, is called its extension: `.pdf`, `.xlsx`, `.py`. The extension is part of the name and nothing more. Renaming `report.pdf` to `report.txt` does not change a single byte inside it; Chapter 2's point that bytes mean nothing until a program decides how to read them applies here too. What the extension does is tell the operating system, and the person looking at the folder, which program is expected to read the bytes. When you double-click a file, the operating system looks at the extension and starts the program registered for it. Rename the file and the wrong program may open it, or none at all, but the contents are what they always were.

From teletype to terminal window

In the 1960s, when a computer filled a room and cost as much as a building, the people who used it did not sit beside it. They sat at a device in another room, or another town, connected to it by a cable or a telephone line. The device was a keyboard joined to a printer. What a person typed was sent to the computer, one character at a time, and what the computer sent back was printed on a roll of paper.

The most common of these devices was the Teletype Model 33, made by the Teletype Corporation from 1963. It was a teleprinter, a descendant of telegraph equipment, and it was cheap, noisy and slow: it printed about ten characters a second. It was also one of the first machines to use the ASCII code described in Chapter 2. More than half a million were made, and from the mid-1960s to the mid-1970s it was the standard way to talk to a minicomputer, including the machines at Bell Labs on which Unix was written. A device at the end of the line, through which a person types to a computer and reads its replies, is called a **terminal**.

terminal A device, or today a window on the screen, through which a person types commands to a computer and reads its replies.

A CLOSER LOOK

Why "terminal", and why "tty"

The word comes from the wiring. In a system of cables running from a central computer, the device where a line ends, and a person sits, is at the terminal point of the line, as a railway terminal is where the track stops. The name stayed when the cable shrank to nothing and the terminal moved inside the computer.

Unix, written on machines with Teletypes attached, named its terminal connections `tty`, short for teletypewriter. The name is still there. A terminal window on a modern Mac or Linux machine is listed by the operating system as a `tty` device, and there is a command called `tty` that prints the name of the one you are typing into. No paper is involved. The machines left another mark too. Many Unix commands have very short names, `ls` for list, `cp` for copy, `rm` for remove, and the usual explanation, plausible but hard to prove, is that typing on a Model 33 was slow and took a firm press on every key.

Paper gave way to glass in the 1970s. A video terminal looked like a television with a keyboard in front of it, and it showed the same lines of text as the Teletype, scrolling up the screen instead of out of the printer. The best known was the VT100, made by the Digital Equipment Corporation from 1978. It was still a separate device, connected by a cable to

Figure 8.3 A DEC VT100 video terminal, introduced in 1978. It had no processing power to speak of: it displayed what the computer sent and passed on what the user typed. The screen shows a listing of files.

a computer elsewhere, but it was silent and fast, and it could move the cursor around the screen, which allowed programs such as text editors to redraw part of the display rather than print a new line.

When computers became small enough to sit on a desk, the separate terminal disappeared, but the way of working did not. Today a terminal is a program that draws a window and behaves as a terminal device would: it shows text, takes what you type and passes it on. Such a program is called a terminal emulator, since it imitates the hardware, though most people just call it the terminal. Every Mac has one, called Terminal, in the Utilities folder; Windows has Windows Terminal; and the code editors that programmers use, such as VS Code, have one built in as a panel at the bottom of the window.

The shell

A terminal only carries characters. Something on the computer has to read what was typed, work out what was meant and do it. That something is an ordinary program, called the **shell**: it waits for a line of text, treats the first word as the name of a program to run and the rest as instructions

shell The program that reads the commands typed into a terminal, starts the programs they name and shows the results.

for that program, starts it, waits for it to finish, and then waits for the next line. The name suggests a layer around the core of the operating system, which is what it is. The distinction is worth keeping clear. The terminal is the window, or once the device, that you type into. The shell is the program that reads what you type.

The first Unix shell was written by Ken Thompson and appeared in the first edition of Unix in 1971. In 1973 Unix gained a feature, suggested by Doug McIlroy, its manager at Bell Labs, called the pipe, written `|`, which sends the output of one program straight into another. It encouraged a way of building software from small tools that each did one job, and that could be joined in a line like sections of pipe.

Write programs that do one thing and do it well. Write programs to work together.

DOUG MCILROY, AS SUMMARISED BY PETER SALUS, 1994

Stephen Bourne, also at Bell Labs, wrote a new shell that became the standard one in 1979, with Version 7 of Unix. The Bourne shell made the shell a programming language in its own right: a list of commands could be saved in a file, with tests and loops, and run whenever needed. In 1989 Brian Fox, working for the GNU Project, released a free shell called Bash, short for "Bourne-again shell". Bash became the standard shell on Linux, and on the Mac until 2019, when Apple switched to another, called zsh. The differences between them matter to specialists. For a newcomer they all behave in the same way.

Because a shell is a program, it is a file like any other, and you can ask to see it. The first command below asks which shell this laptop is using; the second lists the shells installed on it.

Terminal

```
$ echo $SHELL
/bin/zsh
$ ls /bin/*sh
/bin/bash       /bin/dash       /bin/sh         /bin/zsh
/bin/csh        /bin/ksh        /bin/tcsh
```

When a terminal window opens, it shows a short piece of text and waits. On a Mac the text looks something like `asha@laptop ~ %`: the user's name, the computer's name, the current folder, and a sign that the shell is ready. This is called the prompt. In this book, as in most books about computers, the prompt is shortened to a dollar sign. Lines that begin with

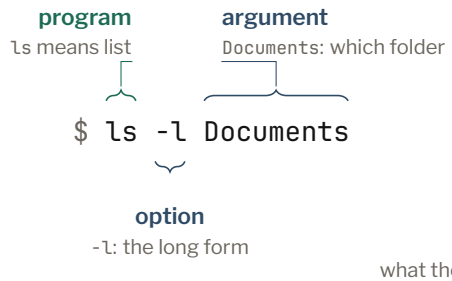


Figure 8.4 A command has three kinds of part. The first word names the program to run; options, beginning with a hyphen, adjust what it does; arguments say what it should work on. Below is what the terminal printed back.

what the terminal prints back

```
$ ls -l Documents
total 120
-rw-r--r--  1 asha  staff   9120 Sep 30 18:45 budget.xlsx
drwxr-xr-x  2 asha  staff     64 Oct  9 10:12 notes
-rw-r--r--  1 asha  staff  48213 Oct  8 15:30 report.pdf
```

`$` are what a person types; the other lines are what the computer prints back.

Anatomy of a command

Most commands follow the same pattern. The first word names a program. After it may come options, which change how the program behaves and usually begin with a hyphen, and arguments, which tell it what to work on. The command `ls -l Documents` runs the program `ls`, which lists the contents of a folder; the option `-l` asks for the long form, with details; and the argument `Documents` says which folder to list.

Here is a short session on a laptop belonging to a student called Asha. When a terminal opens, the shell starts in her home folder.

Terminal

```
$ ls
Desktop         Documents       Downloads       projects
$ ls -l Documents
total 120
-rw-r--r--@ 1 asha  staff   9120 Sep 30 18:45 budget.xlsx
drwxr-xr-x@ 2 asha  staff     64 Oct  9 10:12 notes
-rw-r--r--@ 1 asha  staff  48213 Oct  8 15:30 report.pdf
```

The first command, with no option or argument, lists the home folder: four folders, the same ones a Mac's file browser would show. The second lists the `Documents` folder in long form. Each line is one item. The letters at the start record who may read and change it, and the `d` at the front of

`notes` marks it as a folder; the `@` after them means the Mac has attached a note of its own, such as which program made the file. Then come the owner, `asha`, and a group of users, `staff`; the size in bytes; the date and time of the last change; and the name. The spreadsheet is about nine thousand bytes and the report about 48,000. All of this is information the file system keeps about each file, shown as plain text.

The next commands move into a folder, look inside it and run a program.

Terminal

```
$ cd projects/sum
$ ls
numbers.txt     sum.py
$ cat numbers.txt
3
8
20
$ python3 sum.py
31
```

The command `cd`, for change directory, moves the shell into another folder, here given as a path from where it already was. Nothing is printed: in the Unix tradition, a command that succeeds usually says nothing. The `ls` shows two files, a list of numbers and a Python program. The command `cat` prints a file's contents to the screen; its name comes from *concatenate*, because it can also join several files together. The last command is the recurring example of this book. It asks the shell to start the Python interpreter, which is itself a program stored in a file, and to give it the file `sum.py` to run. The program reads the numbers, adds them up and prints 31.

That last line answers a question left over from the previous chapters. A program written in Python, C or anything else is, to the computer, a file of text in a folder. To run it, someone has to ask the operating system to start a program, either the program itself, if it has been compiled to machine code, or an interpreter that will read it. The shell is one way of asking.

A few more commands do most of the everyday work: `mkdir` makes a folder, `cp` copies a file, `mv` moves or renames one, and `rm` removes one. The last deserves care. A file deleted in a file browser usually goes to a bin from which it can be recovered; a file removed with `rm` is gone at once, with no question asked. The shell assumes that you meant what you typed. Like every layer in this book, it does exactly what it is told.

Programs at work

A program stored in a file is only a list of instructions. When the operating system starts it, it copies the instructions into memory, gives them an area of memory to work in, and adds them to the list of things that take turns on the processor. A program that is running, together with the memory and other resources the operating system has given it, is called a **process**. The difference is like that between a recipe in a book and a meal being cooked from it. The same recipe can be cooked in several kitchens at once, and the same program can run as several processes: each terminal window has its own shell, for example, each one a separate process.

process A running program, with the memory and other resources the operating system has given it.

The command `ps`, for process status, lists processes. Below, the option `-p 1` asks for process number 1 only. The second command lists every process on the machine but, instead of printing the list, sends it through a pipe to `wc -l`, a program that counts lines. The third, `tty`, prints the name of the terminal it was typed into.

Terminal

```
$ ps -p 1
  PID TTY           TIME CMD
    1 ??        97:53.45 /sbin/launchd
$ ps -A | wc -l
     698
$ tty
/dev/ttys047
```

Every process has a number, its PID. Number 1 on a Mac is a program called `launchd`, the first process the operating system starts when the machine is switched on and the one that starts many of the others; the question marks mean it has no terminal of its own, and the time is how long it has used the processor since then. The second answer shows how much is happening on a laptop that seems to be doing very little: nearly 700 processes (one line of the count is the heading), most of them small parts of the operating system waiting for something to do. The third shows the old name of the Teletype still in use, `tty`, for a terminal window with no paper in it.

The full list of processes, in a friendlier form, is shown by Activity Monitor on a Mac and Task Manager on Windows, which are the places to go when a program stops responding. In the terminal, a program that is running too long can be stopped by holding Ctrl and pressing C, which asks the operating system to interrupt it.

Figure 8.5 A Xerox Alto of 1973 at the Computer History Museum. The upright screen was shaped like a page. To the right of the keyboard is the mouse; to the left, a five-key keyset for entering commands with one hand.

Windows, icons and the mouse

Typing commands is precise, but it requires knowing what to type. The graphical interface that most people now use, with windows, icons and a pointer moved by a mouse or a finger, was developed over two decades as a way round that.

Its first public showing was on 9 December 1968 in San Francisco, when Douglas Engelbart and his team from the Stanford Research Institute in Menlo Park gave a 90-minute live demonstration of their system to about a thousand people at a computer conference. Engelbart used a mouse, which his group had invented, to point at text on a screen, follow links from one document to another, and edit a document together with a colleague who was miles away, while the audience watched both of them on a large screen. The computer itself was in Menlo Park, about 30 miles away, connected by leased lines. The event was later nicknamed “the Mother of All Demos”.

At Xerox’s Palo Alto Research Center, a few miles away, a team led on the hardware side by Chuck Thacker built the Alto, which first ran in 1973. It combined a screen that could show any pattern of dots, a mouse, overlapping windows, a network connection and a word processor that showed text on screen as it would look when printed. The Alto was never

sold; it was used inside Xerox and at some universities. In 1979 Steve Jobs and a group of Apple engineers visited the centre and saw it. The visits influenced Apple's next machines, though the common story that Apple simply took the idea is too simple: Xerox was allowed to buy Apple shares before the company went public in return for the visits, and Apple's team designed much of its own interface, including pull-down menus and dragging items with a one-button mouse.

The Apple Macintosh, introduced on 24 January 1984 at a price of $2,495, was the first computer controlled mainly by a mouse to sell in large numbers. Microsoft released Windows 1.0 in November 1985, as a program that ran on top of its text-based operating system, MS-DOS; Windows became dominant only with later versions, in 1990 and 1995. Phones and tablets later replaced the pointer with a finger, but the idea is the same.

A **graphical user interface** is a friendlier layer over the same operating system, and it often does exactly what a command would do. Dragging a file from one folder to another asks the operating system to move it, as `mv` would. Double-clicking an app starts a program, as typing its name would. Even an app's icon is a disguise: on a Mac, an app is a folder with a particular extension, and the program itself sits inside it.

graphical user interface A way of using a computer by pointing at and touching pictures on a screen (windows, icons, menus) instead of typing commands.

Terminal

```
$ ls /System/Applications | head -4
App Store.app
Automator.app
Books.app
Calculator.app
$ ls /System/Applications/Calculator.app/Contents/MacOS
Calculator
```

The first command lists the start of the Mac's built-in applications folder; `head -4` keeps only the first four lines. The second looks inside the Calculator app and finds the program file itself, also called `Calculator`. When you click the Calculator icon, the operating system starts that file as a new process, exactly as the shell started Python above.

Why the terminal is still here

If the graphical interface is easier, why do programmers still type commands, half a century after the Teletype? There are four main reasons, and all of them matter to someone directing an AI agent.

The first is precision. A command says exactly what is to be done, to which file, with which options, and leaves no doubt about which button was pressed. The second is that commands can be saved and repeated. A sequence of clicks has to be performed again by hand each time; a sequence of commands can be written into a file, called a script, and run whenever needed, by anyone, with the same result. Much of the routine work of software, building a program, running its tests, putting it on a server, is done this way. The third is reach. The computers that run websites usually have no screen at all, and the way to work on them is to open a terminal connection across the network, much as the Teletype once connected to a distant machine. The fourth is that most of the tools programmers rely on, from Python and Git to the tools that download libraries and start web servers, are made to be used from the command line first, and some have no other interface.

The same reasons explain why AI coding agents work in the terminal. An agent produces text. It cannot easily look at a screen and click on the right button, but it can type a command, read what the computer prints back, and decide what to do next. A command line is the natural place for a program that writes, because everything it does there is text: the commands, the output and the error messages. It also leaves a record. Every command the agent ran, and every reply it got, can be read afterwards by the person responsible for the work.

AT THE KEYBOARD

Where you will meet the agent

In Volume 2, Chapter 2, you will open a terminal, move into a project folder with `cd`, and start Claude Code by typing its name, `claude`. From then on you write to the agent in plain English, and it replies in the same window. When it needs to look at your files it runs commands such as `ls` and `cat`; when it needs to test the code it runs the program, as the shell ran `sum.py` above. Each time, it shows you the command and usually asks your permission first.

You will not need to type many commands yourself. But you will need to read the ones the agent proposes and judge whether they are sensible. A command that lists or reads files is harmless. One that removes files, or changes things outside the project folder, deserves a second look before you say yes. The anatomy in this chapter, program, options, arguments and path, is enough to tell which is which most of the time.

THE BUSINESS VIEW

The platform underneath

For most of its users an operating system is invisible, but for the companies that make software it is the ground they build on, and whoever owns that ground has power over what is built. An app for the iPhone has to be written for iOS and, in most countries, distributed through Apple's App Store on Apple's terms; an Android app depends on Google's operating system and store in the same way. Rules about payments, privacy and which apps are allowed are set by the owner of the platform, and have been the subject of court cases and new laws in the United States, the European Union and elsewhere.

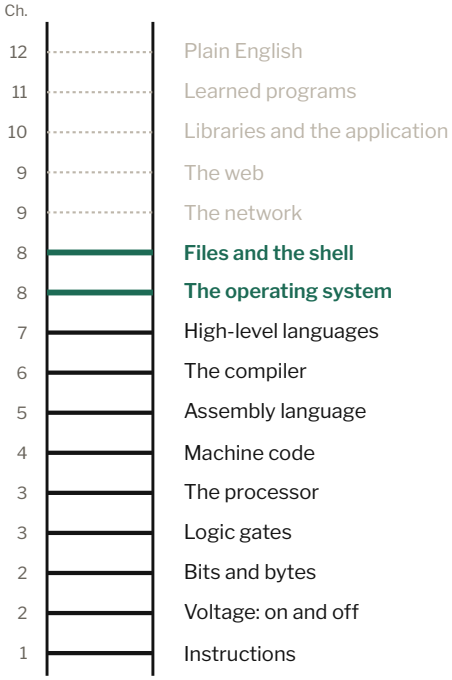
The same logic explains why software companies care about which operating systems their customers use, and why tools that let one program run on several platforms are popular. For a small team, building once for the web, which runs in a browser on every operating system, is often cheaper than building separately for each.

Where this leaves us

We now have a computer that is well organised. The operating system shares the processor among hundreds of processes, keeps each program in its own memory, and turns the disk's numbered blocks into named files in folders, each reachable by a path. Programs ask it for these services instead of driving the hardware themselves, so that one line of code can rely on millions written by others. People reach it through two layers of their own: the graphical interface of windows and icons, and the older, more precise layer of the terminal and the shell, where a typed command names a program, adjusts it with options and points it at a file.

What this computer cannot yet do is reach any other. Everything in this chapter happens inside one machine: its files, its processes, its shell. The Teletype was joined to its computer by a wire, but that wire connected one person to one machine, not machines to one another.

Joining computers so that programs on one can talk to programs on another, across a building and then across the world, was the work of the late 1960s onwards. It produced the internet, and on top of it the web, and those are the subject of Chapter 9.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

PART THREE

The Connected Machine

How computers came to talk to one another, how today's apps are assembled from parts, how programs learned to write themselves, and how plain English became the newest way to program.

The NeXT computer on which Tim Berners-Lee ran the first web server at CERN in 1990. The handwritten label reads: “This machine is a server. DO NOT POWER IT DOWN!!”

CHAPTER NINE

Networks, the Web and the Cloud

A computer on its own can only use what is inside it. Joined to others, it can reach information and programs anywhere in the world. This chapter follows how that joining was done, from a two-letter message in 1969 to the web pages, data services and rented data centres that every modern app now relies on.

BY THE END OF THE 1960S a computer was a well-organised machine. It had an operating system to share it among many users, files to keep their work, and terminals through which people typed commands, as Chapter 8 described. But each computer was an island. A program at one university could not read a file at another, and a researcher who needed a particular machine had to travel to it, or post a box of punched cards and wait for the results to come back.

This chapter describes how the islands were joined. It covers three layers that are easy to confuse because they arrived one on top of another. The first is the *network*: the rules and cables that let any computer send a message to any other. The second is the *web*: a way of publishing linked pages on that network so that anyone can read them. The third is the *cloud*: the practice of renting computers in someone else's building and reaching them over the network. Together they explain what happens in the second or so between typing a web address and seeing a page, and why a modern app is rarely one program on one machine.

Two letters from Los Angeles

The first network of the kind we use today was paid for by the United States Department of Defense, through its Advanced Research Projects Agency, ARPA. Its purpose is often misremembered. It was not built mainly to survive a nuclear war. ARPA funded research computers at universities across the country; they were expensive, each was good at different things, and the agency wanted researchers to be able to use one another's machines without buying more. The network that resulted was called the **ARPANET**.

ARPANET The network funded by the US Defense Department's ARPA from 1969; the forerunner of the Internet.

Its first message was sent at 10.30 in the evening on 29 October 1969. Charley Kline, a student programmer in Leonard Kleinrock's laboratory at the University of California, Los Angeles, tried to log in to a computer at the Stanford Research Institute, about 350 miles away, where Bill Duvall was waiting at the other end of a telephone line. Kline typed L, and it arrived. He typed O, and it arrived. When he typed G, the computer at Stanford crashed. "Hence," Kleinrock later wrote, "the first message on the Internet was 'lo,' as in 'lo and behold!' " About an hour later the full login worked. By December 1969 the network had four computers: at UCLA, Stanford Research Institute, the University of California at Santa Barbara and the University of Utah.

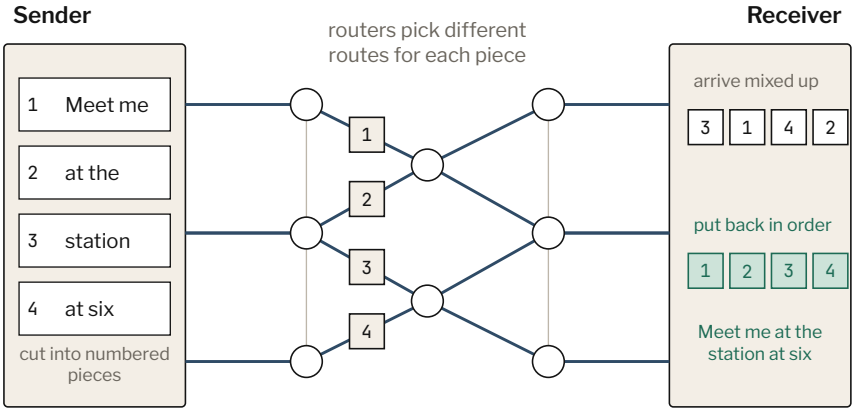


Figure 9.1 A message travels as numbered packets. Routers along the way choose a route for each packet separately, so the pieces may arrive out of order; the receiving computer uses the numbers to put them back together.

Each piece (a packet) carries its number and the address it is going to.

What made the ARPANET unusual was not the telephone lines, which already existed, but the way it used them. A telephone call of the time reserved a complete circuit between two callers for as long as they talked, even during the silences. Computers talk in bursts: a few characters, a pause, a large file, a long pause. Reserving a whole line for each conversation would have wasted most of it.

A letter cut into postcards

The solution was to cut every message into small pieces and send each one separately. Imagine posting a long letter as a series of numbered postcards, each carrying the address it is going to and its place in the sequence. The postcards can travel by different routes and arrive in any order; at the other end, someone sorts them by number and reads the letter. Meanwhile the same roads carry postcards from thousands of other senders, so no road stands idle while one writer pauses.

In a network, each postcard is called a **packet**, and the method is called *packet switching*. The idea was worked out in the early 1960s by Paul Baran at the RAND Corporation in the United States, who was indeed interested in networks that could survive damage, and independently by Donald Davies at the National Physical Laboratory in Britain, who gave the packet its name. The ARPANET was the first large network to use it.

packet A small piece of a message, sent on its own with its destination address and its place in the sequence.

Between sender and receiver stand computers whose only job is to pass packets along. Each looks at a packet's destination, chooses a neighbour that is closer to it, and sends the packet on. Such a machine is called a **router**. No router knows the whole route. Each knows only which way

router A computer in a network that receives packets and passes each one on towards its destination.

to send a packet next, much as a sorting office in Pune need not know the street plan of Chennai to put a letter on the right train. If a line fails, the routers send packets another way.

A common language for different networks

By the early 1970s there were several packet networks, some using radio and some using satellites, and they could not exchange messages with one another. Vinton Cerf and Robert Kahn set out to connect them. Their paper "A Protocol for Packet Network Intercommunication", published in May 1974, described how messages could cross from one network to another through machines they called gateways, which today we call routers. The networks underneath could stay different; what they had to share was a set of rules for how packets are addressed, sent and acknowledged.

Such a set of rules is called a **protocol**. Cerf and Kahn's design was later split into two parts. The Internet Protocol (IP) gives every computer an address and moves packets from one address to another, doing its best but promising nothing. The Transmission Control Protocol (TCP) sits on top: it numbers the packets, checks that all of them arrived, asks again for any that went missing and puts them back in order. Together they are known as TCP/IP. On 1 January 1983 the ARPANET switched over to TCP/IP on a single day, which is why that date is often called the birthday of the Internet. The word itself is short for the interconnected networks that the protocol made possible. Cerf and Kahn received the Turing Award, the highest honour in computing, in 2004.

protocol An agreed set of rules for how two computers format and exchange messages.

Every computer on the Internet has an **IP address**, which in its common form is four numbers between 0 and 255, separated by dots. Numbers suit routers but not people, and as the network grew, keeping track of them by hand became impossible. In November 1983 Paul Mockapetris, at the University of Southern California, published the design of the **Domain Name System**, or DNS. It lets people use names such as `cern.ch` or `example.com`, and it turns each name into an address on request. No single computer holds the whole list. Each organisation runs servers for its own names, and the system passes a question along until it reaches one that knows the answer.

IP address A number that identifies a computer on the Internet, so that packets can be delivered to it.

DNS The Internet's address book: a worldwide system that turns names such as example.com into IP addresses.

Anyone can ask the question from a terminal. The command `nslookup` sends a name to the DNS and prints the reply:

```
Terminal

$ nslookup example.com
Server:         192.168.0.1
Address:        192.168.0.1#53

Non-authoritative answer:
Name:   example.com
Address: 172.66.147.243
Name:   example.com
Address: 104.20.23.154
```

The first two lines say who answered: the home Wi-Fi router, at the private address `192.168.0.1`, which passed the question on and remembered the answer. (The `#53` is the numbered door, or *port*, on which DNS questions are received.) "Non-authoritative" means that the answer came from that remembered copy rather than from the servers responsible for `example.com`. The name has two addresses because busy sites are served by more than one computer; a browser can use either.

A web of linked pages

The Internet carried messages, but in the 1980s finding information on it still meant knowing exactly which computer held it and which commands to type. At CERN, the European particle physics laboratory near Geneva, thousands of scientists from many countries came and went, using different computers, and much of what they knew was lost when they left. Tim Berners-Lee, a British software engineer working there, thought the problem could be solved by *hypertext*: documents containing links that lead, with one click, to other documents, wherever they are stored.

Figure 9.2 Tim Berners-Lee, who proposed the World Wide Web at CERN in 1989, speaking in Lisbon in 2025.

In March 1989 he wrote a proposal called "Information Management: A Proposal". His manager, Mike Sendall, wrote on the cover "Vague but exciting..." and allowed him to work on it. With Robert Cailliau, a Belgian engineer at CERN, he revised the proposal in 1990, and by Christmas of that year he had written the three things on which the web still rests, along with the first browser, called WorldWideWeb, and the first server, both running on a NeXT computer in his office. The address of that first site was `info.cern.ch`. In August 1991 he announced the project to the world outside CERN, in a message to an Internet discussion group.

The three things were these. A **URL** is the address of anything on the web: it says how to ask, which computer to ask and what to ask it for. *HTTP*, the Hypertext Transfer Protocol, is the conversation by which

URL Uniform Resource Locator: the address of a page or file on the web, such as https://example.com/books.

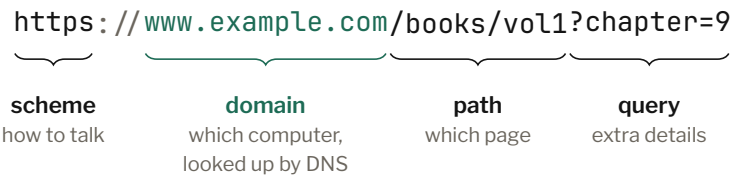


Figure 9.3 The parts of a web address. The scheme says which protocol to use; the domain names the computer, which DNS turns into an IP address; the path says which page; the query, after the question mark, adds details such as a search term.

a browser asks a computer for a page and the computer sends it back. *HTML*, the Hypertext Markup Language, is the way a page is written, so that any browser can display it and follow its links.

It is worth being clear about what the web is not. It is not the Internet. The Internet is the network, running since 1969 and on TCP/IP since 1983. The web, from 1990, is one service that runs on it, alongside email, video calls, online games and the messages that phone apps send to their servers. The confusion is understandable, because for most people the browser is the main window on to the network, but it matters later in this chapter, when programs talk to each other without any web page at all.

A CLOSER LOOK

The decision not to own the web

CERN could have treated the web as its property. It had paid for the work, and other hypertext systems of the time were sold under licence. One of the web's early rivals, the Gopher system at the University of Minnesota, lost ground from 1993, partly because the university announced that some users would have to pay a licence fee.

On 30 April 1993 CERN did the opposite. It released the web's software into the public domain: anyone could use it, change it, build a browser or run a server, and owe nobody anything. Berners-Lee had pressed for this, arguing that the web would spread only if no one had to ask permission. In 1994 he founded the World Wide Web Consortium, which still agrees the web's standards among companies that compete fiercely in everything else.

The result was that the web grew faster than any proprietary system could have. Universities, companies and individuals set up servers by the thousand. By July 2026 there were about 1.5 billion websites, of which around 200 million were active; by the end of 2025 about three-quarters of the world's people used the Internet. A single decision about ownership shaped how all of that grew.

What happens when you open a web address

Two roles run through everything on the network. The program that asks is the **client**: the browser on your laptop, or an app on your phone. The program that answers is the **server**. The word is used for both the program and the machine it runs on, which is usually a computer in a data centre with no screen or keyboard of its own. Berners-Lee's NeXT was both a workstation and a server, which is why someone stuck a note on it asking people not to switch it off.

client A program that sends a request over a network and waits for the answer; a browser is a client.

server A program, or the computer it runs on, that waits for requests from clients and sends back answers.

Suppose you type `https://example.com` into a browser and press Enter. Five things happen, usually in well under a second. The browser first needs the server's IP address, so it asks the DNS, as in the terminal session above. With the address in hand, it opens a connection to that computer, using TCP so that nothing is lost, and sets up encryption; the `s` in `https` means the conversation is scrambled so that the routers along the way cannot read it. It then sends a short request in HTTP, which in essence says "GET the page called /". The server replies with a status code, some descriptive lines called headers, and the page itself, written in HTML. Finally the browser reads the HTML, draws the page, and notices that the page mentions other files, such as images, style sheets and programs, which it fetches in the same way.

A program called `curl` lets you take part in this conversation from the terminal, without a browser. With the option `-I` it asks only for the headers, not the page:

Terminal

```
$ curl -sI https://example.com
HTTP/2 200
date: Sun, 11 Oct 2026 15:35:33 GMT
content-type: text/html; charset=utf-8
server: cloudflare
last-modified: Fri, 09 Oct 2026 20:18:28 GMT
[...]
cf-cache-status: HIT
cf-ray: a48eef748a1e7e53-BOM
alt-svc: h3=":443"; ma=86400
```

The first line is the most important. `HTTP/2` is the version of the protocol, and `200` is the status code, which means that everything went well. Status codes are three-digit numbers with fixed meanings, and some are familiar from daily use: `404` means that the page does not exist, and codes

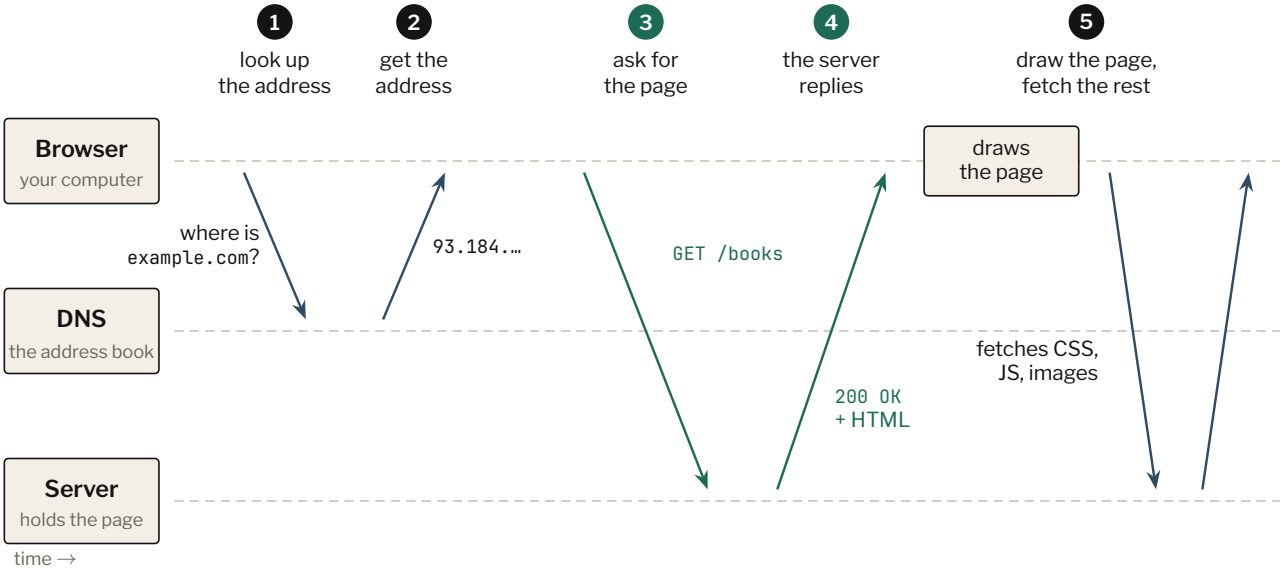


Figure 9.4 What happens when you open a web address. The browser asks the DNS for the server's address (1–2), sends an HTTP request for the page (3) and receives the page with a status code such as 200 OK (4). It then draws the page and fetches the other files the page refers to (5). The whole exchange usually takes well under a second.

beginning with 5 mean that something went wrong on the server. The next lines give the date and time, and say that what follows is HTML written in UTF-8, the encoding for text described in Chapter 2.

The last lines show something the user never sees. The answer did not come from a single computer belonging to the owners of `example.com`. It came from Cloudflare, a company that keeps copies of popular pages in data centres around the world. `HIT` means that a copy was already on hand, and the letters `BOM` at the end of the last line are the airport code for Mumbai, which Cloudflare uses to label its data centre there, the one nearest to where this request was made. A page that seems to live at one address may in fact be served from dozens of places, by a company whose name appears nowhere on the page.

One page, three languages

The page a server sends is a text file, and a browser turns it into what you see. Modern web pages are written in three languages, each with its own job. HTML, from Berners-Lee, describes the structure: this is a heading, this is a paragraph, this is a button, this is a link. CSS, Cascading Style Sheets, proposed by Håkon Wium Lie at CERN in October 1994 and devel-

oped with Bert Bos, describes how things look: colours, fonts, sizes and positions. JavaScript, written by Brendan Eich at Netscape in ten days in May 1995, describes behaviour: what happens when someone clicks, types or scrolls. (Despite its name, JavaScript has nothing to do with Java; the name was chosen for marketing reasons.)

Here is a complete web page that uses all three. It shows a heading and a button, and counts how many times the button has been pressed.

tea.html

```
<!DOCTYPE html>
<style>
  h1 { color: green; font-family: serif; }
  p  { font-size: 28px; }
</style>
<h1>Tea orders</h1>
<button id="add">Add one</button>
<p id="count">0</p>
<script>
  let n = 0;
  add.onclick = () => {
    n = n + 1;
    count.textContent = n;
  };
</script>
```

The first line tells the browser that this is a modern HTML page. Lines 2 to 5 are CSS, between `<style>` tags: headings (`h1`) are to be green and in a serif typeface, and paragraphs (`p`) are to be large. Lines 6 to 8 are HTML: words between angle brackets, called tags, mark what each part of the page is. There is a heading, a button labelled “Add one” and a paragraph containing 0, and the button and paragraph are given names, `add` and `count`, so that the program can refer to them. Lines 9 to 15 are JavaScript, between `<script>` tags. They set a counter, `n`, to zero, and say that whenever the button is clicked, one is added to the counter and the new number is written into the paragraph. Saved as a file and opened in a browser, the page works; after three clicks it shows 3.

Notice how far this is from the voltages of Chapter 2. The browser that reads these fifteen lines is itself a program of tens of millions of lines, written mostly in C++, that runs on an operating system, which runs on a processor, which reads machine code. The person who wrote `tea.html` needs to know none of that. It is the ladder of abstraction at work again: each layer lets people say a great deal with very little, because many others have already written what lies beneath.

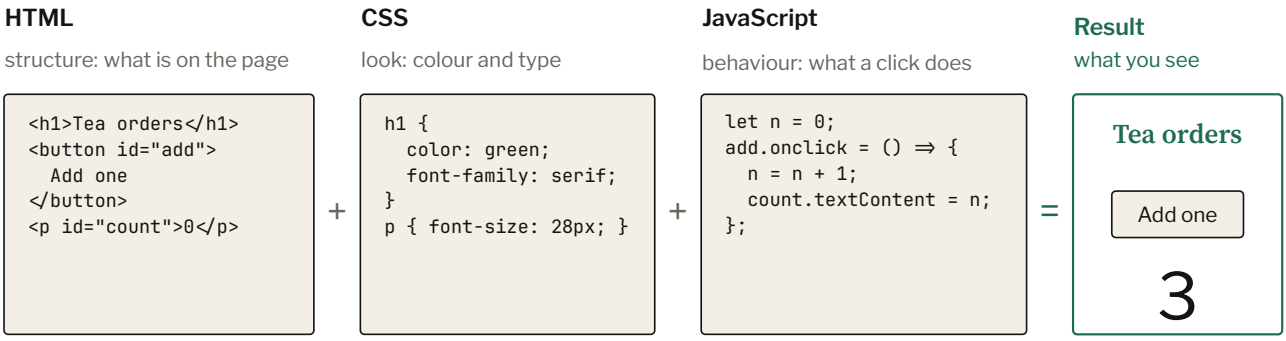


Figure 9.5 The three languages of a web page. HTML says what is on the page, CSS how it looks, and JavaScript what it does when someone interacts with it. The browser combines all three into the page you see.

Programs that ask other programs

Not every request on the network comes from a person reading a page. Most of the traffic is programs asking other programs for information. A weather app on a phone does not contain the weather. It sends a request to a server run by a forecasting service, gets back the numbers, and draws them in its own design. A cricket app does the same for scores, a map for directions, a payment form for approval from a bank.

The rules for such requests are called an **API**, an Application Programming Interface. An API is a kind of menu that a service publishes for programs: the requests you may make, how to word them, and what you will get back. Most web APIs use the same HTTP as browsers, and their requests are ordinary URLs. The difference is in what comes back. Instead of a page meant for people, the server returns data meant for a program.

API Application Programming Interface: a published set of requests one program can make of another, and the answers it will give.

Here is a real request to Open-Meteo, a free weather service, for the current conditions in Mumbai. The latitude and longitude, and the mea-

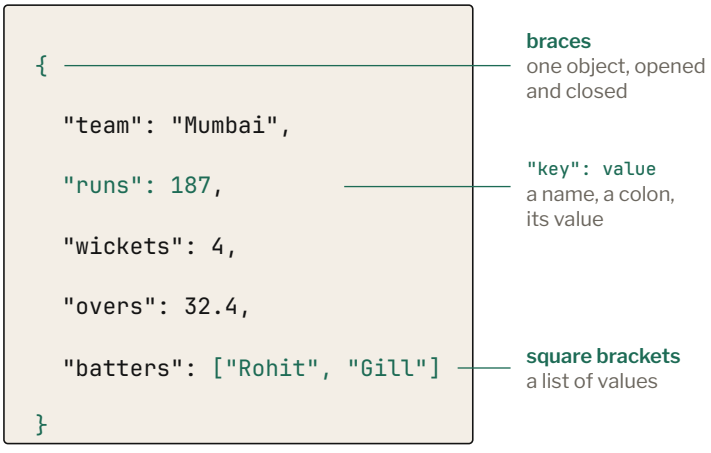


Figure 9.6 A small JSON object describing a cricket score. Braces open and close the object; inside, each line pairs a name in quotation marks with its value; square brackets hold a list. Programs in almost any language can read it.

surements wanted, go in the query part of the URL. A small program called `jq` picks out the part of the answer we need and lays it out neatly. The command is one long line; the backslashes at the ends of the first two lines simply let it continue on the next:

Terminal

```
$ curl -s "https://api.open-meteo.com/v1/forecast?\
latitude=19.08&longitude=72.88&timezone=Asia/Kolkata\
&current=temperature_2m,wind_speed_10m" | jq '.current'
{
  "time": "2026-10-12T07:30",
  "interval": 900,
  "temperature_2m": 28.0,
  "wind_speed_10m": 5.8
}
```

At half past seven in the morning it was 28 degrees in Mumbai, with a light wind of 5.8 kilometres an hour. (The `interval` of 900 says that the readings are updated every 900 seconds, or 15 minutes; the `_2m` and `_10m` say the temperature is measured two metres above the ground and the wind ten metres up.) No person was involved at the other end. A program received the request, looked up the forecast and sent back these few lines.

The format of the answer is called **JSON**, JavaScript Object Notation. It is written as text that people can read: names in quotation marks, each followed by a colon and a value, grouped by braces, with lists in square brackets. Douglas Crockford and Chip Morningstar sent what they regard as the first JSON message in April 2001, and Crockford published a description of it the next year. Crockford has said that he discovered JSON rather than invented it, because it was already a part of JavaScript; his contribution was to notice it, give it a name and write it down. Because it is simple and almost every language can read it, JSON became the common format in which programs exchange data over the web.

JSON JavaScript Object Notation: a plain-text format for data, made of named values in braces and lists in square brackets.

APIs are why modern software is assembled rather than written from scratch. A small company's app can take its maps from one firm, its payments from another, its login from a third and, increasingly, its text-writing from an AI model reached through an API. Each of those services is a large system maintained by other people. The app's own code is the part that joins them together, which is the subject of Chapter 10.

Figure 9.7 Part of the data centre at CERN, where the web began. Cloud providers run buildings like this by the hundred, each holding many thousands of servers, with their own power supplies and cooling.

Someone else's computers

Every server in this chapter is a physical machine somewhere, drawing electricity and giving off heat. For the first decades of the web, a company that wanted a website or an online service bought its own servers, installed them in a cupboard or a rented room, and employed people to keep them running. It had to buy enough machines for its busiest day, which meant that most of them sat idle on the other days, and it had to wait weeks for new ones to arrive when business grew.

In 2006 Amazon, which had built large computing systems to run its own shop, began renting them to others. Its storage service, S3, opened on 14 March 2006, and its computing service, EC2, followed in a limited trial on 25 August, at 10 cents an hour for a virtual computer. A customer could ask for a server through a web page or an API, have it running within minutes, and give it back when finished, paying only for the hours used. Renting computing power and storage over the network in this way, from a provider who owns and runs the machines, is called the **cloud**. Microsoft, Google and many others soon offered the same.

cloud Computers, storage and services rented over the Internet from a provider who owns and runs the machines.

The name suggests something weightless, but the cloud is made of buildings. A large data centre holds tens of thousands of servers in rows of metal racks, with backup generators, cooling systems and several fibre-optic lines to the outside world. Providers group their buildings into regions, and a company can choose to keep its data in, say, Mumbai or Frankfurt, which matters when a country's law says where its citizens'

data may be stored. What the cloud adds is a layer of abstraction over all of this: the customer asks for "a server with this much memory" or "space for this many files" and does not need to know which machine, which rack or which building provides it.

THE BUSINESS VIEW

Renting instead of owning

The cloud changed the economics of starting a software business. Before it, a company needed to buy servers before it had customers, which meant raising money for equipment that might never be fully used. With the cloud, the cost of computing became a running expense that rises and falls with use. A student project can run for nothing, or a few hundred rupees a month, on the free and starter tiers that providers offer; a company whose app suddenly becomes popular can add servers in minutes instead of weeks.

The trade-offs are real. Renting is usually more expensive than owning for steady, predictable loads, and some large firms have moved work back to their own machines for that reason. Bills can grow quickly and unexpectedly if a program misbehaves or an app is left running. And a business that builds on one provider's particular services can find it hard to move to another. For a new product, though, the ability to start small and pay only for what is used has made experimenting far cheaper than it was in 2000.

AT THE KEYBOARD

Front end, back end, API

When you build an app with an AI agent, it will use the words of this chapter without explaining them. The *front end* is the part that runs on the user's device: HTML, CSS and JavaScript in a browser, or an app on a phone. The *back end* is the part that runs on a server: it stores data, checks passwords and talks to other services. The two talk to each other through an *API*, usually sending JSON over HTTP.

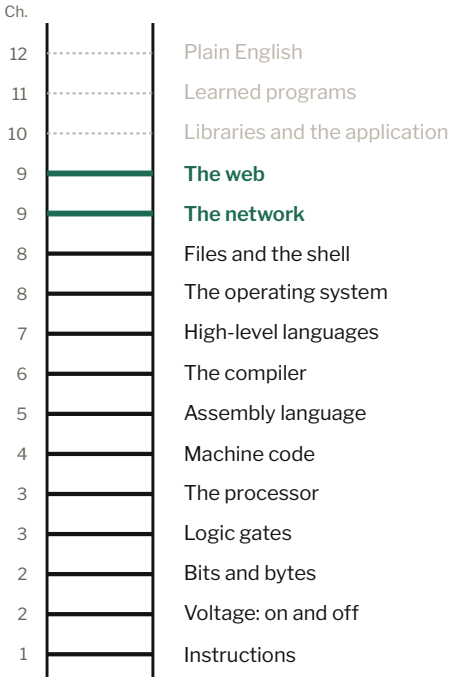
Knowing which part a problem lives in makes your requests far clearer. "The button doesn't do anything" is a front-end problem; "the scores don't save" may be a back-end one; "it works on my laptop but not online" usually concerns hosting, which is to say the cloud. Volume 2 shows how to describe each kind to an agent, and how to read the status codes and error messages it shows you.

Where this leaves us

Computers are no longer islands. Packet switching lets many conversations share the same lines; TCP/IP lets different networks carry one another's packets and makes sure nothing is lost; DNS lets people use names instead of numbers. On top of that network, the web gave everyone a way to publish and read linked pages, using URLs, HTTP and HTML, with CSS and JavaScript added for appearance and behaviour.

The same machinery now carries far more than pages. Programs ask other programs for data through APIs and receive it as JSON, so that one app can draw on maps, payments and forecasts maintained by others. The servers that answer are increasingly rented rather than owned, in data centres that their users never see.

What we have, then, is a set of parts that can talk to one another across the world. A modern app is built by putting such parts together: a front end on the user's device, a back end on a rented server, a database, and a handful of outside services, most of them built from code that someone else wrote and gave away. How those parts fit together in a real app is the subject of Chapter 10.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

Server racks at the headquarters of NOIRLab, the American astronomy research centre. Much of the software in this chapter runs on machines like these, rented by the hour.

CHAPTER TEN

Anatomy of a Modern App

A modern app is assembled more than it is written. A few thousand lines of its own sit on top of hundreds of packages written by strangers, a server somewhere else, and a database that remembers. This chapter opens two of the course's own projects to see how the parts fit.

BITEMAP IS A FOOD-ORDERING APP built during this course. On a phone it looks like any of the delivery apps that crowd Indian home screens: pick a delivery area, search for biryani, add dishes to a cart, apply a discount code, check out, and follow the order on a map. Its own code, the part written for this project, runs to about 5,000 lines, of which some 750 are sample restaurants and dishes. That is a modest amount. A person reading carefully could get through it in a long day.

Yet the folder that holds BiteMap also holds more than 7,600 files that nobody on the project wrote. They were downloaded automatically the first time the project was set up, and together they take up 203 megabytes. They include the code that draws the screens, the code that turns the project into an Android app, and a tool that checks the project's code for mistakes. BiteMap stands on them as the programs in Chapter 8 stand on the operating system.

This chapter is about how such an app is put together: the parts it is divided into, the way a single tap travels between them, the code it borrows, the tools that keep track of changes, and what happens when a borrowed part fails.

The parts of an app

Most apps today divide their work in roughly the same way, building on the client and server of Chapter 9.

The part a person sees and touches is the **frontend**. It runs on the user's own phone or in their browser and is usually written in HTML, CSS and JavaScript, the three languages of the web page in Chapter 9, or in tools that produce them.

frontend The part of an app that runs on the user's phone or in their browser: the screens, buttons and forms.

The part a person does not see is the **backend**. It runs on a server, often in a data centre far from the user, and it holds whatever should not live on a phone: the rules of the business, the passwords and keys for other services, the calculations that must be done the same way for everyone. The frontend and the backend talk to each other through an API, in the requests and replies described in the previous chapter, usually carrying JSON.

backend The part of an app that runs on a server: it applies the rules, holds the secrets and talks to the database.

Behind the backend sits a database, a program whose only job is to store information and find it again quickly: the orders, the users, the menu. Around all three are other companies' services. An app rarely processes card payments itself, draws its own maps or trains its own AI model; it calls a payment company, a map provider or an AI provider

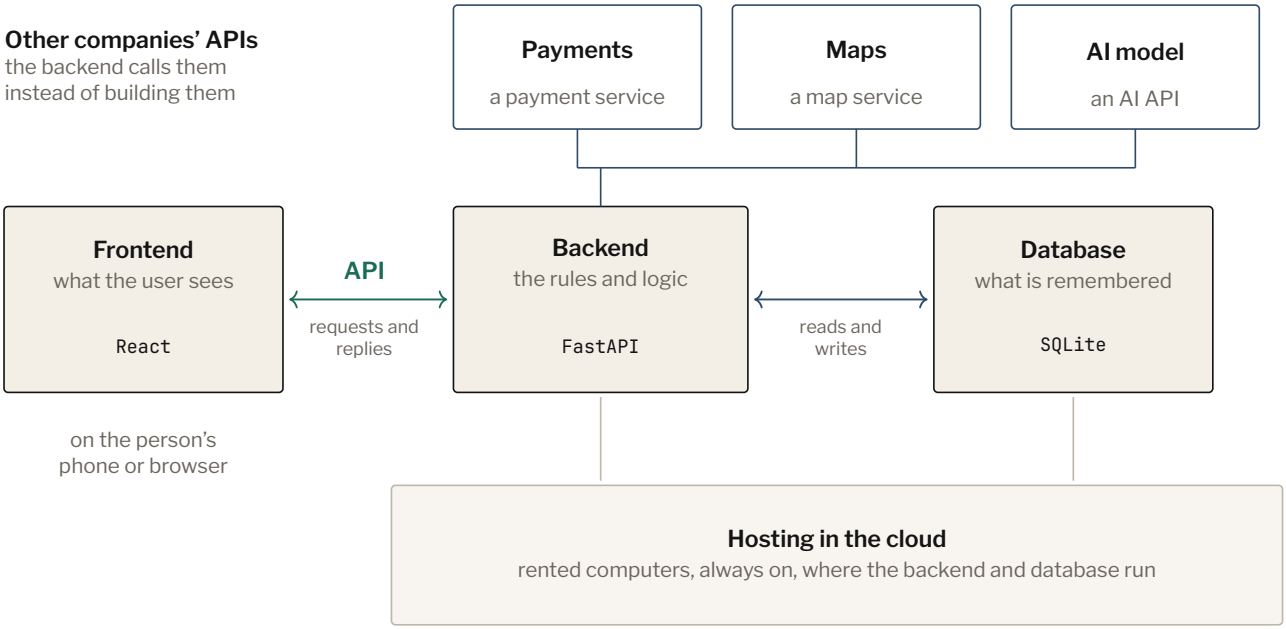


Figure 10.1 The usual parts of a web or phone app. The frontend runs on the user's device; it sends requests through an API to the backend, which applies the rules and reads and writes the database. The backend calls other companies' services for payments, maps or AI, and the whole server side runs on rented machines in the cloud. The labels in typewriter type are the tools used in the course's Options Lab project.

through their APIs, and pays for each use. The backend and the database, finally, run on rented machines in a cloud data centre, as described at the end of Chapter 9.

Not every app has every part. BiteMap, as built in the course, is a frontend only. Its restaurants and dishes come from a file of sample data inside the app, and the user's cart and past orders are kept in a small storage area that every web browser provides for each website. Nothing leaves the phone. That was a deliberate choice for a first version: it can be demonstrated without any server at all. The code is arranged so that a real backend can be added later. One short file decides where the app's data comes from, and a comment in it notes that a hosted database service (Google's Firebase) could be swapped in "without changing consumers", meaning without touching the screens that use the data.

Following one tap

The course's second project, Options Lab, has all the parts, which makes it a better place to watch them work together. It is a research tool for a sin-

gle trader studying index options on India's National Stock Exchange. Its frontend, written with the same tools as BiteMap, shows a table of prices and some charts. Its backend is written in Python, using a library called FastAPI that takes care of receiving web requests and sending replies. Its data comes from a stockbroker's API, Angel One's SmartAPI, and it keeps its records in SQLite, a database that lives in a single ordinary file. While the project is being built, the backend can run in a "fixture mode" in which it replays a stored sample of market data instead of calling the broker, so that nobody needs to log in to a trading account to test a chart.

One of the figures the tool shows is the put–call ratio: the number of open contracts that pay out if the index falls, divided by the number that pay out if it rises. Traders watch it as a rough measure of mood. Here is what happens when the user chooses an index and an expiry date and the panel fills in.

First, the frontend asks. Its code includes a small file, `api.ts`, that collects every question it is allowed to put to the backend. All of them go through a short function that looks like this:

frontend/src/lib/api.ts

```
async function get<T>(path: string): Promise<T> {
  const res = await fetch(`/api${path}`);
  if (!res.ok) throw new Error(`steward answered ${res.status} for
  ↪  ${path}`);
  return res.json() as Promise<T>;
}
```

The word `fetch` is the browser's own instruction for sending a web request, the same kind of request a browser sends when you type an address. The function puts `/api` in front of whatever it has been asked for, sends the request, and waits for the reply. If the reply says something went wrong, the function raises an error with a readable message (the project's author calls the backend the "steward"). Otherwise it reads the JSON in the reply and hands it back to the screen that asked.

Second, the request crosses to the backend. On a developer's laptop both halves run as two programs on the same machine, and a setting passes anything beginning with `/api` to the backend on port 8000. Once deployed, the request would cross the internet instead, as in Chapter 9; the code does not change.

Third, the backend answers. FastAPI looks at the address of the request, finds the function that has been registered for it, and calls it. That function, about a dozen lines of Python, loads the option data, adds up the open contracts on each side, divides one total by the other, and returns the result. The arithmetic itself lives in a separate folder of "pure"

`bitemap/`	the whole project, one folder
`android/`	the phone app wrapper
`dist/`	the built website
`node_modules/`	libraries downloaded for you
`public/`	images and icons
`recordings/`	demo videos
`scripts/`	helper programs, such as tests
`src/`	**your code**
`index.html`	the page the browser opens first
`package.json`	the list of libraries
`vite.config.ts`	settings for the build tool
`README.md`	how to run it

Folders end in a slash. The real top level of the BiteMap project.

Figure 10.2 The top level of the BiteMap project, with what each item is for. The two folders shown in grey are produced by tools or recorded for demonstrations; the green `src` folder is the code written for this project. Everything else is settings, instructions or the finished product.

calculations that never touch the network or the disk, so that it can be tested on its own against answers worked out by hand.

The whole exchange can be seen from the terminal. With the backend running in fixture mode, the same question can be asked directly, without the frontend, using a command-line program that sends web requests:

Options Lab backend, fixture mode

```
$ curl -s "localhost:8000/api/oi/pcr?underlying=NIFTY&expiry=2026-09-08" |
↪  python3 -m json.tool
{
    "underlying": "NIFTY",
    "expiry": "2026-09-08",
    "pcr": 1.0596206817693505,
    "put_oi": 22217443.0,
    "call_oi": 20967355.0,
    "spot": 24850.15
}
```

The address carries the question: which index, which expiry. The reply is JSON: about 22.2 million open put contracts against 21.0 million open calls in the stored sample, a ratio of 1.06. Fourth and last, the frontend receives exactly this text and turns it into a number on the screen and a bar in a chart.

The database plays no part in this tap. The project's plan is for the backend to save a copy of the option data into the SQLite file every 15 minutes during market hours, building up a history that brokers do not sell cheaply to individuals. The tables exist; the program that will fill them on a timer is, at the time of writing, still on the project's to-do list. That is

normal for a project a few weeks old, and it is the kind of thing a reader of the code should notice.

What is in the folder

A project of this kind is a folder, and the folder has a conventional shape: most projects built with the same tools are laid out the same way, so a developer can find their way around a new one in minutes. Here is the top level of BiteMap:

bitemap

```
$ ls
README.md               node_modules           src
android                 package-lock.json      tsconfig.app.json
android-preview.html    package.json           tsconfig.json
capacitor.config.ts     public                 tsconfig.node.json
dist                    recordings             vite.config.ts
index.html              scripts
```

Most of these names follow conventions set by the tools the project uses. `README.md` is a plain-text note to whoever opens the folder next, explaining what the project is and how to run it. `src`, short for source, holds the code written for this project. `package.json` lists the outside packages the project depends on. `node_modules` is where those packages are downloaded to. `dist`, short for distribution, holds the finished product after a build: the version that is actually sent to users. `android` holds the Android project into which the web app is wrapped. The files ending in `.config.ts` and `.json` are settings for the various tools.

Inside `src` the same habit of naming continues. There is a folder called `pages`, with one file for each screen of the app (`HomePage`, `CartPage`, `CheckoutPage`, `TrackingPage`); a folder called `components`, for pieces that appear on more than one screen, such as a button, a rating badge or the small green-or-red mark that shows whether a dish is vegetarian; a folder called `services`, for the code that fetches and stores data; and a folder called `data`, which holds the sample restaurants. None of this is enforced by the computer. It is a convention, shared by thousands of projects, that lets a newcomer guess where things are.

Options Lab follows the same pattern twice. Its `frontend` folder looks much like BiteMap; its `backend` folder has `api` for the functions that answer requests, `core` for the calculations, `models` for the database tables

and `tests` for the checks. A `docs` folder holds the design document, a few pages of plain English setting out what the tool must do.

AT THE KEYBOARD

Read the folder before you ask for changes

When you open a project in Volume 2, whether one you started or one an AI agent built for you, it is worth spending two minutes on its folder before asking for anything. Look for the README and read it. Find where the source code lives, and whether there is a frontend and a backend. Look for a folder of tests. Notice anything that should not be there, such as a file of passwords.

You will not understand every file, and you do not need to. But a prompt that says "on the checkout page, in `src/pages/CheckoutPage.tsx`, the discount is applied twice" gives the agent far less room to guess than "the discount is wrong". Agents also read the folder first; you will steer them better if you have done the same.

Standing on other people's code

A *library* is a collection of code written to be used by other programs. The idea goes back to the subroutine libraries of EDSAC in Chapter 5: if a task has been solved once, carefully, there is no reason to solve it again. Some libraries are small and do one thing, such as drawing icons or formatting dates. Others are large enough to shape the whole program. A library of that size, which provides the skeleton of an app and asks the programmer to fill in the parts, is called a **framework**.

framework A large library that provides the overall structure of a program; the programmer fills in the parts specific to their app.

BiteMap's frontend is built on React, a library for building screens out of reusable components. It was created at Facebook by Jordan Walke and released publicly at a conference in the United States on 29 May 2013. The programmer describes what the screen should look like for any state of the data ("if the cart is empty, show this message; otherwise list the items") and React works out what must change on the screen when the data changes. It is now the most widely used library of its kind. Options Lab's backend makes a similar bargain with FastAPI: the programmer writes short functions that answer particular questions, and the framework handles everything about the web requests that carry them.

Every outside package a project uses is one of its **dependencies**, and dependencies have dependencies of their own. Collecting them all by

dependencies Outside packages that a project needs in order to build or run. Each may have dependencies of its own.

hand would be impossible, so each language has a public registry of packages and a program that fetches them. For JavaScript the registry is npm, first released by Isaac Z. Schlueter in January 2010; by most counts it now holds more than three million packages. For Python it is the Python Package Index, PyPI, which opened in 2003 and listed more than 900,000 projects in October 2026. A program that reads a project's list of dependencies and downloads all of them, together with everything they in turn depend on, is called a **package manager**.

package manager A program that downloads the libraries a project depends on, and the libraries those depend on, in the right versions.

BiteMap's `package.json` names 18 packages. Six are needed by the app itself: React and two companions for screens and navigation, an icon set, and two parts of Capacitor, the tool that wraps a web app as an Android app. The other twelve are tools used only while building: Vite, which assembles the project and runs it on the developer's laptop; TypeScript, which checks the code for certain kinds of mistake; Tailwind, which generates the styling; a checker for common errors; and Playwright, which drives a browser automatically to test the app from end to end. The package manager reads those 18 names and works out everything they require. The full list it records, in `package-lock.json`, has 239 entries. Some are alternative versions for other kinds of computer; on the Mac where this was checked, 150 packages were actually installed.

The packages that go to the user's phone are a small fraction of that. The package manager can show only the ones the app needs at run time, and how they depend on one another:

bitemap

```
$ npm ls --omit=dev --all
bitemap@0.0.0 […]/bitemap
+-- @capacitor/android@8.5.0
| `-- @capacitor/core@8.5.0 deduped
+-- @capacitor/core@8.5.0
| `-- tslib@2.8.1
+-- lucide-react@1.34.0
| `-- react@19.2.8 deduped
+-- react-dom@19.2.8
| +-- react@19.2.8 deduped
| `-- scheduler@0.27.0
+-- react-router-dom@7.18.2
| +-- react-dom@19.2.8 deduped
| +-- react-router@7.18.2
| | +-- cookie@1.1.1
| | +-- react-dom@19.2.8 deduped
| | +-- react@19.2.8 deduped
| | `-- set-cookie-parser@2.7.2
| `-- react@19.2.8 deduped
`-- react@19.2.8
```

Each line is a package and its version number; the indentation shows which package asked for which. "Deduped" means that two packages asked for the same thing and the package manager installed it once for both. Eleven packages in all reach the user. When the project is built, Vite combines them with BiteMap's own code into a single JavaScript file of about 346 kilobytes, plus a style sheet of 36 kilobytes. The other 139 installed packages, and almost all of the 203 megabytes, are the workshop rather than the product: compilers, checkers and test equipment that stay on the developer's machine.

Options Lab's Python backend tells the same story on a smaller scale: nine named packages brought in about thirty. And counted crudely, the code files in BiteMap's `node_modules` folder contain about 1.9 million lines, against the project's own 5,000.

The trade is plainly worth making: nobody building a prototype in a few weeks could also write a framework, a build tool and an Android wrapper. Its cost is that every one of those packages was written by someone, and the app now depends on all of them.

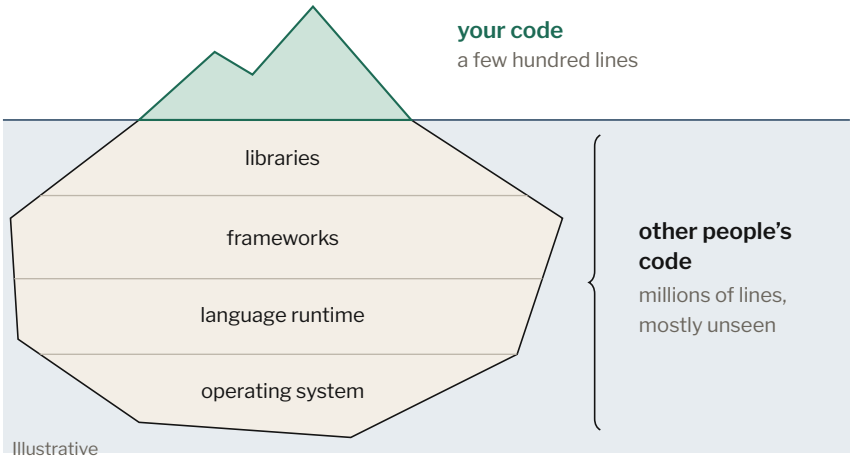


Figure 10.3 What a small app is made of. The part written for the project is the visible tip; beneath it are libraries, frameworks, the language's runtime and the operating system, written by thousands of other people. BiteMap's own code is about 5,000 lines; its downloaded packages contain roughly 1.9 million.

Why code is given away

Almost all of the packages BiteMap uses can be downloaded, read, changed and reused by anyone, free of charge. Of the 239 entries in its lock file, 182 carry the MIT licence, a short legal text that allows nearly any use as long as the copyright notice is kept. This was not always how software worked.

On 27 September 1983 Richard Stallman, a programmer at the Massachusetts Institute of Technology's AI Lab, posted an announcement headed "Free Unix!". It began: "Starting this Thanksgiving I am going to write a complete Unix-compatible software system called GNU (for Gnu's Not Unix), and give it away free to everyone who can use it." Stallman's "free" referred to freedom rather than price: the freedom to read a program's code, change it and share the changed version. The GNU General Public License of 1989 let anyone use and modify a program on condition that, if they distributed their version, they passed the same freedoms on.

Figure 10.4 Linus Torvalds, in a pencil portrait from 2016. He started the Linux kernel as a student in 1991 and the version-control system Git in 2005.

By the early 1990s GNU had most of the tools an operating system needs, but not the kernel that manages the processor and memory. On 25 August 1991 Linus Torvalds, a 21-year-old student at the University of Helsinki, posted to a discussion group: "I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones." The hobby became Linux, released under the GNU licence from 1992; with the GNU tools it made a complete free operating system.

In early 1998 a group of supporters looked for a name that businesses would find less political than "free software". At a meeting in Palo Alto on 3 February 1998, shortly after the browser company Netscape had

announced that it would publish its browser's code, Christine Peterson of the Foresight Institute suggested **open source**. Eric Raymond and Bruce Perens founded the Open Source Initiative later that month. The two terms describe largely the same software, for different reasons.

open source Software whose code is published under a licence that lets anyone read, use, change and share it.

THE BUSINESS VIEW

Why companies give code away

Firms such as Meta, Google and Microsoft publish code that cost them millions to write, for sound reasons. A company that releases a framework such as React gets thousands of outside programmers testing and fixing it, recruits who arrive already trained, and the standard that others build on.

For most firms the decision runs the other way: they are users, not publishers. Open-source code lets a small team build in weeks what would otherwise take years, and it costs nothing to license. The bill arrives elsewhere. Someone has to keep track of which packages are in use, update them when faults are found, and check that their licences permit what the business intends. Linux is the clearest example of where this leads: it now runs most of the world's servers, all of the 500 fastest supercomputers, and, as the base of Android, most of the world's phones.

Keeping track of changes

Software changes every day, and usually several people change it at once. Without some discipline, two people editing the same file overwrite each other's work, and nobody can say what the program looked like last Tuesday, when it still worked. A **version control** system solves both problems. It keeps a complete history of the project. Each time the author reaches a sensible stopping point, they record a **commit**: a snapshot of every file at that moment, with a short note in plain words saying what changed and why.

version control A system that records every change to a project's files, who made it and why, so that any earlier state can be recovered.

commit A saved snapshot of a project's files at one moment, with a short note describing what changed.

A commit works like a save point in a long document, except that every save point is kept, labelled and can be returned to. The history of a folder used in this course for early exercises shows the idea at its smallest:

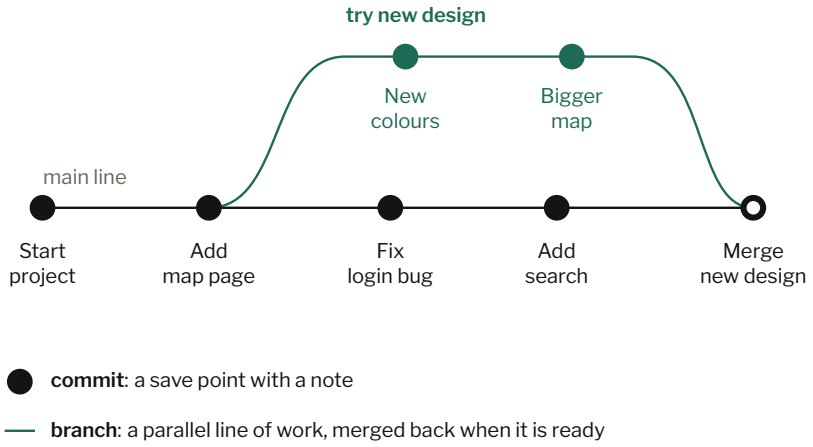


Figure 10.5 A project's history in a version-control system. Each dot is a commit, a saved state with a note. A branch lets someone try a new design without disturbing the main line, and is merged back when it works.

vibe-coding

```
$ git log --oneline
09b4243 Add arithmetic examples to hello.py
30f3c3b Add "AI fluency" line to hello.py
9b6eda1 first commit
```

Each line is one commit, newest first: an identifier worked out from the commit's contents, then the author's note. Any of these moments can be restored exactly. A team can also start a *branch*, a separate line of work for trying out a change, and merge it back when it is ready or throw it away.

The tool behind that listing is Git, and its origins lie in the Linux project. From 2002 the Linux developers had used a commercial system called BitKeeper under a free licence. In April 2005 the licence was withdrawn, after another developer tried to work out how BitKeeper communicated. Torvalds began a replacement within days; Git's first commit is dated 7 April 2005, and in July he handed its maintenance to Junio Hamano.

Git keeps its history inside the project's folder. GitHub, a website where teams keep a shared copy, opened to the public on 10 April 2008; on it anyone can look at a public project, copy it and propose changes to its owners. Microsoft bought GitHub in 2018 for US$7.5 billion in shares. By late 2025 GitHub reported more than 180 million developers, and most of the open-source packages in this chapter are developed there in public.

Volume 2, Chapter 3 shows Git in practice. The concept is what matters here. Version control makes it safe to let an agent change many files at once, because if the result is wrong, the previous commit is still there.

When a dependency fails

Borrowing code means borrowing its faults, and sometimes its owners' decisions. Three episodes from the past decade show the range of what can go wrong.

On 22 March 2016 a programmer named Azer Koçulu removed all of his roughly 270 packages from npm, after npm had given one of his package names, `kik`, to the messaging company Kik in a trademark dispute. One of the packages he removed was `left-pad`, an 11-line function that adds spaces to the left of a piece of text until it reaches a given length. Thousands of projects, including tools used to build React sites, depended on it several layers down without knowing, and stopped building. Within hours npm took the unusual step of restoring the package, and it later changed its rules so that a widely used package could not simply be withdrawn.

Log4j is a free Java library, maintained by volunteers under the Apache Software Foundation, that records what a program is doing in a log file. On 24 November 2021 Chen Zhaojun of Alibaba Cloud privately reported a flaw in it, made public on 9 December and since known as Log4Shell: an attacker could run their own code on a server simply by getting it to write a specially crafted line of text into its log. The fault had been there since 2013. It received the highest severity score, 10 out of 10, and Jen Easterly, head of the American Cybersecurity and Infrastructure Security Agency, called it one of the most serious vulnerabilities she had seen in her career. Companies spent weeks finding where Log4j lay among their dependencies.

A CLOSER LOOK

Linux, Git and the people underneath

Linux began as one student's hobby; today thousands of developers, most of them paid by companies, contribute to it, and every change is reviewed in public.

The other side is that many widely used packages are maintained by one or two volunteers in their spare time. A much-shared cartoon by Randall Munroe shows "all modern digital infrastructure" as a tower of blocks resting on one small block: "a project some random person in Nebraska has been thanklessly maintaining since 2003". When an AI agent adds a package to your project, it may be adding a dependency on someone like that.

The third case was deliberate. On 29 March 2024 Andres Freund, a Microsoft engineer, reported that two recent versions of xz Utils, a compression library included in many Linux systems, contained a hidden back door that could let an attacker with the right key take control of a machine through its remote-login service. He noticed because logging in had become about half a second slower than usual. The code had been added by a contributor calling themselves "Jia Tan", who had spent about two years winning the trust of the project's overstretched, essentially solo volunteer maintainer. It was caught before it reached most stable versions of Linux; who "Jia Tan" is remains unknown.

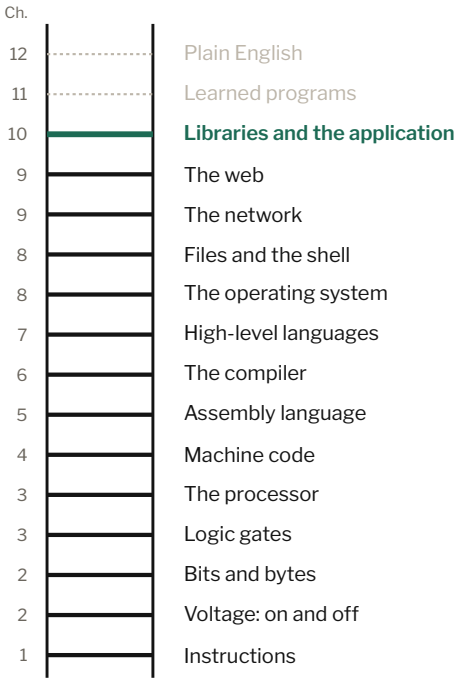
None of this makes borrowing a mistake, but an app inherits responsibilities with its packages. Package managers can check them against public lists of known faults, and the people who build an app should be able to say, roughly, what it depends on and why.

Where this leaves us

We now have the layer at which most software is built. An app is divided into a frontend, a backend, a database and other companies' services, running on rented machines. It is assembled from open-source libraries and frameworks fetched by a package manager, and its history is kept in version control so that many people, and now AI agents, can change it without losing their way. BiteMap's 5,000 lines rest on hundreds of packages, which rest on the languages, operating systems and networks of earlier chapters.

Every one of those layers has something in common. From the loom cards of Chapter 1 to the 11 lines of left-pad, each was written down, instruction by instruction, by people who understood the task well enough to spell it out. Even the largest framework is, at bottom, a long list of rules that someone thought of.

Some problems have never yielded to that approach. Nobody has managed to write down the rules for recognising a face, telling a cat from a dog or understanding a spoken sentence, though many people tried for decades. Chapter 11 describes a different kind of program, one that is learned from examples instead of written rule by rule.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

The Mark I Perceptron at the Cornell Aeronautical Laboratory in Buffalo, New York. Its camera-like eye, lit by two lamps, faces a printed letter C; the racks behind it hold the units and the motor-driven dials that changed as it learned. US Navy photograph.

CHAPTER ELEVEN

Programs That Learn

Every program in this book so far was written by people, rule by rule. Some tasks have no rules that anyone has managed to write down. This chapter follows a second kind of software, whose rules are learned from examples, from a 1943 paper about neurons to the models that now write code.

TRY TO WRITE THE RULES for deciding whether a photograph shows a cat. A cat has fur, but so does a dog, and a hairless cat has none. It has pointed ears, unless they are folded, or hidden, or the cat is facing away. It has whiskers, which may be too fine to see. Every rule needs exceptions, and every exception needs exceptions of its own. A two-year-old child can pick out a cat in a blurred photograph, from behind, in poor light, and cannot say how. Nor, it turns out, can anyone else, at least not precisely enough for a machine to follow.

The same is true of many tasks that people find easy. Reading a handwritten postcode, recognising a friend's face, hearing a spoken word through the noise of a street, telling whether a sentence is a joke: none of these comes with a list of steps. For decades programmers tried to write such lists, and the programs worked on the examples they had been designed around and failed on almost everything else.

This chapter describes how a different approach was found. Instead of writing the rules, people built programs that start with no useful rules at all, show them a large number of examples, and let them adjust themselves until their answers are mostly right. The idea is older than it looks. Most of its parts were in place by 1960, worked out by a small group of people who knew one another, read one another's papers and argued about what a brain and a machine had in common.

One conversation

Between 1946 and 1953 the Josiah Macy Jr. Foundation in New York paid for ten small conferences on a subject that did not yet have a name. Their first title was "Feedback Mechanisms and Circular Causal Systems in Biological and Social Systems", and they brought together mathematicians, engineers, physiologists, psychologists and anthropologists, among them Margaret Mead and Gregory Bateson. The chairman throughout was Warren McCulloch, a neurophysiologist. In 1948 the mathematician Norbert Wiener published a book called *Cybernetics*, from the Greek word for a steersman, about control and communication in animals and machines, and the conferences took the word into their title.

The core group included McCulloch, his young collaborator Walter Pitts, Wiener, and, in the early years, John von Neumann, whose report on the stored-program computer appeared in Chapter 4. Claude Shannon, the Bell Labs engineer of Chapters 2 and 3, came as a guest from 1950. They did not all sit in one room at once: von Neumann had stopped at-

tending before Shannon began. But they knew one another through other routes as well, at the Massachusetts Institute of Technology, in wartime research and at the Institute for Advanced Study in Princeton, and they were working on related questions. What is information? How does a system correct itself by feeding back its own errors? Is a brain a kind of computing machine?

The ideas that led to programs that learn came out of this community, more as one long conversation than as separate inventions. Three of its contributions matter here: a way of measuring language by guessing it, a model of a nerve cell as a switch, and the suggestion that learning happens in the connections between cells. A fourth, from outside the group, put the three together in a machine.

Guessing the next letter

The first idea concerns language. Shannon was not the first to notice that language has statistical patterns. Code-breakers had long used the fact that some letters are far more common than others. In 1906 the Russian mathematician Andrei Markov had described what are now called Markov chains, sequences in which each step depends on the one before, and in January 1913 he applied them to a text: he took the first 20,000 letters of Pushkin's verse novel *Eugene Onegin*, counted by hand how often a vowel followed a vowel and how often it followed a consonant, and showed that the chance of the next letter being a vowel depended on the letter before it.

Shannon's paper of 1948, "A Mathematical Theory of Communication", the paper that introduced the word *bit*, turned observations of this kind into a theory. It gave a way of measuring how much new information a message carries, and it showed that the more predictable a message is, the less information each symbol adds. To show how predictable English is, Shannon built a series of increasingly faithful imitations of it. The crudest picked letters and spaces at random:

```
XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGHYD
QPAAMKBZAACIBZLHJQD.
```

A better one picked each letter with the frequency it has in English, and a better one still chose each letter according to the two letters before it, as English does:

```
IN NO IST LAT WHEY CRATICT FROURE BIRS GROCID PONDENOME OF
DEMONSTURES OF THE REPTAGIN IS REGOACTIONA OF CRE.
```

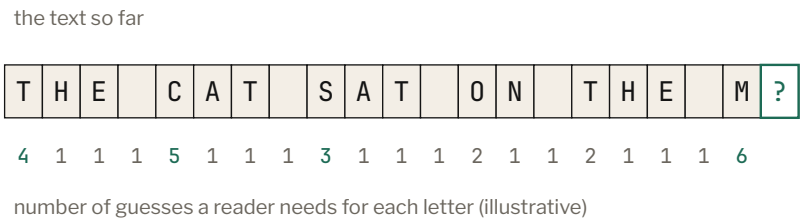


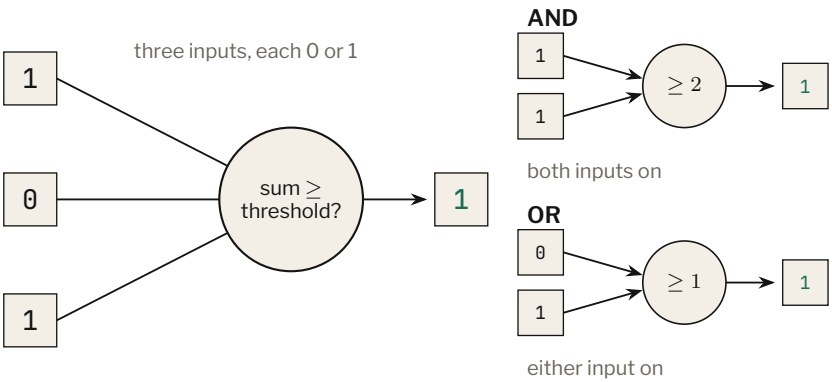
Figure 11.1 Shannon's guessing game. A reader sees the text so far and guesses each next letter. Inside a word the letters are usually guessed first time; at the start of a word there are more possibilities and more guesses are needed. The numbers here are illustrative. The fewer guesses a text needs, the less new information it carries.

Predictable text carries little new information.

He described how to produce these by hand. To choose each letter, one opened a book at a random page, read until he found the letters he had just written down, and recorded the letter that came next. Working with whole words instead of letters, choosing each word according to the word before it, he got: "THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED." It means nothing, but it has the rhythm of English, and each short stretch of it could occur in a real sentence. Shannon had made, with a book and a pencil, a small model of a language: a procedure that produces text by repeatedly choosing a likely next symbol given the ones before it.

In 1951, in a paper called "Prediction and Entropy of Printed English", he turned the method round. He showed a person a passage of text up to a certain point and asked them to guess the next letter, then the next, keeping count of how many guesses each letter took. Most letters were guessed at the first attempt; in one passage of 129 letters, 89 were. The hardest letters were usually at the start of a word, where many continuations are possible. From experiments like this Shannon estimated that ordinary English carries only about one bit of information per letter, and that it is roughly 75 per cent redundant: most of what we write could be predicted by a reader who knows the language well.

The guessing game was a way of measuring English, with a person as the guesser. Seventy years later the same game, played by a machine, became the way large language models are trained. Before that could happen, the machine needed a way to learn, and that idea came from another part of the conversation.



Threshold 2: here the sum is 1 + 0 + 1 = 2, so the output is 1.

Figure 11.3 A McCulloch–Pitts neuron adds its inputs, each 0 or 1, and fires if the total reaches its threshold. With two inputs and a threshold of 2 it behaves as an AND gate; with a threshold of 1, as an OR gate.

A nerve cell as a switch

Warren McCulloch wanted to know how a lump of living tissue could hold a thought. Walter Pitts was almost twenty-five years younger and had no degree, and would never earn one. Born in Detroit in 1923, he had, according to friends who later told the story, spent three days in a public library at about the age of twelve reading Russell and Whitehead's *Principia Mathematica*, a dense treatise on the foundations of mathematics, and written to Bertrand Russell about problems he had found in it. He left home in his teens, lived precariously in Chicago, and around 1941 was introduced to McCulloch, who took him into his family's home.

Their joint paper of 1943, "A Logical Calculus of the Ideas Immanent in Nervous Activity", started from a fact physiologists already knew. A nerve cell, or neuron, either fires, sending an electrical pulse along its fibre, or it does not; there is nothing in between. That is a binary event, and McCulloch and Pitts treated the neuron as a switch. Their idealised neuron received signals from other neurons, each either 0 or 1, added them up, and fired if the total reached a set level, its threshold.

The consequence was the one Shannon had drawn for switching circuits in Chapter 3. A unit with two inputs and a threshold of 2 fires only when both inputs are on: it is an AND gate. With a threshold of 1 it is an OR gate, and a unit whose input holds it back can act as NOT. Since every logical function can be built from these, a network of such **artificial neurons** could compute any logical function, and with enough memory anything a Turing machine could. The paper joined Turing's theory of computation to the physical brain, and engineers noticed. When von Neumann wrote his First Draft report on the EDVAC in June 1945, he described the

Figure 11.2 Walter Pitts with his friend and collaborator Jerome Lettvin, who introduced him to McCulloch, and one of the frogs from their later work on vision.

artificial neurons Simple units, loosely modelled on nerve cells, that combine their inputs and produce an output. Networks of them compute.

machine's switching elements as idealised neurons, "following W. Pitts and W. S. MacCulloch". It is the only published paper the report cites.

The McCulloch–Pitts neuron had one limitation that shaped everything that followed. Its connections were fixed. The designer decided which units were joined to which, and with what thresholds, and the network did the same thing forever. It showed that a network of simple units could compute. It said nothing about how such a network could get better at anything.

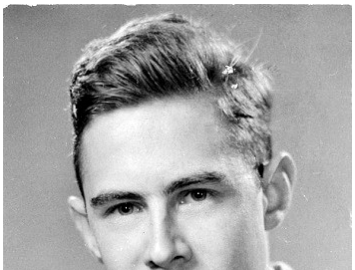
Pitts's own life ended badly. In 1952 Wiener abruptly broke off relations with McCulloch's group, after his wife made false accusations against some of its members, and Pitts, who had regarded Wiener as a mentor, was devastated. He withdrew, drank heavily, and burned his unfinished dissertation and many of his notes. He died in 1969, aged 46, four months before McCulloch.

The missing piece came from a psychologist. In *The Organization of Behavior* (1949) Donald Hebb of McGill University proposed that learning happens in the connections between neurons. When one cell repeatedly helps to fire another, he wrote, "some growth process or metabolic change takes place in one or both cells" so that the first becomes more effective at firing the second. The idea is now usually summed up as "cells that fire together wire together", but that phrase is not Hebb's; it is a later summary, popularised by the neuroscientist Carla Shatz in 1992. Hebb's point was that memory and skill need not mean new cells or new logic. They could be stored in the strengths of the connections that already exist.

A machine that turned its own dials

Frank Rosenblatt, a psychologist at the Cornell Aeronautical Laboratory in Buffalo, New York, put these ideas together in a design he called the perceptron. His first report on it is dated January 1957, and the work was paid for by the US Office of Naval Research. The perceptron took the McCulloch–Pitts neuron and added the two things it lacked. Each connection was given an adjustable strength, a **weight**, by which its signal was multiplied before the signals were added up. And there was a rule for changing the weights. Show the machine an example, let it answer, and if the answer is wrong, nudge each weight a little in the direction that would have made it less wrong. Repeat over many examples, and the weights settle at values that give the right answers.

weight A number that sets how strongly one input counts towards a unit's output. Learning means changing the weights.



Nobody tells the perceptron the rule for telling the patterns apart. It finds a set of weights that works by correcting itself against its own mistakes. That process of adjusting a system's weights from examples is called **training**. A modern computer can show the idea in a dozen lines of Python. The program below teaches a single unit the AND rule from Chapter 3, starting with every weight at zero.

training Adjusting a learning system's weights by showing it examples and correcting its errors, over and over.

perceptron.py

```
examples = [((0, 0), 0), ((0, 1), 0), ((1, 0), 0), ((1, 1), 1)]
w1, w2, bias = 0, 0, 0                  # three dials, all set to
↪ zero

for round_no in range(1, 7):
    mistakes = 0
    for (a, b), right in examples:
        guess = 1 if w1*a + w2*b + bias > 0 else 0
        error = right - guess           # 1, 0 or -1
        w1 = w1 + error*a               # nudge each dial
        w2 = w2 + error*b
        bias = bias + error
        mistakes = mistakes + abs(error)
    print("round", round_no, "mistakes:", mistakes, "dials:", w1,
    ↪ w2, bias)
```

The first line lists the four possible pairs of inputs with the right answer for each. The second sets up three dials: one weight for each input, and a third, the *bias*, which makes the unit more or less ready to fire, like the threshold of the McCulloch–Pitts neuron. In each round the program goes through the four examples. For each one it adds up the inputs times their weights, plus the bias, and guesses 1 if the total is above zero. If the guess was right, the error is zero and nothing changes. If it was wrong, each weight whose input was on moves one step towards the right answer. This is what the program prints when it runs:

Terminal

```
$ python3 perceptron.py
round 1 mistakes: 1 dials: 1 1 1
round 2 mistakes: 3 dials: 2 1 0
round 3 mistakes: 3 dials: 2 1 -1
round 4 mistakes: 2 dials: 2 2 -1
round 5 mistakes: 1 dials: 2 1 -2
round 6 mistakes: 0 dials: 2 1 -2
```

By the sixth round the unit makes no mistakes. The settings it found, weights of 2 and 1 and a bias of -2, are not the tidy ones a person would choose, but they work: only when both inputs are on does the total rise

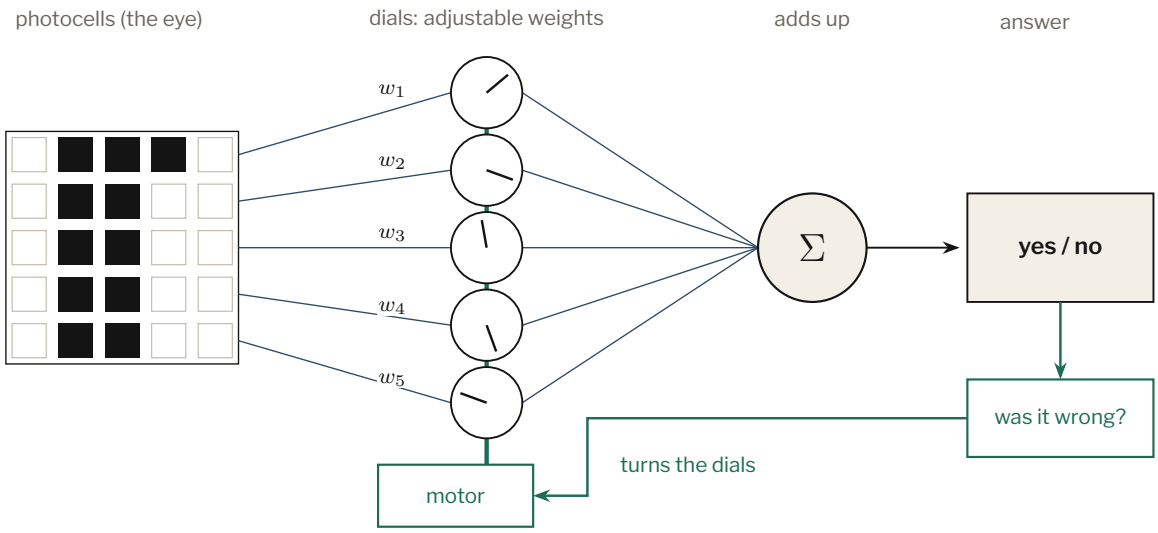


Figure 11.5 The perceptron drawn as hardware. Photocells form an eye; each sends its signal through a dial, or weight, to a unit that adds them up and answers yes or no. When the answer is wrong, a motor turns the dials a little. The Mark I had 400 photocells; this drawing shows 25 and five of the dials.

above zero. Nobody wrote the rule. The program was told only what counted as wrong.

Rosenblatt's ideas were first demonstrated in July 1958, not on special hardware but as a program on an IBM 704 computer belonging to the US Weather Bureau. After 50 trials it had learned to tell cards marked on the left from cards marked on the right. The Navy presented it to the press, and on 8 July the *New York Times* reported it under the headline "New Navy Device Learns by Doing". Its first sentence said the Navy expected the machine, eventually, to "be able to walk, talk, see, write, reproduce itself and be conscious of its existence". The gap between those expectations and a program that could tell left from right was large, and the field would pay for it.

The purpose-built machine, the Mark I Perceptron, was assembled in 1959 and first shown publicly on 23 June 1960. It was the physical form of the diagram above, filling several racks of equipment.

A CLOSER LOOK

The Mark I's motors and dials

The Mark I's eye was a grid of 20 by 20 light-sensitive cells, 400 in all, on which a camera focused a picture of a printed shape. The cells were wired, deliberately at random, to a bank of intermediate units, and these were connected to the output units through the machine's weights.

Each weight was a potentiometer, a variable resistor of the kind used as a volume knob on a radio: turning it changes how much current passes through. During training, small electric motors turned these knobs. When the machine gave a wrong answer, the motors moved the relevant knobs a little, so that the same picture would push the answer the other way next time. Learning was something you could watch and hear. The settings the machine reached were stored nowhere except in the positions of hundreds of knobs. The Mark I is now in the Smithsonian's National Museum of American History in Washington.

The reaction came in 1969, when Marvin Minsky and Seymour Papert of MIT published *Perceptrons*, a careful mathematical study of what a single layer of such units could and could not do. Minsky and Rosenblatt had attended the same high school in the Bronx, a year apart, and had long disagreed about the promise of the approach. The book proved that a single-layer perceptron can only separate patterns that can be divided by a straight line, or its equivalent in more dimensions. The simplest thing it cannot learn is the rule called XOR, "one or the other but not both". Change the examples in the program above to XOR and it never stops making mistakes: after 1,000 rounds it is still getting all four examples wrong in every round from the third on. Several layers of units could in principle handle such cases, but nobody knew how to train them, and Minsky and Papert doubted it could be done.

The book was widely read as a verdict against neural networks. That reading went further than its proofs, and funding was also shifting for other reasons, but interest and money moved elsewhere for more than a decade. Rosenblatt did not see the revival. He drowned in a sailing accident on Chesapeake Bay on 11 July 1971, his 43rd birthday.

The thaw

What had been missing was a way to train a network with several layers. The answer came in pieces. In 1970 a Finnish student, Seppo Linnainmaa, described an efficient method for working out how much each step in a long calculation contributes to its final result. In 1974 Paul Werbos, at Harvard, proposed using such a method to train neural networks. And in October 1986 David Rumelhart, Geoffrey Hinton and Ronald Williams published a short paper in *Nature* showing that it worked: networks with hidden middle layers trained this way learned useful internal features of

their own. The method, called **backpropagation**, starts from the error at the output and passes it back through the network, layer by layer, using a rule from school calculus, the chain rule, to work out how much each weight contributed to the mistake. Every weight can then be nudged at once. It is, in effect, a way of assigning blame to every dial.

backpropagation A method for training a network with many layers: the error at the output is passed backwards to work out how each weight should change.

Two further ingredients made the layers useful. Each unit applies a small bend, a nonlinearity, to its total before passing it on, such as replacing any negative number with zero. Without that bend, a stack of a thousand layers would collapse mathematically into a single layer and could still draw only straight lines. With it, depth becomes useful: early layers can learn to detect edges in a picture, later ones shapes built from edges, and later ones still the parts of a face.

Figure 11.6 Geoffrey Hinton in 2024. He co-wrote the 1986 paper that made backpropagation widely known and, with two students, the 2012 image-recognition system usually taken as the start of the present boom.

Even so, neural networks were not the obvious winner. For about fifteen years from the mid-1990s other methods, such as the support vector machines described by Corinna Cortes and Vladimir Vapnik in 1995, were often as accurate and easier to use. There were exceptions: networks designed by Yann LeCun and his colleagues at Bell Labs were reading a large share of the cheques written in the United States by the late 1990s. But most researchers in machine learning worked on other things.

What changed was scale. The internet produced vast amounts of labelled examples: photographs with captions, text in every language. And graphics processing units, or GPUs, chips designed to draw the images in video games, turned out to be very good at the arithmetic neural networks need, which consists mostly of multiplying large grids of numbers. In 2012 Alex Krizhevsky, Ilya Sutskever and Hinton, at the University of Toronto, entered a deep network trained on two gaming GPUs for about a week in the ImageNet competition, in which programs label photographs from a thousand categories. It got the right label in its top five guesses for all but 15.3 per cent of the test images; the next-best entry, using other methods, missed 26.2 per cent. Within a few years almost every serious system for recognising images or speech was a neural network.

The architecture won for practical reasons more than biological ones. Its “neurons” are a long way from real nerve cells; each is simply a weighted sum of inputs passed through a bend. What mattered was that it is general enough to learn almost any pattern, that a single training method works for all of it, and that it keeps improving as it is given more data and more computing power, on hardware that was getting faster every year.

A program nobody wrote

From the point of view of this book, the important change is not about brains. It is about where programs come from.

In Chapter 4 the stored-program computer put the instructions in memory, as numbers, beside the data. A learned program goes one step further: the most important numbers, the weights, are not written by anyone. A person chooses the general shape of the network, gathers examples, and says what counts as a wrong answer. Training then adjusts the weights, millions or billions of them, until the answers are mostly right. The result, the shape plus its trained weights, is called a **model**, and it is run like any other program. Nobody has to know the rule for recognising a cat. The rule, such as it is, ends up spread across the weights.

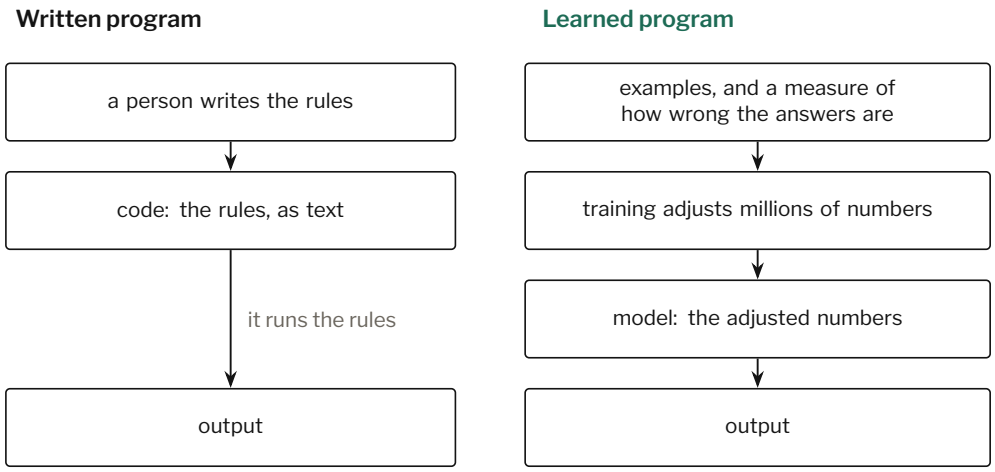
model A trained network: its fixed structure plus the learned weights. It is run like a program but was not written as one.

Turing had sketched this possibility in 1948, in a report for Britain's National Physical Laboratory called "Intelligent Machinery". He described "unorganised machines", random networks of simple units whose connections could be modified by training, and suggested that a machine might be educated as a child is instead of programmed in full. The laboratory's director is said to have dismissed it as a schoolboy essay, and it was not published until 1968. In 2017 Andrej Karpathy, then in charge of AI at Tesla, gave the idea its software form in an essay called "Software 2.0". In traditional software, he wrote, people write explicit instructions. In the new kind, they specify a goal, such as matching a set of examples, and let computing power search for a program that meets it. The weights are the code, and "No human is involved in writing this code".

Neural networks are not just another classifier, they represent the beginning of a fundamental shift in how we write software.

ANDREJ KARPATHY, "SOFTWARE 2.0", 2017

The dials of the Mark I have not disappeared; they have shrunk. A modern model's weights are numbers stored in the memory chips beside a GPU, built from the same transistors as everything else in Chapter 2. Running the model means streaming those numbers through the processor's arithmetic units. Training means overwriting each of them with a slightly adjusted value, billions of times over. Rosenblatt's motor turning a knob has become a processor writing a new number into memory.



Who writes it?	a person	training, from data
How do you fix it?	edit the rule	change the examples, train again
How do you check it?	read it, then test it	test it on new examples
Can you read its rules?	yes	not really: they are numbers

Figure 11.7 Two ways of making a program. On the left, a person writes rules as code and the computer runs them. On the right, people supply examples and a measure of how wrong the answers are, and training adjusts millions of numbers until the answers improve. The table beneath compares how each kind is written, fixed, checked and understood.

The comparison in the figure has practical consequences. A written program can be read: if it goes wrong, someone can find the line that is wrong and change it. A learned model cannot be read in that way. Its behaviour is the combined effect of millions of numbers, none of which means much on its own. It is fixed by changing the examples or the training and training again, and it is checked by testing it on examples it has not seen. A written program fails in ways its authors can usually explain. A model can be right on a thousand examples and wrong on the next, with equal confidence.

THE BUSINESS VIEW

When the data is the source code

For a firm, learned software changes where the value and the risk lie. With a written program the asset is the code and the people who understand it. With a model it is also the data it was trained on: a bank

that has twenty years of labelled fraud cases can train a better fraud detector than a newcomer, whatever code either of them writes. Collecting, cleaning and labelling data becomes engineering work.

Learned software also has to be checked in a different way. Nobody can read a model's rules to confirm that it treats customers fairly or follows a regulation. It has to be tested on many examples, including awkward ones, and watched after it is put into service, because the world it meets may drift away from the examples it learned from. Firms that adopt AI tools take on that testing, whether or not they built the model themselves.

Guessing the next word, at scale

The two halves of this chapter meet here. Shannon measured language by how well a reader could guess the next letter. Suppose that, instead of a person, the guesser is a neural network, and that it is trained on the guessing game itself: shown a passage, asked for the next word, told how wrong it was, and adjusted. Repeat that over a large part of all the text that has been published online, and the result is a **language model**.

language model A model trained to predict the next word (or piece of a word) in a text. Large ones are trained on vast amounts of writing.

To guess the next word well, a model has to absorb a great deal: spelling and grammar, which facts tend to go together, how an argument usually proceeds, what a polite reply to a complaint looks like. None of this is written into it. It is whatever the weights had to capture for the guesses to improve. Once trained, a language model produces text the way Shannon did with his book, one step at a time: it guesses a likely next word, adds it to the text, and guesses again.

Modern language models are built on a design called the transformer, introduced by researchers at Google in 2017. Its central idea, called attention, lets the model weigh every earlier word in a passage when it guesses the next one, so that a word at the end of a paragraph can depend on a name mentioned at the start. It also suits GPUs well, which made it practical to train on far more text than before. The internal workings of these models are the subject of a book of their own; the sibling volume to this course, *From Switches to Souls*, describes them in detail.

The text these models learn from includes code. Programmers publish an enormous amount of it, in open-source projects of the kind described in Chapter 10, along with the questions and answers, tutorials and documentation that surround it. In 2021 a team at OpenAI described Codex, a language model trained further on 159 gigabytes of Python taken from 54

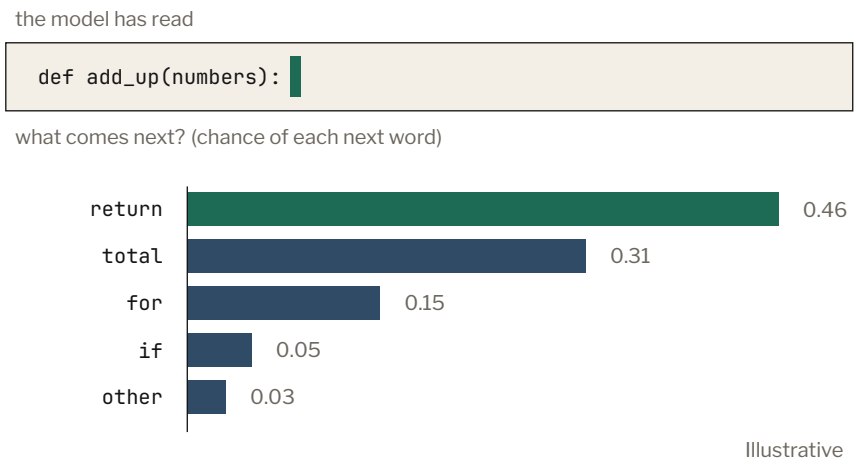


Figure 11.8 A language model reading the first line of a Python function and giving its chances for the next word. The numbers are illustrative. The same next-word guessing that completes English sentences also completes programs, because programs were part of what the model learned from.

million public projects on GitHub; given the first line and a description of a function, it could often write the rest. A version of it powered GitHub Copilot, an early and widely used AI assistant for writing code. General-purpose models now include code as a deliberate part of their diet: Meta reported in 2024 that about 17 per cent of the training text for its Llama 3 models was code. Most companies do not publish such figures.

To a language model, a program is one more kind of text with strong patterns. After `def add_up(numbers):` certain words are very likely and others are not, just as after "THE CAT SAT ON THE M" a reader expects an A. A model that has seen millions of functions that add up lists can produce one more, and, because it has also read the English around the code, it can produce one in response to a request written in English. The guessing game Shannon played with a book and a pencil, run on billions of weights trained on the world's text and code, produces programs.

AT THE KEYBOARD

Confident, and sometimes wrong

When an AI agent writes code for you in Volume 2, it will do so by this kind of guessing: producing the code that is most likely, given your request, everything else in the conversation and everything it learned in training. Most of the time the most likely code is also correct, because correct code is what it mostly saw.

But a guess is not a rule followed. The model has no separate step in which it checks that the code is right, and it writes a wrong line in the same fluent, confident style as a right one. It may call a library function that does not exist because one with that name would be likely, or handle the common case and quietly miss the unusual one. This is why Vol-

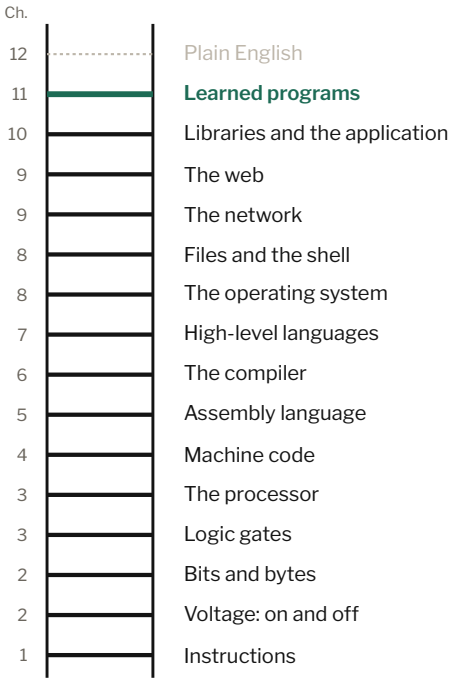
ume 2 asks you to run what the agent produces, test it on real examples and read the results, and why Chapter 8 of that volume is about what to do when it breaks.

Where this leaves us

We now have a second kind of program. The first kind, from Jacquard's cards to the apps of Chapter 10, is written: a person works out the rules and writes them down. The second kind is learned: a person supplies examples and a measure of error, and training adjusts millions or billions of weights until the errors are small. The pieces of the second kind came from one community of researchers. Shannon gave the task, guessing the next symbol, and showed that language is full of predictable pattern. McCulloch and Pitts showed that a network of simple neuron-like switches could compute. Hebb proposed that learning lives in the strengths of the connections. Rosenblatt made those strengths adjustable and built a machine that tuned them against its own mistakes. Most of what has happened since is those ideas run at a scale none of them could have imagined, on hardware they would not have recognised.

What a learned program gives up is the ability to read its rules. It can be tested but not inspected line by line, and its mistakes come without warning and without explanation.

A language model trained on the guessing game over a large share of the world's text, including billions of lines of code, can turn a request in English into a working program. Chapter 12 describes what happened when such models were put to work as programming tools, and what it means for English to become a programming language.



THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

A Franklin typewriter from the 1890s. For well over a century people have put words into machines by pressing keys. The newest layer of software takes those words, typed in plain English, as a request for a program.

CHAPTER TWELVE

English Becomes Code

For seventy years people hoped to give computers their instructions in ordinary language. Since 2021 it has become possible, through programs that learned to write code from the text of the internet. This chapter follows one plain request down every layer of the book, and asks what is left for the person who makes it.

THE TASK OF ADDING UP a list of numbers has appeared in this book as switches, as raw numbers, as assembly code and in several programming languages. In October 2026 it can also be done like this. A person opens a terminal in a folder that holds a file of numbers, starts an AI coding agent, and types a sentence asking for a program. The agent writes the program, runs it, checks the answer and reports back, in English, within a few seconds. The person who asked need not know any Python at all.

Chapter 1 promised that the book would end at this point, with plain English as the newest layer on top of all the others. This chapter describes how that layer arrived and what it does. It begins with an ambition much older than artificial intelligence, the hope that programs could be written in ordinary language, and with the strongest argument against it. It then traces the tools that brought the ambition within reach, follows a single English request all the way down to the voltages of Chapter 2, and ends with what the newest layer leaves for people to do.

An old ambition

The idea that computers might one day be programmed in English is almost as old as programming. Chapter 6 described how Grace Hopper's team built FLOW-MATIC in the 1950s around English words, and how COBOL, designed by a committee in 1959, followed it so that managers and auditors could read the programs that ran their businesses. Early compilers were sold as "automatic programming", on the understanding that the computer would now write the program. Each of these steps was real progress. None of them let people write in English as they speak it. COBOL used English words, but in a fixed order with fixed meanings, and a misplaced full stop could change what a program did.

The reason was spelt out most sharply by Edsger Dijkstra, whose campaign for structured programming appeared in Chapter 7. In a short essay written around 1978 and titled "On the foolishness of 'natural language programming' ", he argued that the strictness of programming languages was their great advantage. A formal notation has a few simple rules, and those rules are, in his words, "an amazingly effective tool for ruling out all sorts of nonsense that, when we use our native tongues, are almost impossible to avoid." Ordinary language is ambiguous, and the ambiguity is usually harmless, because a listener fills the gaps from context. A machine following instructions cannot be trusted to fill them

correctly. Dijkstra concluded that machines to be programmed in our native tongues would be "as damned difficult to make as they would be to use."

He was right about the first half for nearly half a century. Every attempt to accept free English, as opposed to English-flavoured code, failed or shrank into something narrower. On the second half the argument is still open. It is now possible to give a computer instructions in ordinary language and get a working program back, which Dijkstra doubted. Whether that makes programming easy, or only moves its difficulty somewhere else, is the main question of this chapter.

A CLOSER LOOK

Seventy years of "almost here"

Claims that ordinary people would soon be able to program without programmers have come round regularly. In the 1950s compilers were advertised as "automatic programming". FLOW-MATIC and COBOL used English words so that business people could read, and it was hoped write, their own programs. In 1974 the database language that became SQL was first called the Structured English Query Language.

In 1981 a British company, D.J. "AI" Systems, released a program called The Last One. It generated programs in the BASIC language from a series of menu choices, and was marketed on the idea that it would be the last program anyone needed, since it would write all the others. In the same years James Martin's book *Application Development Without Programmers* popularised "fourth-generation languages", tools for producing database reports and forms with little code. Later decades brought visual programming tools and expert systems with similar promises.

Each of these was useful, and none abolished programming. Fred Brooks, writing in 1986, quoted the computer scientist David Parnas on the pattern: "automatic programming always has been a euphemism for programming with a higher-level language than was presently available to the programmer." Each time the language rose a level, more software was written by more people, not less. That is a pattern from history, not a law, and the present change is larger than any of the earlier ones. But it is a reason to treat confident predictions with care.

Help before the machines could write

Long before any machine could write a program, programmers had tools to help them write their own. From the 1980s onwards most professional programming was done in an *integrated development environment*, a single program that combined a text editor with the compiler, the debugger and a view of the project's files. By the 1990s such editors offered autocomplete: type the first few letters of a name and the editor suggests the rest, drawn from the names defined in the program and its libraries. The editor did not understand the program. It knew which names existed and offered them, which saved typing and caught many spelling mistakes before they reached the compiler.

The bigger help came from other people. Every language had its reference manuals, and every library its documentation, but much of a working programmer's day was spent finding out how someone else had solved a problem. For years this meant books, mailing lists and forums of mixed quality. In September 2008 two American programmers, Jeff Atwood and Joel Spolsky, launched Stack Overflow, a free question-and-answer site for programmers on which answers were voted up or down and the best rose to the top. Within a few years it was where most programming questions were answered. A programmer stuck on an error message would paste it into a search engine, find a Stack Overflow page where someone had met the same error, and adapt the accepted answer.

That habit, search, copy, adapt, was the everyday practice of programming for more than a decade, and it matters for what came next. Stack Overflow's millions of questions and answers, together with the public code on GitHub described in Chapter 10, are the kind of text from which language models later learned to write code. The answers that programmers wrote for each other became part of what the machines learned from.

Models that write code

Chapter 11 explained how a network trained to predict the next word over a vast amount of text, including a great deal of public code, becomes a large language model, and why the same guessing game produces programs. An early and widely used product built on this was GitHub Copilot, announced as a preview on 29 June 2021. It ran inside the program-

mer's editor and was built on a model from OpenAI called Codex, trained on code from GitHub. As the programmer typed, Copilot suggested what might come next, often a whole function, shown in grey until accepted. A comment such as "check whether this email address is valid" could be enough for it to propose the code. It became generally available a year later, at ten dollars a month for individuals.

On 30 November 2022 OpenAI released ChatGPT as a free "research preview". It was not built for programmers, but it could hold a conversation in English and write code on request, and many of its first users discovered that it would explain an error message, write a short script or translate a program from one language to another. Within five days, according to OpenAI's chief executive, it had more than a million users. For the first time, people with no programming experience could describe a small program and get one back.

Two months later Andrej Karpathy, the researcher whose essay on "Software 2.0" appeared in Chapter 11, summed up the moment in one line on social media.

The hottest new programming language is English

ANDREJ KARPATHY, 24 JANUARY 2023

The line was half a joke and half a description. English was not a programming language in Dijkstra's sense: it had no fixed rules, and the same sentence could produce different programs on different days. But for the first time it could serve as the input to a chain of translations that ended in a running program, which is what a programming language had always been for. Figure 12.1 shows how the share of the work done by the machine grew over these years, from the end of a word to a whole function.

From suggestions to agents

Copilot and ChatGPT made suggestions. A person still had to copy the code into a file, run it, read the error, and paste the error back. In 2025 and 2026 a new kind of tool took over those steps. An AI **coding agent** works inside a project folder. Given a request in English, it reads the relevant files, writes or changes code, runs the program or its tests, reads what the computer prints back, and repeats until the job seems done. It then reports what it did. Claude Code, from Anthropic, is the agent used

coding agent A program, driven by a language model, that reads a project's files, writes code, runs commands and checks the results in a loop.

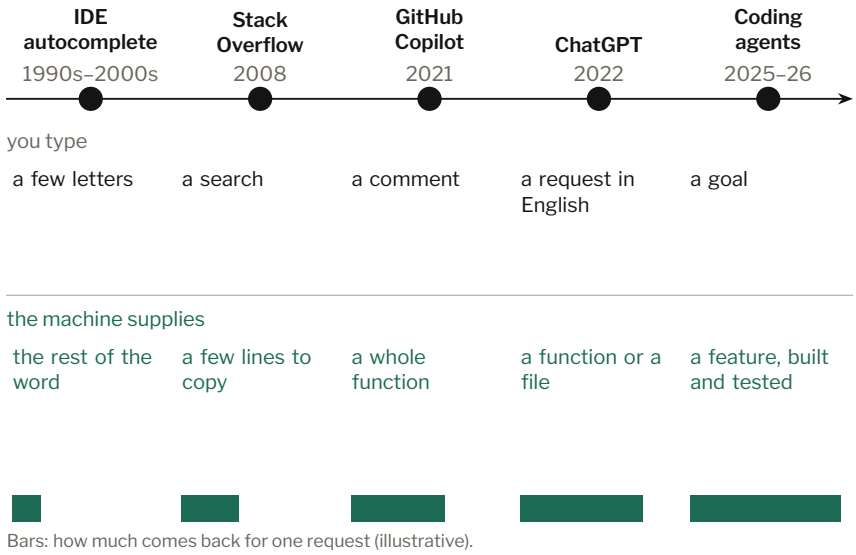


Figure 12.1 Help for programmers, from editors that finish a word to agents that build a feature. Over 30 years the request has grown more general, from a few letters to a goal, and the machine's share of the work has grown with it.

in Volume 2; Cursor, OpenAI's Codex and GitHub Copilot's agent mode do broadly similar work. These products change every few months, and their names and features will have moved on by the time you read this.

Here is a real exchange of that kind, run for this chapter in a folder holding the file of numbers from Chapter 8. The `-p` option tells Claude Code to answer one request and stop; normally the conversation would continue in the same window.

ch12

```
$ claude -p "There is a file called numbers.txt in this folder with one
↪  number on each line. Please write a short Python program called sum.py
↪  that adds them up and prints the total, then run it to check that it
↪  works."
Done. `sum.py` reads `numbers.txt`, skips blank lines, and prints the
↪  total.

```python
with open("numbers.txt") as f:
    total = sum(int(line) for line in f if line.strip())

print(total)
```

Running it prints `31` — correct for the three values in the file (3 + 8 +
↪ 20). Note it resolves `numbers.txt` relative to the current directory,
↪ so run it from `ch12/`.
```

The agent created the file, ran it with the Python interpreter, compared the result with the numbers it had read, and said so. It also made decisions nobody asked it to make. It chose to skip blank lines, and it

chose to treat every number as a whole number, which is what the `int` on the second line means. Both choices suit this file. Whether they suit the next one is a question the request did not answer, and we will come back to it.

In February 2025, three and a half years after the Copilot preview, Karpathy gave a name to the most relaxed way of working with such tools. "There's a new kind of coding I call 'vibe coding'," he wrote, "where you fully give in to the vibes, embrace exponentials, and forget that the code even exists." He described accepting every change without reading it and pasting error messages back to the AI until they went away, and said that this was "not too bad for throwaway weekend projects". **Vibe coding** in that original sense meant building software without looking at the code at all. The phrase spread far beyond its first meaning, and in November 2025 Collins named it its Word of the Year, defined as using AI prompted by natural language to write computer code.

Vibe coding Building software by describing it to an AI and accepting the code it writes without reading it.

The tools changed quickly in the year that followed, and so did the way professionals used them. A year after his first post, Karpathy suggested a different name for the careful version, *agentic engineering*: the programmer rarely writes the code directly, but directs agents that do, and acts as the oversight on their work. Between not reading the code at all and checking every line lies a range of practice, and much of Volume 2 is about choosing where on that range a given project should sit.

One request, all the way down

An English request does not replace the layers described in this book. It sits on top of them, and every one of them is still at work when the request is carried out. Figure 12.3 follows a request through the layers, from a sentence to a voltage.

Suppose someone working on a small web app asks an agent to "add a button that shows today's total". The words themselves already pass through several layers before any program is written. They are stored as numbers, one or more bytes per character in the Unicode encoding of Chapter 2, sent across the network in packets as in Chapter 9, and fed to a language model running on processors in a data centre, where, as Chapter 11 described, billions of learned numbers are multiplied together to choose the reply one piece at a time.

The reply includes a few lines of JavaScript, the language of web pages from Chapter 7. The agent saves them in a file inside the project, beside

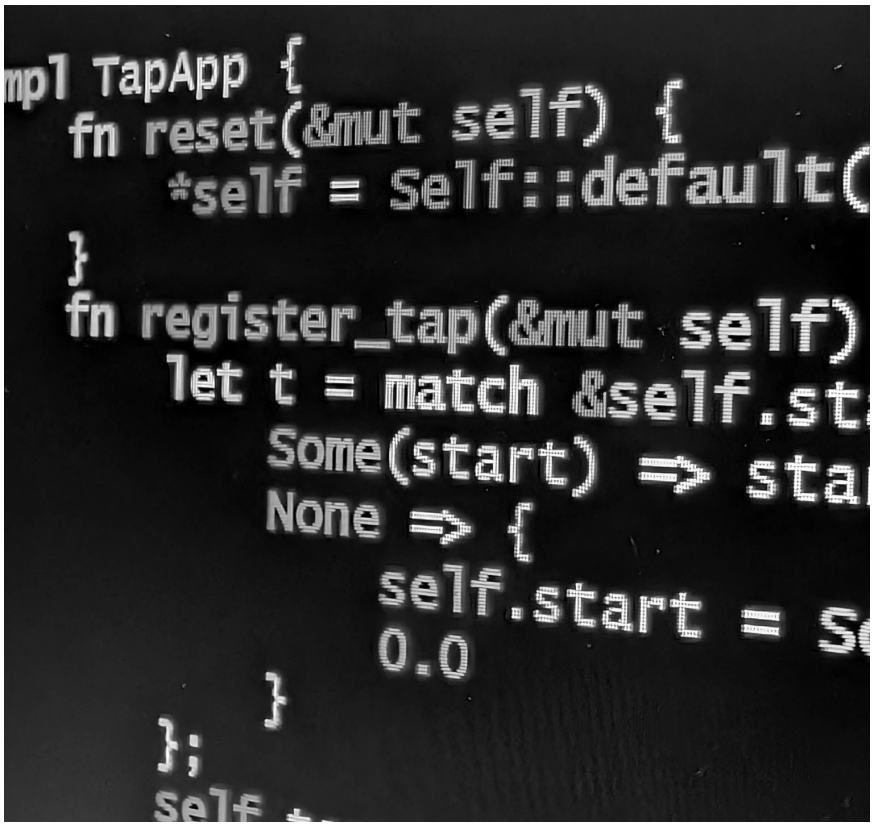


Figure 12.2 Source code on a screen. However a program is requested, what the agent produces is still text in a programming language, saved in files, which other programs translate further down.

the libraries the project already depends on (Chapter 10). When the page is opened, the browser reads the JavaScript and, as Chapter 6 described for compilers, translates it into the processor's own instructions; modern browsers do this while the page is running. The operating system of Chapter 8 decides when the browser gets its turn on the processor and which part of memory it may use. The processor then fetches those instructions one by one, as numbers, and carries them out, as in Chapters 3 and 4. And each number, at the bottom, is a pattern of high and low voltages in the switches of a chip.

None of this is new. The browser, the operating system and the processor would treat the agent's JavaScript exactly as they would treat the same lines typed by a person. What the newest layer changes is the top step, the one between what someone wants and the first line of code. For seventy years a person had to take that step, by learning a programming language. Now a program can take it on their behalf.

There is one important difference between this step and every step below it. A compiler follows fixed rules, so the same program always produces the same machine code, and a program with a mistake in its grammar is refused. The agent works from patterns learned from examples. It

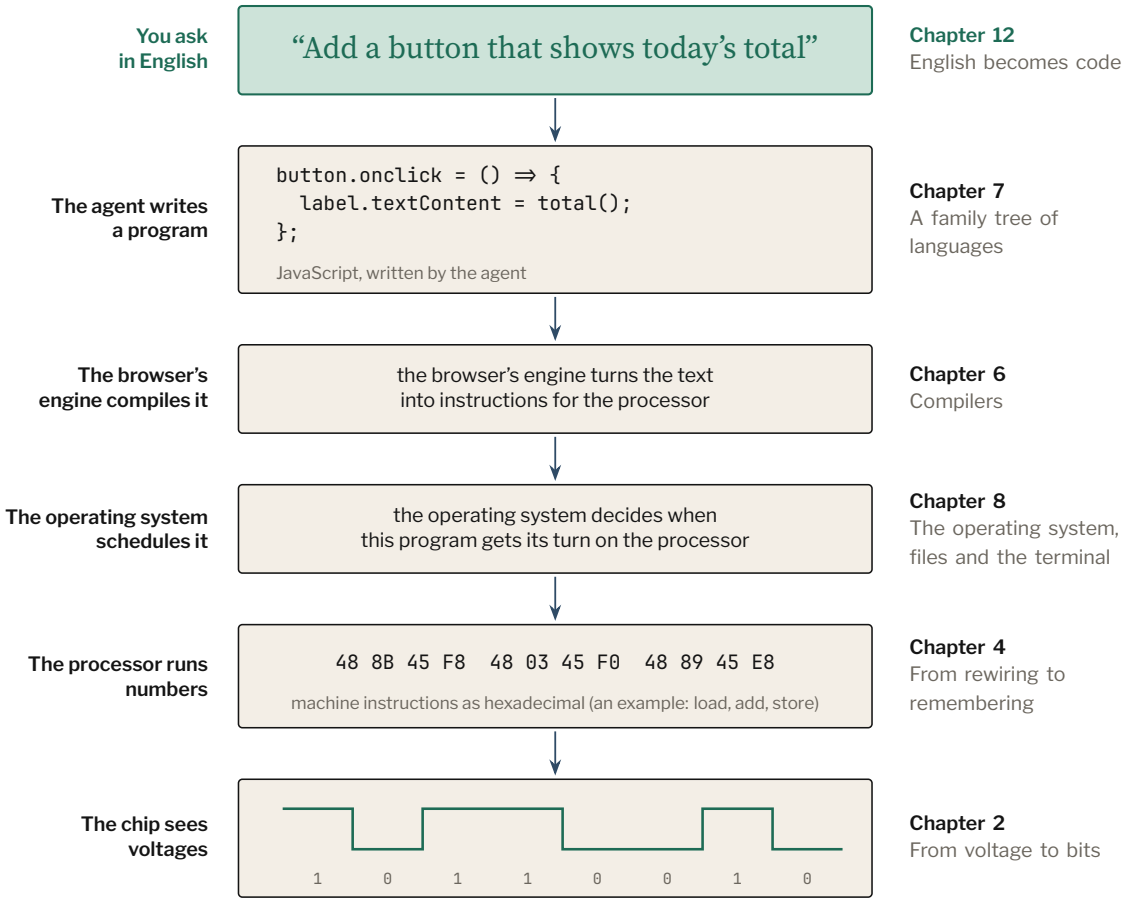


Figure 12.3 One request, all the way down. A sentence of English becomes a few lines of JavaScript written by the agent; the browser compiles them into machine instructions; the operating system gives them a turn on the processor; the processor carries out numbers; and the chip sees only voltages. Each step is labelled with the chapter that explained it.

accepts any English at all, fills the gaps with its best guess, may produce a different program if asked twice, and rarely refuses. Every layer below the top one is exact. The top one is not, and the person who asks has to allow for that.

Every layer leaks

In November 2002 Joel Spolsky, who would co-found Stack Overflow six years later, published an essay called "The Law of Leaky Abstractions". Its law was a single sentence: "All non-trivial abstractions, to some degree, are leaky." An abstraction, as Chapter 1 explained, hides the details of a lower layer behind a simpler way of working. Spolsky's point was that the hiding is never complete. His main example was the network: the Internet's TCP protocol promises that messages will arrive whole and

in order over cables that sometimes lose them, and almost always keeps its promise, but when a cable is cut or a link is slow, the problem below shows through and no amount of knowledge about the layer above will explain it. An abstraction whose lower layers show through when something goes wrong is called a **leaky abstraction**.

leaky abstraction A layer that hides the detail beneath it until something goes wrong, when that detail shows through.

Plain English is the highest abstraction in this book, and it leaks like the others. Return to the program the agent wrote above. If a decimal number, 4.5, is added to the end of the file, the program stops working.

ch12

```
$ python3 sum.py
[…]
    total = sum(int(line) for line in f if line.strip())
                ~~~^^^^^^
ValueError: invalid literal for int() with base 10: '4.5\n'
```

The request was made in English, but the error comes back from a lower layer, in that layer's language. It quotes a line of Python, points at the `int` in it, and complains that `'4.5\textbackslash n'` is not a whole number in base 10, the `\textbackslash n` being the invisible character that ends a line in the file. Nobody who asked for "a program that adds up the numbers" mentioned whole numbers. The agent had filled a gap with a guess, as Chapter 1 warned it would, and the guess was wrong for this file.

Pasting the error back to the agent fixed it. The agent changed `int` to `float`, the Python name for a number with a decimal point, ran the program again and reported the new total, 35.5. It also mentioned that it had added a rule of its own, so that a file of whole numbers would still print 31 and not 31.0, and it said that it had checked only the first of those two cases by running the program: "that half is reasoned, not run". That is a good report. It says what was done, what was decided without being asked, and what was not tested. A person who can read such a report, and knows roughly what an interpreter, a file and an error message are, can decide whether to accept the change. A person who cannot is reduced to pasting errors back and hoping.

Most of the time the leaks are small, like this one. Sometimes they are not: a page that works on the developer's laptop and fails on a phone, a database that was left open to anyone on the Internet, a bill from a cloud provider for a program that kept running all night. In each case the cause lies in a layer the English request never mentioned. Knowing that those layers exist, and roughly what each one does, is what lets someone guess where to look.

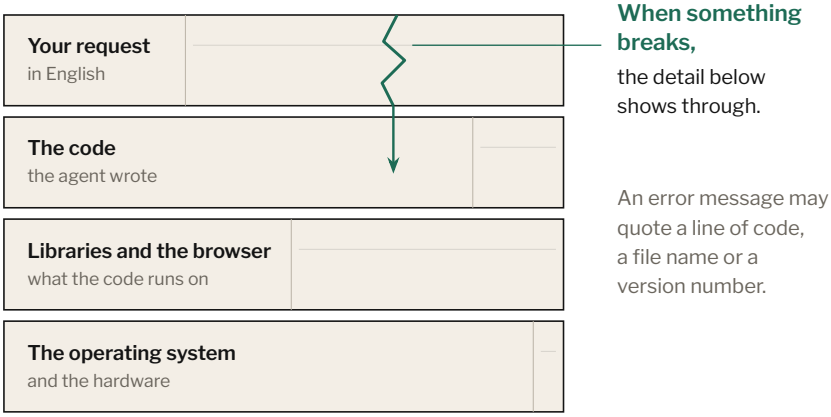


Figure 12.4 Every layer leaks. A request in English hides the code, the code hides the libraries and the browser, and those hide the operating system and the hardware. When something breaks, an error message from a lower layer shows through, quoting a line of code, a file name or a version number.

AT THE KEYBOARD

When you open Claude Code

When you start an agent in Volume 2, you will type requests like the one above, and the agent will reply with a summary of what it did. Read the summary as you would a report from a capable new colleague. Look for three things: what it changed, what it decided on its own, and what it says it tested. If it does not say what it tested, ask.

When an error appears, notice which layer it comes from. A message that quotes a line of code comes from the program; one that mentions a missing package comes from the libraries of Chapter 10; one about a refused connection comes from the network of Chapter 9; one about permissions comes from the operating system of Chapter 8. You do not need to understand the message fully. Knowing which floor of the building it comes from is usually enough to ask a better question. Chapter 8 of Volume 2, “When It Breaks”, turns this into a routine.

The hard part was never the typing

If a machine can now write the code, what is left? The best answer was given almost 40 years before the machines could do it. In 1986 Fred Brooks, whose experience managing IBM’s System/360 software appeared in Chapter 7, presented an essay called “No Silver Bullet”. He divided the difficulties of building software into two kinds. The *accidental* difficulties come from the tools: remembering the exact grammar of a language, keeping track of memory, typing the same routine code again and again. The *essential* difficulties come from the problem itself:

working out what the software should do, in every case, and making sure it does it. Better tools, he argued, could remove much of the first kind, as high-level languages already had, but could not remove the second, because it was the work itself.

The hardest single part of building a software system is deciding precisely what to build.

FRED BROOKS, "NO SILVER BULLET", 1986

AI agents have removed a great deal of accidental difficulty, more than any earlier tool. Someone who has never learned a programming language can now get a working program without knowing where the brackets go. The essential difficulty is where it was. The agent that wrote `sum.py` did not know whether the file might contain decimals, because the person asking had not said, and probably had not thought about it. Deciding such things in advance is what Brooks meant by deciding precisely what to build. A clear statement of what a program must do, written before it is built, is called a **specification**, and Volume 2 spends a chapter on writing one.

specification A clear statement, written in advance, of what a program must do, including the unusual cases, and how to tell whether it does.

Seen this way, Dijkstra's objection has not been refuted so much as relocated. English is still imprecise. The agent copes with the imprecision by guessing, and its guesses are often good, because it has seen a great many similar requests. But the precision that a programming language used to force on the programmer now has to be supplied on purpose, by the person asking: in what they specify, in the examples they give, and in the checks they make afterwards. Whether all this makes software faster to build is itself debated. In one careful study in 2025, experienced developers working on code they knew well took about a fifth longer with AI tools, while believing they had been faster; later studies, with newer tools, have pointed towards modest gains. The answer depends on who is doing what, and the tools change faster than the studies can be run.

THE BUSINESS VIEW

What changes for managers

For most of the history of software, building even a rough version of an idea took a programmer, and usually a team, weeks or months. A manager who wanted to test an idea wrote a document and waited. AI agents have changed that arithmetic for small things. A person who is not a programmer can now build a working prototype of a simple app

> in an afternoon, show it to customers, and change it the next day. The people who can build software now include many who could not before.
>
> What has not changed is the cost of being wrong. A prototype that works in a demonstration is not yet a product that thousands of people can rely on, with their data kept safe and their payments handled correctly. The scarce resource moves from writing code to judging it: deciding what is worth building, specifying it precisely, checking that it works, and knowing when something needs an experienced engineer. Those are management skills as much as technical ones, which is one reason this course is taught in a business school.

There remains the question Chapter 1 left open. Ada Lovelace wrote in 1843 that the Analytical Engine "has no pretensions whatever to originate anything. It can do whatever we know how to order it to perform." Alan Turing, in 1950, called this "Lady Lovelace's objection" and argued that it might not hold for machines that learn. Chapter 11 showed that the programs inside a language model were not ordered by anyone, line by line; they were learned. The agent that wrote `sum.py` was not told to skip blank lines. In a narrow sense, then, it originated something.

In a wider sense the question is open, and people who have thought hard about it disagree. Everything the model produces is shaped by text that people wrote, and it acts only when someone asks. What has changed for certain is the second half of Lovelace's sentence. What we "know how to order it to perform" has widened enormously: a vague sentence now gets a working program. What has not changed is that someone has to want the program, say what it is for, and decide whether the result is right.

Good enough to steer

Chapter 1 described the stance of the course this book belongs to. You are visiting the city of software, not moving there. A visitor does not need to know how every building was constructed, but one who knows the layout of the streets gets lost less often. The chapters since have been that layout: the switches, the numbers, the languages, the operating system, the network, the libraries and the learned programs, each resting on the one below.

Most education carries a silent expectation: become good at this. A student of accounting is expected, in the end, to be able to do the accounts. This course asks for something different. The agent can already

write code that is good in the narrow sense of fluent and usually correct. The person directing it needs to be good enough: good enough to know what to ask for, good enough to tell a decent result from a bad one, and good enough to steer when the work goes off course. That is a smaller demand than becoming a programmer, but it is not a small one, and it is the kind of generalist's competence that management education has always aimed at.

For a long time one argument for teaching programming to everyone was that code teaches clear thinking. A compiler forgives nothing, so writing for it forces you to say exactly what you mean. That argument has weakened, since the machine now meets people halfway and forgives almost anything. But the rigour it taught has not disappeared. It has changed hands. The people who build compilers, operating systems, libraries and language models still need all of it, because nothing covers for them. The person directing an agent needs it in a different form: in a precise request, a sensible check and the habit of asking what was decided on their behalf.

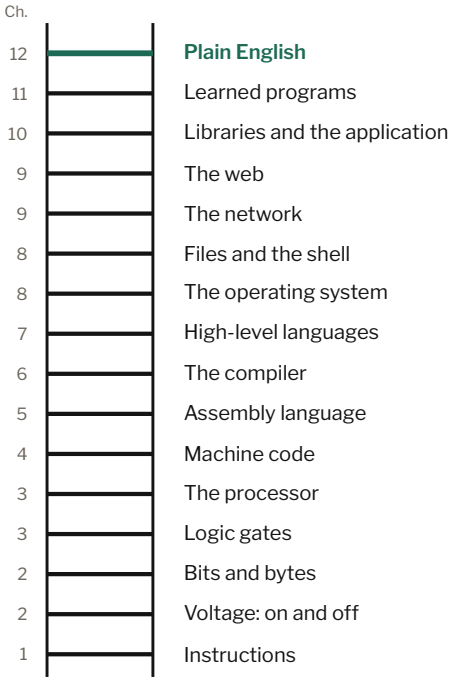
Where this leaves us

We now have the whole ladder. At the bottom, switches hold voltages that are high or low; bits name them, gates combine them, and a processor follows numbered instructions very fast. Assembly gave the numbers names, compilers let people write in something close to mathematics or English, and operating systems, networks and libraries let a few lines of code rest on millions written by others. Learned programs added a second kind of software, and on top of everything sits plain English, translated into code by a model that learned to write it from what people had written before.

Every layer was built by people to let other people say more with less, and every layer still works underneath the ones above. Each hides detail, and each leaks when something goes wrong. The newest layer is the most powerful and also the least exact, because it is the only one that guesses. The essential work Brooks described, deciding precisely what to build and checking that it was built, remains with the person who asks.

What this volume cannot give is practice. Knowing the layout of a city is not the same as finding your way through it with a deadline and a client waiting. Volume 2, *Good Enough to Build*, is the practical guide: how to take an idea to a working prototype with an AI coding agent, how to spec-

ify it, test it, keep it safe and put it in front of people. It assumes the picture this book has drawn, of a tower of translations from English down to voltage, and the habit of asking which layer a problem comes from.



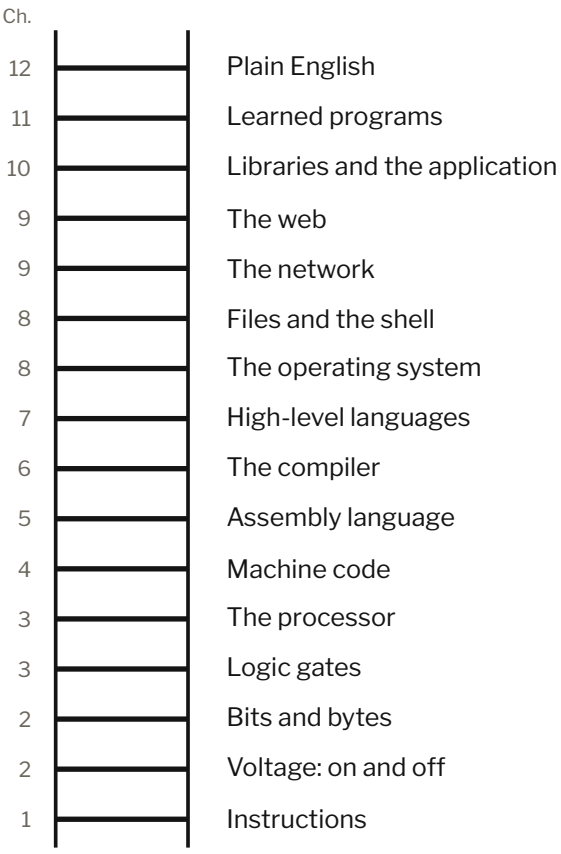
THE LADDER SO FAR

Each layer of software rests on the one below it and hides its detail. Rungs added in this chapter are marked in green; the dotted rungs are still to come. The numbers on the left are chapters.

BACK MATTER

The Whole Ladder

Every layer of software rests on the ones below it and hides their detail. Read from the bottom up, the ladder retells this book in sixteen steps. Read from the top down, it shows what lies beneath a sentence of English typed to an AI agent.



PART THREE

Rungs 11–15: the connected machine, learned programs and plain English (Chapters 9–12).

PART TWO

Rungs 5–10: from machine code to the operating system (Chapters 4–8).

PART ONE

Rungs 0–4: instructions, voltages, bits, gates and the processor (Chapters 1–3).

BACK MATTER

A Timeline

Dates are those of first publication, demonstration, release or public announcement. Chapter numbers show where each event is discussed.

1801 Joseph Marie Jacquard demonstrates in Paris a loom whose patterns are controlled by a chain of punched cards. Ch. 1

1834 Babbage begins to design the Analytical Engine, a general-purpose calculating machine driven by instructions on cards. It is never completed. Ch. 1

1843 Ada Lovelace publishes her translation of Menabrea's article on the Analytical Engine, with notes three times its length. Note G sets out a method for calculating Bernoulli numbers. Ch. 1, 12

1854 George Boole publishes *An Investigation of the Laws of Thought*, treating logic as an algebra of true and false. Ch. 2, 3

1890 Herman Hollerith's punched-card machines are used to tabulate the United States census. Ch. 1

1936 Alan Turing describes a universal machine that reads its rules from the same tape as its data. Ch. 1, 4

1937 Claude Shannon's master's thesis shows that circuits of switches can carry out Boole's logic. Ch. 3

1943 Warren McCulloch and Walter Pitts publish their model of the nerve cell as a logical switch. Ch. 4, 11

1945 ENIAC is running by the end of the year. On 30 June John von Neumann's *First Draft of a Report on the EDVAC* describes the stored-program computer. Ch. 4

1946 ENIAC is announced to the press on 14 February. Its six programmers set it up by plugging cables and setting switches. Ch. 4

1947 On 9 September a moth is taped into the Harvard Mark II logbook. In December John Bardeen and Walter Brattain, in William Shockley's group at Bell Labs, demonstrate the transistor. Ch. 2, 5

1948 Shannon's 'A Mathematical Theory of Communication' introduces the word *bit* in print. On 21 June the Manchester Baby runs the first program held in an electronic memory. Ch. 2, 4

1949 EDSAC at Cambridge runs its first program on 6 May. David Wheeler's initial orders let its users write in letters and decimal numbers rather than binary. Ch. 4, 5

1952 Grace Hopper describes her A-0 system, an early compiler for the UNIVAC, at a meeting in Pittsburgh in May. Ch. 6

1956 Werner Buchholz coins the word *byte* in June while helping to design IBM's Stretch computer. Ch. 2

1957 IBM delivers the first FORTRAN compiler, led by John Backus, to customers of the 704 in April. Ch. 6

1958 Jack Kilby builds several components on one piece of germanium. In the same year John McCarthy begins work on LISP, and Frank Rosenblatt first demonstrates the perceptron. Ch. 2, 6, 11

1959 Robert Noyce describes a way of making a silicon integrated circuit. In May about 40 people meet at the Pentagon to start the design of COBOL. Ch. 2, 6

1964 IBM announces System/360 in April. On 1 May the first BASIC programs run at Dartmouth College. Ch. 2, 5, 7

1965 Gordon Moore predicts in *Electronics* that the number of components on a chip will double every year. Ch. 2

1968 Dijkstra's letter against the `GO TO` statement appears in March. The NATO conference at Garmisch in October gives *software engineering* its name. Douglas Engelbart shows the mouse and windows in December. Ch. 7, 8

1969 ARPANET sends its first message on 29 October. Unix is begun at Bell Labs. The Apollo 11 landing on 20 July relies on software led by Margaret Hamilton's team. IBM begins to charge separately for software. Minsky and Papert publish *Perceptrons*. Ch. 1, 7, 8, 9, 11

1970 Edgar Codd proposes the relational model for storing data. Federico Faggin joins Intel to lead the design of the 4004. Ch. 3, 7

1971 Intel advertises the 4004 microprocessor on 15 November. Rosenblatt dies on 11 July. Ch. 2, 3, 11

1973 The core of Unix is rewritten in C. Doug McIlroy's pipe is added. The Xerox Alto runs for the first time. Ch. 7, 8

1975 Moore revises his forecast to a doubling every two years. Microsoft is founded; one of its first products is BASIC for the Altair 8800. Ch. 2, 7

1978 Kernighan and Ritchie publish *The C Programming Language*. Digital's VT100 terminal appears. Ch. 7, 8

1983 ARPANET switches to TCP/IP on 1 January. Richard Stallman announces the GNU project on 27 September. The Domain Name System is designed in November. Ch. 9, 10

1984 Apple introduces the Macintosh on 24 January. Ch. 8

1985 The first ARM chips work on 26 April. C++ is released commercially in October. Microsoft releases Windows 1.0 in November. Ch. 3, 7, 8

1986 Rumelhart, Hinton and Williams publish their account of backpropagation in *Nature*. Fred Brooks writes 'No Silver Bullet'. Ch. 11, 12

1989 Tim Berners-Lee writes his proposal for the web in March. Guido van Rossum starts Python in December. Ch. 7, 9

1991 Python is first released in February. Linus Torvalds announces Linux on 25 August. The first Unicode standard appears in October. Ch. 2, 7, 10

1993 CERN places the web's software in the public domain on 30 April. Ch. 9

1995 Java is announced in May. Brendan Eich writes the first JavaScript at Netscape. Ch. 7, 9

1998 The term *open source* is proposed at a meeting in Palo Alto on 3 February. Ch. 10

2005 Torvalds begins Git; its first commit is dated 7 April. Ch. 10

2006 Amazon opens its storage service S3 on 14 March and its computing service EC2 later in the year. Ch. 9

2008 GitHub opens to the public on 10 April. Stack Overflow is launched in September. Ch. 10, 12

2012 AlexNet, trained on gaming graphics processors, wins the ImageNet competition. Ch. 11

2017 Google researchers introduce the transformer. Andrej Karpathy publishes 'Software 2.0'. Ch. 11

2021 OpenAI describes Codex. GitHub Copilot is announced as a preview on 29 June. Log4Shell is made public on 9 December. Ch. 10, 11, 12

2022 OpenAI releases ChatGPT on 30 November. Ch. 12

2023 Karpathy writes on 24 January that 'the hottest new programming language is English'. Ch. 12

2024 A hidden back door is found in xz Utils on 29 March. Ch. 10

2025 In February Karpathy names *vibe coding*. Collins makes it Word of the Year in November. Ch. 12

2026 Unicode 18.0 is published in September. Margaret Hamilton dies on 30 September. Ch. 2, 7

BACK MATTER

Cast of Characters

The people who appear in this book, in alphabetical order by surname. Where a birth year is not widely documented, no dates are given. Hamilton died on 30 September 2026, shortly before this volume was finished.

Howard Aiken 1900–1973
American engineer who led the building of the Harvard Mark I and Mark II, relay calculators made for the US Navy. Grace Hopper worked on both. Ch. 6

Charles Babbage 1791–1871
English mathematician who designed the Difference Engine and the Analytical Engine, a general-purpose calculating machine driven by punched cards. The Analytical Engine was never completed. Ch. 1

John Backus 1924–2007
American computer scientist at IBM who led the team that built FORTRAN and its compiler, first delivered in 1957. Ch. 6

John Bardeen 1908–1991
American physicist at Bell Labs who, with Walter Brattain, demonstrated the transistor in December 1947. He later won a second Nobel Prize in Physics, for a theory of superconductivity. Ch. 2

Jean Bartik 1924–2011
One of the six original ENIAC programmers, born Betty Jean Jennings. With Betty Snyder she led the work on the trajectory program shown to the press in 1946. Ch. 4

Tim Berners-Lee born 1955
English computer scientist who proposed the World Wide Web at CERN in 1989 and wrote its first browser and server in 1990. He founded the World Wide Web Consortium in 1994. Ch. 9

George Boole 1815–1864
English mathematician whose book of 1854 treated logic as an algebra of true and false, the logic that Shannon later found in circuits of switches. Ch. 2, 3

Kathleen Booth 1922–2022
British mathematician who, with Andrew Booth, described in 1947 a compact symbolic notation for machine instructions, an early form of assembly language. Ch. 5

Walter Brattain 1902–1987
American physicist at Bell Labs who carried out the experiments with Bardeen that produced the transistor. He shared the 1956 Nobel Prize in Physics with Bardeen and Shockley. Ch. 2

Fred Brooks 1931–2022
American computer scientist who managed the development of IBM's System/360 software. He is remembered for Brooks's law and for the essay 'No Silver Bullet' (1986). Ch. 7, 12

Werner Buchholz 1922–2019
German-born engineer at IBM who coined the word *byte* in June 1956 while helping to design the Stretch computer. Ch. 2

Vinton Cerf born 1943
American computer scientist who, with Robert Kahn, designed the TCP/IP protocols that join networks into the Internet. They shared the 2004 Turing Award. Ch. 9

Edgar F. Codd 1923–2003
English-born mathematician at IBM who proposed in 1970 the relational way of storing data that underlies SQL databases. Ch. 7

Edsger Dijkstra 1930–2002
Dutch computer scientist who argued in 1968 against the `GO TO` statement and who doubted that programs could safely be written in natural language. Ch. 7, 12

J. Presper Eckert 1919–1995
American engineer who, with John Mauchly, designed ENIAC and began planning its stored-program successor. They went on to build UNIVAC. Ch. 4, 6

Brendan Eich born 1961
American programmer who wrote the first version of JavaScript at Netscape in about ten days in May 1995. Ch. 7, 9

Douglas Engelbart 1925–2013
American engineer whose team at the Stanford Research Institute demonstrated the mouse, windows and linked text on 9 December 1968. Ch. 8

Federico Faggin born 1941
Italian-born engineer who led the design of the Intel 4004 microprocessor, having joined Intel in April 1970. Ch. 3

Margaret Hamilton 1936–2026
American computer scientist who led the software team at MIT's Instrumentation Laboratory for the Apollo guidance computer, including the Apollo 11 landing in 1969. She was an early champion of the term *software engineering*. She died on 30 September 2026. Ch. 7

Mary Hawes
American programmer at the Burroughs Corporation who in April 1959 convened the group that began the design of COBOL. Ch. 6

Geoffrey Hinton born 1947
British-Canadian computer scientist. He co-wrote the 1986 paper on backpropagation and, with Krizhevsky and Sutskever, the 2012 AlexNet system. He shared the 2024 Nobel Prize in Physics. Ch. 11

Betty Holberton 1917–2001
One of the six original ENIAC programmers, born Frances Elizabeth Snyder. She later worked on the instructions for commercial machines and on early languages. Ch. 4

Herman Hollerith 1860–1929
American engineer who devised the punched-card tabulating machines used for the 1890 census. His company became part of IBM. Ch. 1

Grace Hopper 1906–1992
American mathematician and naval officer. She wrote the A-0 compiler for UNIVAC in 1951–52 and led the team behind FLOW-MATIC. She retired from the Navy in 1986 as a rear admiral. Ch. 5, 6

Joseph Marie Jacquard 1752–1834
French weaver and inventor whose loom, demonstrated in Paris in 1801, was controlled by a chain of punched cards. Ch. 1

Robert Kahn born 1938
American engineer who, with Vinton Cerf, designed TCP/IP and shared the 2004 Turing Award. Ch. 9

Andrej Karpathy born 1986
Slovak-Canadian computer scientist who wrote 'Software 2.0' (2017), observed in January 2023 that English had become a programming language, and named *vibe coding* in February 2025. Ch. 11, 12

Alan Kay born 1940
American computer scientist at Xerox PARC who coined the phrase *object-oriented programming* and worked on Smalltalk. Ch. 7

John Kemeny 1926–1992
Hungarian-born American mathematician who co-created BASIC with Thomas Kurtz and later became president of Dartmouth College. Ch. 7

Brian Kernighan born 1942
Canadian-American computer scientist at Bell Labs. He and Dennis Ritchie wrote *The C Programming Language* (1978). Ch. 7

Tom Kilburn 1921–2001
English engineer at the University of Manchester who, with F. C. Williams and Geoff Tootill, built the Baby. He wrote the first program that ran on it, on 21 June 1948. Ch. 4

Jack Kilby 1923–2005
American engineer at Texas Instruments who built several components on one piece of germanium in 1958, the first integrated circuit. Ch. 2

Kathy Kleiman
American lawyer and author who, from 1985, tracked down the six ENIAC programmers. She made the documentary *The Computers* (2014) and wrote *Proving Ground* (2022). Ch. 4

Leonard Kleinrock born 1934
American computer scientist whose laboratory at UCLA sent the first ARPANET message, on 29 October 1969. Ch. 9

Thomas Kurtz 1928–2024
American mathematician who, with John Kemeny, created BASIC at Dartmouth College in 1964 so that every student could use a computer. Ch. 7

Yann LeCun born 1960
French-American computer scientist whose neural networks at Bell Labs read many American cheques by the late 1990s. Ch. 11

Gottfried Wilhelm Leibniz 1646–1716
German mathematician and philosopher whose paper of 1703 made binary arithmetic widely known. Ch. 2

John Mauchly 1907–1980
American physicist who, with Presper Eckert, designed ENIAC and UNIVAC. Ch. 4

John McCarthy 1927–2011
American computer scientist at MIT who began LISP in 1958 and helped to organise the Dartmouth workshop on artificial intelligence. Ch. 6, 11

Warren McCulloch 1898–1969
American neurophysiologist who chaired the Macy conferences on cybernetics. With Walter Pitts he published the 1943 model of the neuron as a logical switch. Ch. 4, 11

Doug McIlroy born 1932
American computer scientist who managed the Unix group at Bell Labs and suggested the pipe, added in 1973. Ch. 8

Marvin Minsky 1927–2016
American computer scientist at MIT. With Seymour Papert he published *Perceptrons* (1969), which set out the limits of single-layer networks. Ch. 11

Gordon Moore 1929–2023
American engineer, a founder of Fairchild Semiconductor and later of Intel. His observation of 1965 on the doubling of components on a chip became known as Moore's Law. Ch. 2

Robert Noyce 1927–1990
American engineer at Fairchild Semiconductor who in 1959 described a practical silicon integrated circuit. He later co-founded Intel. Ch. 2

Seymour Papert 1928–2016
South African-born mathematician at MIT, co-author with Minsky of *Perceptrons*. Ch. 11

Walter Pitts 1923–1969
American logician who, as a young man, co-wrote the 1943 paper on neurons with Warren McCulloch. Ch. 11

Dennis Ritchie 1941–2011
American computer scientist at Bell Labs who created the C language and, with Ken Thompson, Unix. Ch. 7, 8

Frank Rosenblatt 1928–1971
American psychologist at the Cornell Aeronautical Laboratory who designed the perceptron. He died in a sailing accident on his 43rd birthday. Ch. 11

David Rumelhart 1942–2011
American psychologist who, with Hinton and Ronald Williams, published the 1986 paper that made backpropagation widely known. Ch. 11

Jean Sammet 1928–2017
American computer scientist at Sylvania who served on the committee that designed COBOL in 1959. Ch. 6

Claude Shannon 1916–2001
American mathematician and engineer. His 1937 thesis linked switching circuits to logic, and his 1948 paper founded information theory and introduced the word *bit* in print. Ch. 2, 3, 11

William Shockley 1910–1989
American physicist who led the Bell Labs group in which the transistor was invented. He shared the 1956 Nobel Prize in Physics. Ch. 2

Joel Spolsky born 1965
American software developer who wrote 'The Law of Leaky Abstractions' (2002) and co-founded Stack Overflow with Jeff Atwood in 2008. Ch. 12

Richard Stallman born 1953
American programmer who announced the GNU project on 27 September 1983 and wrote the GNU General Public License. Ch. 10

Bjarne Stroustrup born 1950
Danish computer scientist at Bell Labs who began adding classes to C in 1979, producing C++. Ch. 7

Ken Thompson born 1943
American computer scientist at Bell Labs who began Unix in 1969 with Dennis Ritchie and wrote the first Unix shell. He also designed UTF-8 with Rob Pike. Ch. 2, 7, 8

Linus Torvalds born 1969
Finnish-American programmer who started the Linux kernel in 1991 and the version-control system Git in 2005. Ch. 10

Alan Turing 1912–1954
English mathematician. He described the universal machine in 1936, designed the ACE in 1945, and in 1948 wrote 'Intelligent Machinery', which sketched machines that learn. Ch. 1, 4, 11, 12

Guido van Rossum born 1956
Dutch programmer who began Python as a holiday project in December 1989 and led its development for almost three decades. Ch. 7

John von Neumann 1903–1957
Hungarian-born mathematician whose *First Draft of a Report on the EDVAC* (1945) first described the stored-program computer. Ch. 4

David Wheeler 1927–2004
English computer scientist at Cambridge who wrote EDSAC's initial orders, the first assembler-like program, and invented the subroutine library. Ch. 5

Norbert Wiener 1894–1964
American mathematician who named the field of cybernetics in his book of 1948. Ch. 11

Maurice Wilkes 1913–2010
English computer scientist who led the building of EDSAC at Cambridge, which ran its first program on 6 May 1949. Ch. 4, 5

Sophie Wilson born 1957
British computer scientist who with Steve Furber designed the first ARM processor at Acorn Computers, whose chips first worked on 26 April 1985. Ch. 3

BACK MATTER

Glossary

Key terms as they are defined in the margins of the book, with the chapter where each first appears.

A

Abstraction Hiding the details of how something works behind a simpler way of using it, so that people can work at a higher level. Ch. 1

Accumulator A working register in the processor in which a result is built up, one operation at a time. Ch. 4

Address The number that identifies one cell of memory, so that the processor can find what is stored there. Ch. 3

Algorithm A precise, step-by-step method for solving a problem, written so that it can be followed without judgement. Ch. 1

API Application Programming Interface: a published set of requests one program can make of another, and the answers it will give. Ch. 9

ARPANET The network funded by the US Defense Department's ARPA from 1969; the forerunner of the Internet. Ch. 9

Artificial neurons Simple units, loosely modelled on nerve cells, that combine their inputs and produce an output. Networks of them compute. Ch. 11

ASCII An early standard table (1963) that gives each English letter, digit and punctuation mark a number from 0 to 127. Ch. 2

Assembler A program that translates assembly language into machine code, usually one line into one instruction. Ch. 5

Assembly language A way of writing machine instructions with short names instead of numbers. Each line stands for one instruction. Ch. 5

B

Backend The part of an app that runs on a server: it applies the rules, holds the secrets and talks to the database. Ch. 10

Backpropagation A method for training a network with many layers: the error at the output is passed backwards to work out how each weight should change. Ch. 11

Binary A way of writing numbers using only two digits, 0 and 1, in which each column is worth twice the column to its right. Ch. 2

Bit A single binary digit, 0 or 1: the smallest unit of information a computer stores. Ch. 2

Bug A mistake in a program that makes it behave differently from what its author intended. Ch. 5

Byte A group of eight bits. It can hold any of 256 patterns: a number from 0 to 255, or one letter of English text. Ch. 2

C

Client A program that sends a request over a network and waits for the answer; a browser is a client. Ch. 9

Clock A circuit that sends out a steady stream of pulses, so that all parts of the processor change step at the same moment. Ch. 3

Cloud Computers, storage and services rented over the Internet from a provider who owns and runs the machines. Ch. 9

Coding agent A program, driven by a language model, that reads a project's files, writes code, runs commands and checks the results in a loop. Ch. 12

Commit A saved snapshot of a project's files at one moment, with a short note describing what changed. Ch. 10

Compiler A program that translates a whole program from a programming language into machine code before it runs. Ch. 6

D

Database An organised store of data, managed by a program that lets people and other programs add, change and ask for it. Ch. 7

Debugging Finding and removing the mistakes in a program, usually by observing what it actually does and comparing that with what was intended. Ch. 5

Dependencies Outside packages that a project needs in order to build or run. Each may have dependencies of its own. Ch. 10

Digital Representing information by a small number of distinct states (in computers, two) instead of a smoothly varying quantity. Ch. 2

DNS The Internet's address book: a worldwide system that turns names such as example.com into IP addresses. Ch. 9

F

File A named collection of bytes stored on a disk, which the operating system keeps track of and can find again. Ch. 8

Framework A large library that provides the overall structure of a program; the programmer fills in the parts specific to their app. Ch. 10

Frontend The part of an app that runs on the user's phone or in their browser: the screens, buttons and forms. Ch. 10

G

General-purpose computer A machine that can carry out any calculation, given the right program, rather than one fixed task. Ch. 1

Graphical user interface A way of using a computer by pointing at and touching pictures on a screen (windows, icons, menus) instead of typing commands. Ch. 8

H

Half adder A circuit of two gates that adds two single bits, giving a sum bit and a carry bit. Ch. 3

High-level language A programming language that lets people write in terms of their problem, such as formulas or words, rather than the machine's steps. Ch. 6

I

Instruction set The full list of operations a processor can carry out. It is the only vocabulary the processor understands. Ch. 3

Integrated circuit Many transistors, and the wires between them, made together on one small slice of silicon. Also called a chip. Ch. 2

Interpreter A program that reads another program and carries out its statements directly, one at a time, each time it runs. Ch. 6

IP address A number that identifies a computer on the Internet, so that packets can be delivered to it. Ch. 9

J

JSON JavaScript Object Notation: a plain-text format for data, made of named values in braces and lists in square brackets. Ch. 9

L

Language model A model trained to predict the next word (or piece of a word) in a text. Large ones are trained on vast amounts of writing. Ch. 11

Leaky abstraction A layer that hides the detail beneath it until something goes wrong, when that detail shows through. Ch. 12

Library A collection of ready-made, tested pieces of program that other programmers can use in their own work. Ch. 5

Logic gate A small circuit, built from a few transistors, that takes one or more bits in and gives one bit out by a fixed rule. Ch. 3

M

Machine code Instructions written as the numbers that a processor reads and carries out directly; the lowest layer of software. Ch. 4

Mnemonic A short, memorable name for a machine instruction, such as ADD or LOAD, used in place of its number. Ch. 5

Model A trained network: its fixed structure plus the learned weights. It is run like a program but was not written as one. Ch. 11

O

Objects Bundles of data together with the actions that can be performed on it, handled as one unit in a program. Ch. 7

Open source Software whose code is published under a licence that lets anyone read, use, change and share it. Ch. 10

Operating system The program that runs all the time and manages the processor, memory, files and devices on behalf of other programs. Ch. 8

Operation code The part of an instruction that says which operation to perform, such as add or store; often shortened to opcode. Ch. 4

Optimising compiler A compiler that rearranges and simplifies a program as it translates it, so that the machine code runs faster. Ch. 6

P

Package manager A program that downloads the libraries a project depends on, and the libraries those depend on, in the right versions. Ch. 10

Packet A small piece of a message, sent on its own with its destination address and its place in the sequence. Ch. 9

Path The address of a file: the folders you pass through to reach it, written in order and separated by slashes. Ch. 8

Pixels The tiny squares, each a single colour, that make up a digital image. Short for “picture elements”. Ch. 2

Portable Able to run on different kinds of computer with little or no change to its source code. Ch. 6

Process A running program, with the memory and other resources the operating system has given it. Ch. 8

Processor The part of a computer that carries out instructions: it fetches each one from memory, works out what it means and does it. Ch. 3

Program A set of instructions, written in advance, that tells a machine exactly what to do. Ch. 1

Protocol An agreed set of rules for how two computers format and exchange messages. Ch. 9

Punched cards Stiff cards in which the presence or absence of holes at set positions records information or instructions. Ch. 1

Q

Query A request for data, written in a language such as SQL, that describes what is wanted rather than how to find it. Ch. 7

R

Register A small, very fast storage place inside the processor that holds one number while it is being worked on. Ch. 3

Router A computer in a network that receives packets and passes each one on towards its destination. Ch. 9

S

Server A program, or the computer it runs on, that waits for requests from clients and sends back answers. Ch. 9

Shell The program that reads the commands typed into a terminal, starts the programs they name and shows the results. Ch. 8

Software engineering The application of disciplined, measurable methods to the design, building, testing and maintenance of software. Ch. 7

Source code The text of a program as a person writes it, in a programming language, before it is translated into machine code. Ch. 6

Specification A clear statement, written in advance, of what a program must do, including the unusual cases, and how to tell whether it does. Ch. 12

Stored-program computer A computer that keeps its instructions in memory as numbers, alongside the data they work on. Ch. 4

Structured programming Building programs only from blocks of steps, loops and choices, each with

one way in and one way out, instead of free jumps. Ch. 7

Subroutine A self-contained piece of program that does one job and can be used from anywhere in a larger program. Ch. 5

Syntax The rules of spelling, punctuation and word order that a program must follow for a language's translator to accept it. Ch. 7

T

Terminal A device, or today a window on the screen, through which a person types commands to a computer and reads its replies. Ch. 8

Training Adjusting a learning system's weights by showing it examples and correcting its errors, over and over. Ch. 11

Transistors Tiny electronic switches with no moving parts. A small voltage on one connection turns the flow of current between the other two on or off. Ch. 2

Truth table A list of every possible combination of inputs to a logic circuit, with the output each one produces. Ch. 3

U

Unicode A single international table that gives every character in the world's writing systems, and symbols such as emoji, its own number. Ch. 2

URL Uniform Resource Locator: the address of a page or file on the web, such as https://example.com/books. Ch. 9

V

Version control A system that records every change to a project's files, who made it and why, so that any earlier state can be recovered. Ch. 10

Vibe coding Building software by describing it to an AI and accepting the code it writes without reading it. Ch. 12

Virtual machine A program that imitates a computer, so that code written for the imaginary machine can run on any real one that has the imitator. Ch. 7

Von Neumann architecture The standard layout of a computer: one memory holding both program and data, and a processor that fetches instructions from it. Ch. 4

W

Weight A number that sets how strongly one input counts towards a unit's output. Learning means changing the weights. Ch. 11

BACK MATTER

Further Reading

Nothing here is compulsory. Each chapter has a short list of books, papers, articles and videos. Where an item is the original source it is usually more readable than people fear. If you read one thing per chapter, read the one marked ■. Items marked *advanced* go beyond the level of this book. Websites and videos move; if a link has gone, searching for the title will usually find it.

Chapter 1. What Is a Program?

■ **Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*** BOOK Builds a computer up from flashlights and switches in plain steps. It suits the whole first half of this book.

Ada Lovelace, 'Notes' to Menabrea's *Sketch of the Analytical Engine* (1843) PRIMARY SOURCE Note G contains the table for Bernoulli numbers, and the remark that the engine 'has no pretensions whatever to originate anything'.

Stephen Wolfram, 'Untangling the Tale of Ada Lovelace' (2015) ESSAY A careful look at what Lovelace did and did not contribute, with her own documents.

Doron Swade, *The Cogwheel Brain* (2000) BOOK The story of Babbage's engines, and of the team that built his Difference Engine No. 2 at the Science Museum.

Walter Isaacson, *The Innovators* (2014) BOOK The first chapters cover Lovelace, Babbage and Hollerith in a readable popular history.

Alan Turing, 'On Computable Numbers' (1936) PAPER The universal machine. *Advanced*, though the early pages explain the idea in words.

Chapter 2. From Voltage to Bits

■ **Charles Petzold, *Code*** BOOK The chapters on binary numbers, bytes and ASCII are the clearest introduction for beginners.

Joel Spolsky, 'The Absolute Minimum Every Software Developer Absolutely, Positively Must Know About Unicode and Character Sets' (2003) ESSAY Short and funny, and explains why text sometimes arrives as garbage.

Gordon Moore, 'Cramming More Components onto Integrated Circuits' (1965) PAPER The original article behind Moore's Law, four pages long.

Chris Miller, *Chip War* (2022) BOOK How chips came to be a matter of national strategy, and why so few places make the best ones.

James Gleick, *The Information* (2011) BOOK How the bit changed what people mean by information.

Computerphile, 'Characters, Symbols and the Unicode Miracle' VIDEO Tom Scott explains how Unicode and UTF-8 fit together.

Chapter 3. Gates and the Processor

- **Crash Course Computer Science (2017), episodes on logic gates, the ALU and the CPU** VIDEO Short animated episodes that cover the same ground as this chapter, at a gentle pace.

Claude Shannon, 'A Symbolic Analysis of Relay and Switching Circuits' (1938) PAPER The thesis that linked logic to circuits. The opening is readable by anyone.

Jimmy Soni and Rob Goodman, *A Mind at Play* (2017) BOOK A biography of Shannon, with his juggling and his homemade machines.

Ben Eater, 'Building an 8-bit breadboard computer!' VIDEO A processor built from simple chips on a breadboard, with every wire explained.

Noam Nisan and Shimon Schocken, *The Elements of Computing Systems* (2005) BOOK Builds a computer from logic gates upward as a series of exercises. *Advanced* for non-programmers; the first chapters are approachable.

Chris Miller, *Chip War* BOOK The business of making processors, from Fairchild to TSMC.

Chapter 4. From Rewiring to Remembering

- **Kathy Kleiman, *Proving Ground* (2022)** BOOK The story of the six women who programmed ENIAC and of their long wait for recognition.

  ***The Computers: The Remarkable Story of the ENIAC Programmers* (2014)** FILM Kleiman's documentary, with interviews with several of the six.

  Jean Jennings Bartik, *Pioneer Programmer* (2013) BOOK A memoir by one of the six.

  John von Neumann, *First Draft of a Report on the EDVAC* (1945) PRIMARY SOURCE The document that described the stored-program computer. *Advanced*, but short.

  George Dyson, *Turing's Cathedral* (2012) BOOK The Institute for Advanced Study and the early machines built after von Neumann's report.

  Maurice Wilkes, *Memoirs of a Computer Pioneer* (1985) BOOK Wilkes on building EDSAC and on discovering how much of life would go on finding mistakes in programs.

Chapter 5. Assembly and the First Translators

- **Maurice Wilkes, *Memoirs of a Computer Pioneer* (1985)** BOOK The chapters on EDSAC tell, from the inside, how the first programs and the first subroutine library came about.

  Maurice Wilkes, David Wheeler and Stanley Gill, *The Preparation of Programs for an Electronic Digital Computer* (1951) BOOK The first textbook of programming. *Advanced* as reading, but historic.

  Compiler Explorer (godbolt.org) TOOL A free website by Matt Godbolt. Type a few lines of C and see the assembly language beside them.

  Ben Eater, 'Hello, world from scratch on a 6502 system' VIDEO Programs written in machine code and assembly on a small real chip.

  The 1947 Harvard Mark II moth, National Museum of American History MUSEUM OBJECT The Smithsonian's catalogue page and logbook photograph.

  Charles Petzold, *Code* BOOK The chapters on assembly language and on how a processor runs a program.

Chapter 6. Compilers

- **Kurt Beyer, *Grace Hopper and the Invention of the Information Age* (2009)** BOOK A full biography, with the story of the A-0 compiler and of FLOW-MATIC.

  Grace Hopper, 'The Education of a Computer' (1952) PAPER Her own account of the A-0 system.

  John Backus, 'The History of FORTRAN I, II and III' (1978) PAPER Backus on the team's work and the doubts that surrounded it.

  Jean Sammet, 'The Early History of COBOL' (1978) PAPER By a member of the committee. Good on how a language gets designed by many hands.

  Robert Nystrom, *Crafting Interpreters* (2021) BOOK Free online. Builds two interpreters step by step. *Advanced*.

  Compiler Explorer (godbolt.org) TOOL Shows what several compilers produce from the same source code.

Chapter 7. A Family Tree of Languages

- **Brian Kernighan, *UNIX: A History and a Memoir* (2019)** BOOK A short, warm account by someone who was there, covering Unix and the birth of C.

  Frederick Brooks, *The Mythical Man-Month* (1975; anniversary edition 1995) BOOK Essays on why large software projects run late, with the origin of Brooks's law.

  Dennis Ritchie, 'The Development of the C Language' (1993) PAPER How C grew out of B and Unix, in Ritchie's own words.

  Edsger Dijkstra, 'Go To Statement Considered Harmful' (1968) PAPER Two pages, still much argued about.

  David Mindell, *Digital Apollo* (2008) BOOK The flight software of the Moon landings, including the alarms of Apollo 11.

  Peter Naur and Brian Randell (eds), *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee* (1968) REPORT Free online. The discussion at Garmisch reads like a conversation.

Chapter 8. The Operating System, Files and the Terminal

- **Neal Stephenson, *In the Beginning…Was the Command Line* (1999)** ESSAY A lively and opinionated essay on typing versus clicking and on what an operating system does for you.

  The Missing Semester of Your CS Education (MIT, 2020) COURSE Free lectures on the shell, files and the tools programmers use daily. Start with the first two.

  William Shotts, *The Linux Command Line* BOOK Free to read online. A patient introduction to the terminal.

  Douglas Engelbart, 'The Mother of All Demos' (1968) VIDEO The recording of the 90-minute demonstration, available online. Watch the first fifteen minutes.

  Michael Hiltzik, *Dealers of Lightning* (1999) BOOK The story of Xerox PARC, the Alto and the people who built them.

  Brian Kernighan, *UNIX: A History and a Memoir* (2019) BOOK The pipe, the shell and the Unix philosophy of small tools.

Chapter 9. Networks, the Web and the Cloud

- **Katie Hafner and Matthew Lyon, *Where Wizards Stay Up Late* (1996)** BOOK The origins of the ARPANET, from the people who built it.

  Tim Berners-Lee, *Weaving the Web* (1999) BOOK The inventor's own account of the web's beginnings at CERN.

  Tim Berners-Lee, 'Information Management: A Proposal' (1989) PRIMARY SOURCE Free from CERN's archive. Notice the manager's hand-written comment on it, 'Vague but exciting'.

  Andrew Blum, *Tubes: A Journey to the Centre of the Internet* (2012) BOOK A visit to the cables and buildings that carry the network.

  Mozilla, 'How the Web works' (MDN Web Docs) TUTORIAL A free, plain explanation of browsers, servers, DNS and HTTP.

  Vinton Cerf and Robert Kahn, 'A Protocol for Packet Network Intercommunication' (1974) PAPER The original design of TCP/IP. *Advanced*.

Chapter 10. Anatomy of a Modern App

- **Nadia Eghbal, *Roads and Bridges: The Unseen Labor Behind Our Digital Infrastructure* (2016)** REPORT Free from the Ford Foundation. Why so much of the world runs on code kept up by volunteers.

  Nadia Eghbal, *Working in Public* (2020) BOOK What maintaining an open-source project is like.

  Eric Raymond, 'The Cathedral and the Bazaar' (1997) ESSAY The argument for developing software in the open.

  Russ Cox, 'Surviving Software Dependencies' (2019) ARTICLE Why depending on other people's packages is both necessary and risky.

  Russ Cox, 'Timeline of the xz open source attack' (2024) ARTICLE A day-by-day account of the back door described in this chapter.

  Scott Chacon and Ben Straub, *Pro Git* BOOK Free online. The first two chapters explain what version control is for.

  Randall Munroe, xkcd 2347, 'Dependency' CARTOON The cartoon quoted in the chapter.

Chapter 11. Programs That Learn

- **Vipul Mathur, *From Switches to Souls*** BOOK The sibling to this book, from the same author and course. It follows neural networks, language models and prompting at greater length.

  3Blue1Brown, 'But what is a neural network?' (2017) VIDEO A clear animated introduction, the first of a series.

  Andrej Karpathy, 'Software 2.0' (2017) ESSAY The short essay quoted in the chapter.

  Melanie Mitchell, *Artificial Intelligence: A Guide for Thinking Humans* (2019) BOOK A balanced account of what these systems can and cannot do.

  Cade Metz, *Genius Makers* (2021) BOOK The people behind deep learning, from Hinton's group onward.

  Frank Rosenblatt, 'The Perceptron' (1958) PAPER The original description. *Advanced*.

  Mark Chen and others, 'Evaluating Large Language Models Trained on Code' (2021) PAPER The Codex paper. *Advanced*.

Chapter 12. English Becomes Code

- **Vipul Mathur, *Good Enough to Build* (Volume 2 of this series)** BOOK The practical guide: taking an idea to a working, shipped prototype with an AI coding agent.

  Frederick Brooks, 'No Silver Bullet' (1986) ESSAY On essential and accidental difficulty in software. Reprinted in the anniversary edition of *The Mythical Man-Month*.

  Joel Spolsky, 'The Law of Leaky Abstractions' (2002) ESSAY Why every layer eventually shows what lies beneath it.

  Edsger Dijkstra, 'On the foolishness of "natural language programming"' (EWD 667) ESSAY The argument for formal notation, free online.

  Andrej Karpathy, 'Intro to Large Language Models' (2023) VIDEO A one-hour talk for non-specialists.

  METR, 'Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity' (2025) STUDY The randomised study mentioned in the chapter, which found experienced developers slower with AI tools.

  Simon Willison, 'Not all AI-assisted programming is vibe coding (but vibe coding rocks)' (2025) ARTICLE A careful distinction between reviewing AI-written code and accepting it unread.

BACK MATTER

Picture Credits

All diagrams and charts were drawn for this book. Photographs are reproduced under the licences shown; all come from Wikimedia Commons or equivalent free-licensed sources, and have been converted to greyscale for printing. 'CC' licences are Creative Commons licences, whose terms are published at creativecommons.org. 'Public domain' means that no copyright restricts the use of the image.

CHAPTER 1

Jacquard loom (opening). Stephencdickson / Wikimedia Commons; CC BY-SA 4.0

Analytical Engine, trial portion. Charles Babbage, photo by Science Museum, London / Wikimedia Commons; CC BY-SA 2.0

Ada Lovelace. Alfred Edward Chalon / Wikimedia Commons; Public domain

80-column punched card. Harke / Wikimedia Commons; CC BY-SA 3.0

CHAPTER 2

MOS 6502 die (opening). John McMaster / Wikimedia Commons; CC BY 4.0

First transistor, replica. Lucent Technologies (Federal employee) / Wikimedia Commons; Public domain

CHAPTER 3

RP2350 microcontroller (opening). Profpcde / Wikimedia Commons; CC0

Claude Shannon. Unknown, Tekniska Museet, Sweden / Wikimedia Commons; CC BY 2.0

Relays in a Flexowriter. Douglas W. Jones / Wikimedia Commons; CC0

CHAPTER 4

ENIAC programmers (opening). U.S. Army photographer / Wikimedia Commons; Public domain

ENIAC. U.S. Army photographer / Wikimedia Commons; Public domain

John von Neumann. Los Alamos National Laboratory / Wikimedia Commons; Public domain

Manchester Baby, replica. Paul Hermans / Wikimedia Commons; CC BY-SA 4.0

Ferrite core memory. Orion 8 / Wikimedia Commons; CC BY 2.5

CHAPTER 5

EDSAC, replica (opening). Geni / Wikimedia Commons; CC BY-SA 4.0

Harvard Mark II logbook, 9 September 1947. U.S. Naval Surface Warfare Center / Wikimedia Commons; Public domain

CHAPTER 6

Grace Hopper (opening). James S. Davis, U.S. Navy / Wikimedia Commons; Public domain

CHAPTER 7

DEC PDP-11/40 (opening). Bobo11 / Wikimedia Commons; CC BY-SA 3.0

Margaret Hamilton. Draper Laboratory/MIT / Wikimedia Commons; Public domain

Guido van Rossum. Alessio Bragadini / Wikimedia Commons; CC BY-SA 2.0

CHAPTER 8

Teletype Model 33 ASR (opening). Arnold Reinhold / Wikimedia Commons; CC BY-SA 3.0

DEC VT100 terminal. Jason Scott / Wikimedia Commons; CC BY 2.0

Xerox Alto. The wub / Wikimedia Commons; CC BY-SA 4.0

CHAPTER 9

First web server (NeXT) (opening). Coolcaesar (en.wikipedia) / Wikimedia Commons; CC BY-SA 3.0

Tim Berners-Lee. Sam Barnes/Sportsfile / Wikimedia Commons; CC BY 4.0

CERN data centre. Hugovanmeijeren / Wikimedia Commons; CC BY-SA 3.0

CHAPTER 10

NOIRLab server racks (opening). NOIRLab/NSF/AURA/T. Slovinsky / Wikimedia Commons; CC BY 4.0

Linus Torvalds (portrait drawing). JericoDelayah / Wikimedia Commons; CC BY-SA 3.0

CHAPTER 11

Mark I Perceptron (opening). National Museum of the U.S. Navy / Wikimedia Commons; Public domain

Walter Pitts and Jerome Lettvin. Iapx86 / Wikimedia Commons; CC BY-SA 3.0

Frank Rosenblatt. Anonymous / Wikimedia Commons; CC BY-SA 4.0

Geoffrey Hinton. John Sears / Wikimedia Commons; CC BY-SA 4.0

CHAPTER 12

Franklin typewriter (opening). Hannes Grobe / Wikimedia Commons; CC BY-SA 4.0

Source code on a screen. Slashme / Wikimedia Commons; CC0